

Line Coffee V3

Independent Master Audit

Read-only codebase audit of **muhmiied/line-coffee-fainal**.

Branch: main · Last commit: 069a460

Audit date: 2026-07-13

10 parts	A → J covering public site, admin, data flow, security, SEO, performance, accessibility, architecture
24 findings	Plotted on severity × launch-blocker matrix (see Figure 3)
6 diagrams	Architecture, order lifecycle, risk matrix, data flow, disconnects, roadmap
1 P0 bug	Cart vs checkout delivery fee mismatch — customer sees different totals
3 P1 disconnects	Espresso, flavor, and legal admin edits don't reach the public site
2 launch blockers	SMTP not configured + confirm which migrations are applied

Visual Summary

Six diagrams summarize the key findings of this audit. Each is also embedded in its relevant section.

Line Coffee V3 — Architectural Overview

Browser-only Supabase app. Trust boundary is the database, not the client.

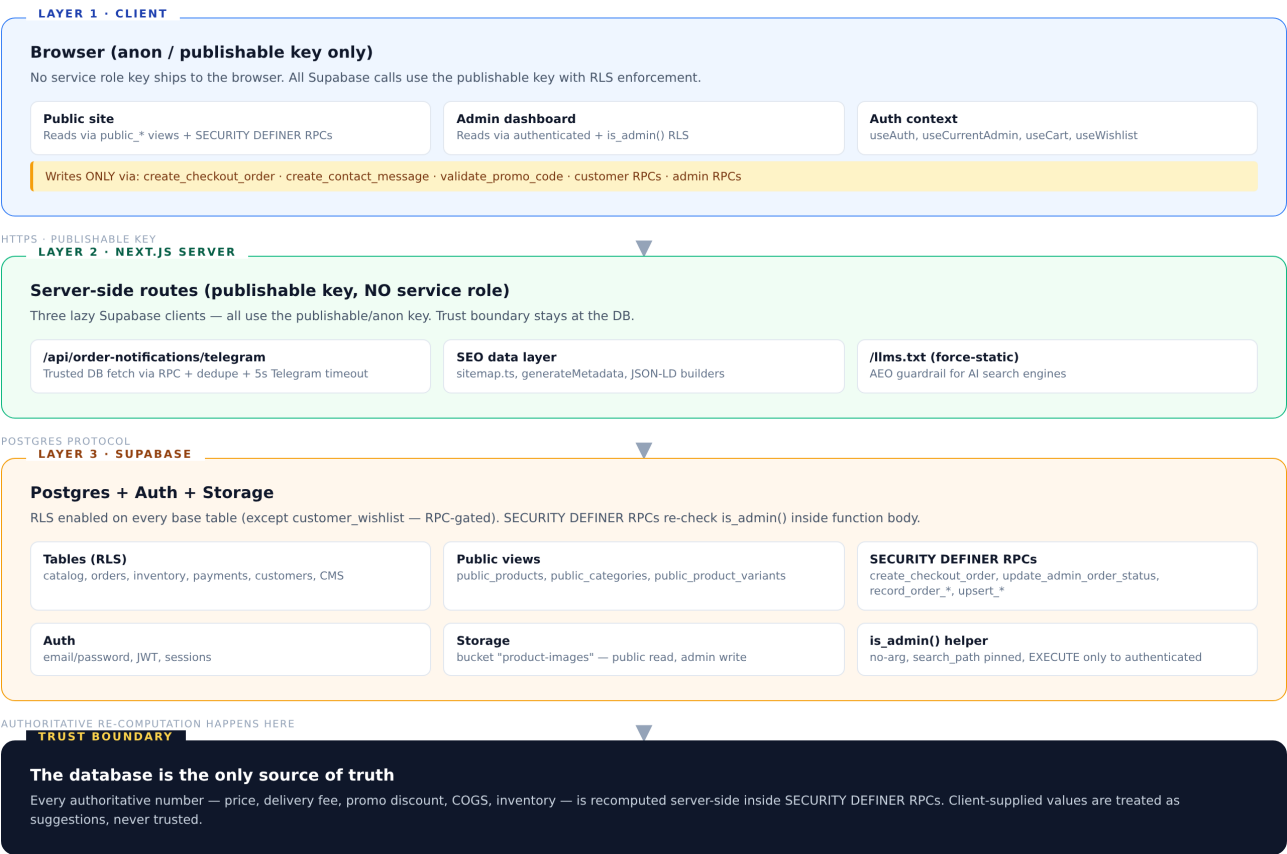
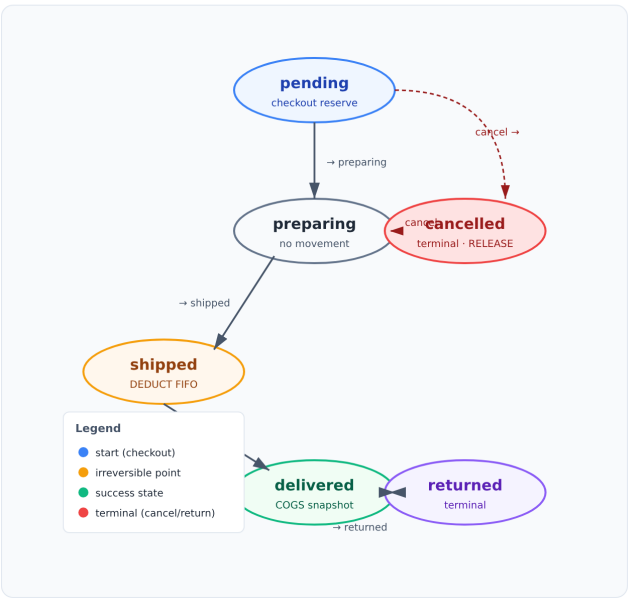


Figure 1 — Architectural overview (see §C.0)

Order Lifecycle — Status Transitions & Inventory Effects

Atomic RPC: update_admin_order_status (SECURITY DEFINER, SELECT ... FOR UPDATE on order row)

STATE DIAGRAM



TRANSITION EFFECTS

TRANSITION	INVENTORY EFFECT
checkout → pending	RESERVE available_kg -= qty, reserved_kg += qty, order_lot_allocations status=reserved
pending → preparing	NONE no movement
pending → cancelled	RELEASE available_kg += reservation, reserved_kg -= reservation
preparing → shipped	DEDUCT reserved_kg -= reservation, order_lot_allocations.status=deducted. COGS-irreversible point.
preparing → cancelled	RELEASE same as pending → cancelled
shipped → delivered	SNAPSHOT orders.cogs_total + order_items.line_cogs finalized
delivered → returned	VIA record_order_return sellable condition restocks FIFO lots; damaged/other never restock. Does NOT refund money.
cancelled → [terminal]	no further transitions allowed
returned → [terminal]	no further transitions allowed

Pre-condition: every stock-tracked product line must have an open reservation (sum of reserve minus release/deduct > 0). If not, RPC raises Inventory reservation is missing or already consumed. and aborts — no status change.

Figure 2 – Order lifecycle (see §C.3)

Line Coffee V3 — Risk Severity Matrix (24 findings)



Figure 3 – Risk severity matrix (see §E.13)

Data Flow — Admin → Supabase → Public

Where admin edits propagate (green) vs where they hit a disconnect (red)

ADMIN ACTION	→	SUPABASE	→	PUBLIC SITE
Add/edit product ProductDrawer → create_admin_product	SECURITY DEFINER RPC →	products + product_variants	public_products view →	/products + /products/[slug] getPublicProducts()
Upload product image Media tab → uploadProductImage	Storage upload + DB write →	product-images bucket + products.gallery	getPublicUrl →	ProductCard images public read
Create/edit category CategoryDrawer → updateAdminCategory	direct INSERT/UPDATE →	categories + trigger sync_slug	public_categories view →	/products sidebar getPublicCategories()
Toggle best seller ProductDrawer → updateAdminProduct	column update →	products.best_seller boolean flag	public_products view →	Home BestSellersSection getPublicBestSellers()
Create promo code PromoCodesPanel → upsert_promo_code	SECURITY DEFINER RPC →	promo_codes + promo_redemptions	validate_promo_code RPC →	/checkout PromoSection applied at checkout
Publish blog post CMS Blog → save_admin_blog_post	SECURITY DEFINER RPC →	blog_posts status=published	listPublishedBlogPosts →	/blog + /blog/[slug] public-blog.ts
Approve review CMS Reviews → save_admin_review	SECURITY DEFINER RPC →	reviews status=approved	listApprovedHomepageReviews →	Home TestimonialsSection public-cms.ts
Create announcement AnnouncementsPanel → direct INSERT	RLS-gated write →	announcements active=true	getPublicAnnouncements →	PublicHeader top bar + DEFAULT fallback
Edit site settings Settings → saveAdminSettings	upsert site_settings →	site_settings scope=public	getPublicSettings →	Header / Footer / Checkout phone, social, brand
Edit espresso bean Espresso Manager → upsert_espresso_bean	SECURITY DEFINER RPC →	espresso_beans persisted ✓	NO LIVE READ ✗	EspressoBlendStudio reads espressoBeans.ts STATIC ⚠️ DISCONNECT
Edit flavor item Flavor Manager → upsert_flavor_item	SECURITY DEFINER RPC →	flavor_items persisted ✓	NO LIVE READ ✗	FlavorMixStudio reads flavorData.ts STATIC ⚠️ DISCONNECT
Edit legal page CMS Legal → save_admin_legal_page	SECURITY DEFINER RPC →	legal_pages persisted ✓	NOT CONSUMED ✗	/privacy /terms /shipping /returns reads STATIC arrays ⚠️ DISCONNECT

DATA FLOW SCORECARD

9 flows connected 3 critical disconnects (P1) Admin edits persist but never reach the public site

Fix paths: (a) refactor public reads to fetch from DB at runtime, OR (b) add a sync script that regenerates the .ts files / page arrays from the DB after admin edits.

Figure 4 – Data flow Admin → Public (see §C.9)

Critical Disconnects — P1 Issues

Admin edits persist to Supabase but the public site reads static files/arrays. 3 confirmed cases.

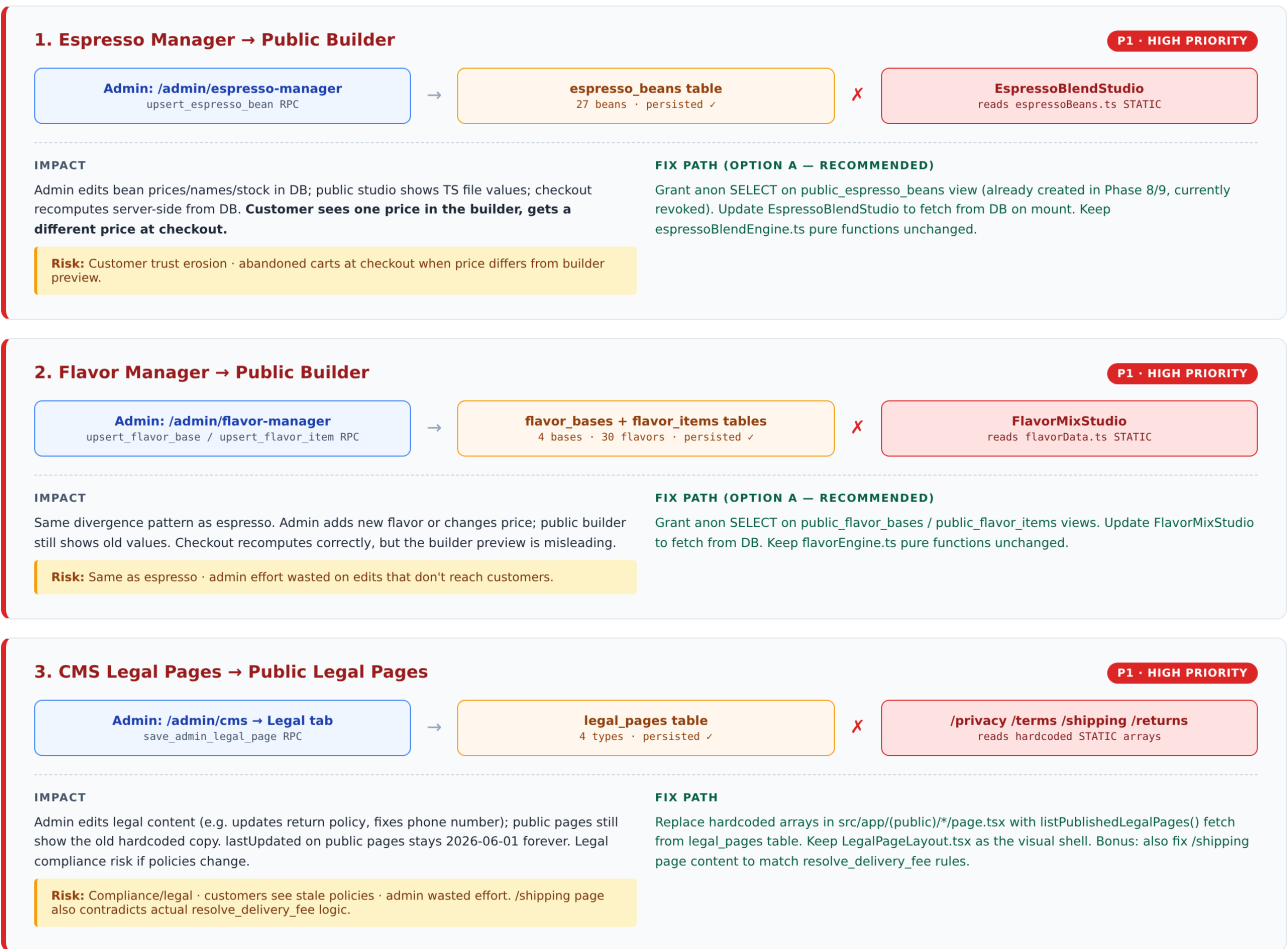


Figure 5 — 3 P1 critical disconnects (see §C.10)

Phased Refactor Roadmap — 9 Phases

Recommended implementation order. Each phase unblocks the next. Total: ~9 sprints.



Figure 6 — 9-phase refactor roadmap (see §I.9)

LINE COFFEE V3 — Independent Master Audit

Date: 2026-07-13

Repository: <https://github.com/muhmiied/line-coffee-fainal.git>

Branch: `main` | **Last commit:** `069a460 polish admin visuals dropdowns and numeric rendering`

Audit mode: Read-only. No files were created, edited, deleted, moved, or refactored in the repository. No commit, no push, no migration touched.

What I inspected

I cloned <https://github.com/muhmiied/line-coffee-fainal.git> to `/home/z/my-project/audit/line-coffee-v3` for **inspection only**. I read:

- `git status --short` → clean tree, branch `main`, last commit `069a460 polish admin visuals dropdowns and numeric rendering`
- All 38 SQL migrations in `supabase/migrations/*.sql` (~14k lines)
- `supabase/seeds/20260625_catalog_seed.sql` + `scripts/generate-catalog-seed.mjs`
- `supabase/config.toml`, `next.config.ts`, `package.json`, `tsconfig.json`
- All public routes under `src/app/(public)/**/*.page.tsx` + their `layout.tsx`
- All admin routes under `src/app/admin/**/*.page.tsx`
- All service files under `src/lib/admin/*.ts` (19 files)
- `src/lib/{checkout,delivery,catalog,cms,content,seo,auth,hooks,context,supabase,validation,utils}/**`
- `src/features/website/{home,checkout,make-your-espresso,make-your-flavor}/**`
- `src/components/{layout,product,admin,ui,shared,icons,error}/**`
- `src/middleware.ts`, `src/app/api/**`
- All `.md` docs at root + `docs/**` + `docs/ai/**` + `docs/archive/**`
- `src/lib/mock-data/**`, builder data files, engine files

Confirmation upfront

- ☐ No files created, edited, deleted, moved, or refactored
 - ☐ No commit, no push, no migration touched
 - ☐ All work was read-only (`git status` is clean)
 - ☐ Final audit lives only in this file (and the chat messages it was originally delivered in)
-

Table of Contents

- [Part A — Public Website Content Audit](#part-a--public-website-content-audit)
 - [Part B — Admin Dashboard Audit](#part-b--admin-dashboard-audit)
 - [Part C — Data Flow / System Map](#part-c--data-flow--system-map)
 - [Part D — Mock / Static / Dead Code Audit](#part-d--mock--static--dead-code-audit)
 - [Part E — Security Audit](#part-e--security-audit)
 - [Part F — SEO / GEO / AEO / AI Search Audit](#part-f--seo--geo--aeo--ai-search-audit)
 - [Part G — Performance Audit](#part-g--performance-audit)
 - [Part H — Accessibility / UX / State Audit](#part-h--accessibility--ux--state-audit)
 - [Part I — Architecture / Refactor Plan](#part-i--architecture--refactor-plan)
 - [Part J — Final Master Guide Proposal](#part-j--final-master-guide-proposal)
 - [Final Summary](#final-summary)
-

Part A — Public Website Content Audit

A.0 Shared public infrastructure (used by every route)

Layer	File	Source
Root layout	src/app/layout.tsx	LanguageProvider + CartProvider + PublicHeader + PublicFooter; cookie-driven lang/dir; fonts Playfair + Aligarh (self-hosted) + Cairo/Tajawal (Google); injects organizationJsonLd() + websiteJsonLd() on every page
Brand constants	src/lib/seo/site.ts	SITE_NAME, SITE_NAME_AR = "ي فوك نيال", FOUNING_YEAR="2015", SITE_URL, SITE_WHATSAPP_PHONE (env), DEFAULT_OG_IMAGE="/assets/hero/dark-roast.png", SITE_KEYWORDS (18 bilingual terms), SITE_CATEGORIES (7 slugs) — all STATIC
Public layout	src/app/(public)/layout.tsx	Pure visual wrapper <div className="line-public">
Header	src/components/layout/public/PublicHeader.tsx (1205 lines)	Nav links STATIC (Home/Products/About/Contact/Blog); announcement bar reads announcements table → fallback DEFAULT_ANNOUNCEMENTS in src/lib/content/announcements.ts; cart + wishlist drawers owner-scoped via useCart() + useWishlist()
Footer	src/components/layout/public/PublicFooter.tsx	Logo /brand/logo-white.svg STATIC; 4 link columns STATIC; contact info REAL from getPublicSettings() (site_settings); social icons only render when URL set
Asset map	src/lib/mock-data/visual-content.ts:17-36	assets object — STATIC paths under /public/assets/{hero,story,categories,products}/*. Note: duplicate copies exist under /public/images/generated/* but the code only references /assets/* — those duplicates are dead weight

A.1 / — Homepage

Files: `src/app/page.tsx` (12 lines, server) → `src/features/website/home/LineCoffeeHome.tsx` (client) → 8 sections under `src/features/website/home/sections/`. Scroll-reveal via `useLuxuryScrollReveal.ts` (single `IntersectionObserver` on `[data-reveal]`).

Hero (deep focus) — `HeroSection.tsx`

3 slides, all STATIC in `visual-content.ts:40-89`:

#	Image	Title EN	Subtitle EN	Primary CTA → route	Secondary CTA → route
0	<code>/assets/hero/dark-roast.png</code>	"Coffee Crafted for Quiet Luxury"	"Selected beans, slow-roasted..."	"Shop Coffee" → <code>/products</code>	"Our Story" → <code>/about</code>
1	<code>/assets/story/roastery.png</code>	"Craft Your Own Espresso Blend"	"Balance beans by ratio..."	"Make Your Espresso" → <code>/products?category=make-your-espresso</code>	"Explore Espresso Blends" → <code>/products?category=espresso-blends</code>
2	<code>/assets/categories/flavor.png</code>	"Design Your Flavored Coffee"	"Pick a base and layer warm flavors..."	"Make Your Flavor" → <code>/products?category=make-your-flavor</code>	"Explore Flavor Coffee" → <code>/products?category=flavor-coffee</code>

- Carousel logic: `useState(0) + setInterval(goNext, 5600ms)` (lines 98-103). Prev/Next arrow buttons + dot indicators. RTL-aware (arrow labels swap on `dir`).
- **Count-up stats** (3): `15+ Origins, 72h Fresh Roast Window, 100% Arabica Focus` — STATIC `heroStats`. Animated via `useCountUp()` (36-frame eased interval). Re-animates on **every** home navigation (no `sessionStorage` guard like `/order-success` has).
- Scroll cue: `HeroScrollCue` hides when `window.scrollingY > 80`.
- **Source label: MOCK / STATIC** — no Supabase reads, no admin UI to edit hero.

Owner edit guide:

- Text → edit `src/lib/mock-data/visual-content.ts` lines 40-89
- Images → replace files in `public/assets/hero/` and `public/assets/story/`
- Stats numbers → `heroStats` array (lines 93-97) + `heroStatDetails` (lines 18-40)

Remaining 7 sections

Section	Source	Notes
<code>CategoriesSection</code>	STATIC <code>visualCategories</code> (7 items)	Marquee animation, CSS-driven, 4 reps × 2 loops
<code>FeaturesSection</code>	STATIC <code>visualFeatures</code> (4 items)	Lucide icons: support/delivery/coffee/quality
<code>StorySection</code>	STATIC <code>storyCopy</code> + reused <code>heroStats</code>	Image <code>/assets/story/roastery.png</code>

Section	Source	Notes
BestSellersSection	REAL <code>getPublicBestSellers()</code> — Supabase <code>public_products</code> where <code>best_seller=true</code>	<code>MARQUEE_THRESHOLD=4</code> (1-4 grid, 5+ marquee). Loading/error/empty states bilingual
JournalSection	MOCK / STATIC <code>visualJournal</code> (3 items)	△ Links to <code>/blog/{slug}</code> for slugs <code>roast-notes</code> , <code>blend-guide</code> , <code>freshness</code> — likely 404 because no matching rows in <code>blog_posts</code>
TestimonialsSection	REAL <code>listApprovedHomepageReviews()</code> — Supabase <code>reviews</code> where <code>status='approved'</code> AND <code>hidden=false</code> AND <code>show_on IN ('homepage', 'both')</code>	Empty state: "Approved customer reviews will appear here." (no mock fallback)
SocialGallerySection	MIXED — images STATIC <code>socialGalleryImages</code> (6 paths), social URLs REAL from <code>getPublicSettings().social</code>	Falls back to <code>@linecoffee.eg</code> Instagram handle
ContactSection	MIXED — <code>contactItems</code> structure STATIC, phone populated at runtime from <code>getPublicSettings().contact.supportPhone</code> , email hardcoded <code>info@linecoffee.com</code> (STATIC — should come from settings). Form → <code>submitContactMessage()</code> → Supabase RPC <code>create_contact_message</code> (REAL)	

Homepage risks

1. **DEAD LINK RISK (P1):** `JournalSection` mock blog slugs likely don't exist → 404s on click
2. **MOCK content with no admin UI:** hero, categories, features, story, social gallery all require code edits
3. **Email mismatch (P2):** homepage `info@linecoffee.com` vs site URL `linecoffee.eg`
4. **Hero count-up** re-runs on every navigation (minor visual noise — P3)

A.2 /products — Catalog

Files: `src/app/(public)/products/page.tsx` (360 lines, "use client"), `src/lib/catalog/public-catalog.ts`

Sections:

- `ProductsHero` — banner with hardcoded `/assets/story/roastery.png` bg, title "Our Products / انتاجاتنا" (STATIC)
- Sidebar — REAL categories from `getPublicCategories()` + injected studio pseudo-categories `make-your-espresso` / `make-your-flavor` (STATIC labels)
- Main grid — `ProductCard` per product

- Dynamic studio import — `next/dynamic` for `EspressoBlendStudio` / `FlavorMixStudio` (`ssr:false`, "Loading studio..." fallback)
- Search input — filters by `name.en` / `name.ar` (no `aria-label` [⚠](#))
- Empty / loading / error states for each branch

Data sources (REAL):

- `getPublicCategories()` → Supabase view `public_categories` (filters `status='visible'` AND `show_on_website=true`)
- `getPublicProducts()` → Supabase view `public_products` + `public_product_variants`. **Mini-waterfall:** round-trip 1 (categories + products parallel) → round-trip 2 (variants by `product_id` IN). **No pagination** — entire catalog fetched.

Numbers:

- Prices from `public_product_variants.price` per size (250g/500g/1kg)
- Currency: `language === "ar" ? "ج.م" : "EGP"` (hardcoded in components)
- `?previewProduct=&previewImage=` admin preview override (validates image is one of `product.image/gallery/category.image`)

Interactions:

- Category select updates URL via `router.replace('/products?category=${cat}', {scroll: false })`
- Dead code branch: lines 242-258 define a "Soon" disabled button — never triggered because no item sets `disabled:true`

Source label: REAL (Supabase) + STATIC hero/studio labels

Risks (P1):

- Client-only fetch → empty HTML shell for crawlers (SEO body content invisible without JS)
- No pagination — payload grows linearly with catalog
- Studio slugs `make-your-espresso` / `make-your-flavor` are hardcoded, not DB-driven

A.3 /products/[slug] — Product detail (deep focus)

Files: `src/app/(public)/products/[slug]/page.tsx` (599 lines, client),
`src/app/(public)/products/[slug]/layout.tsx` (77 lines, server — SEO + JSON-LD)

Sections:

- Hero band with blurred product image as bg + breadcrumb (Home / Products / [Category] / [Product])
- 3-column grid: `ProductGallery` (left), product info panel (middle), purchase sidebar (right, sticky)
- `ProductGallery` — main image + 4 thumbnails via `getGalleryImages()` = unique set of `[product.image, ...product.gallery, category?.image, "/assets/story/roastery.png", "/assets/hero/dark-roast.png"]`. [⚠](#) **Hardcoded fallbacks always appended** even for products with rich galleries — clutters thumbnails

- Product info — category badge, name, description (note), "Taste Profile" metric bars (5 metrics from `getProductMetrics()`), "Blend Composition" list (if `product.blend`)
- Purchase sidebar — weight selector (3 buttons: 250g/500g/1kg), quantity selector, total = `selectedPrice × quantity`, "Add to Cart" + "Save for Later" (wishlist) buttons

Variants:

- `getPriceOptions(product)` reads from `product.sizes[].salePrice`. Default = first size
- 3 sizes only (no admin UI to add more sizes)

Builder categories:

- "Make Your Espresso" / "Make Your Flavor" are NOT on this page directly — they live in `/products` sidebars as dynamic imports
- The customData payload (`EspressoBuilderPayload` / `FlavorBuilderPayload`) lives in cart items added from those studios, handled by `buildCheckoutItem()` in `src/features/website/checkout/checkout-rpc.ts`

Related products: ☐ Not implemented (missed SEO + cross-sell opportunity)

JSON-LD (server layout):

- `productJsonLd(product) + breadcrumbJsonLd([Home, Products, Category?, Product])`
- Server reads `getSeoProduct(slug)` from `public_products + public_product_variants` (min price)
- Missing product → `noindex` + fallback title
- ⚠ `availability` hardcoded `InStock` — does not reflect `inventory_stock / product_variants.stock_state` (P2 SEO)

Taste profile (MOCK — misleading):

`getProductMetrics()` (lines 60-117) — computes 4-5 metric values per category using **fixed base numbers** (e.g. `4.1 + strengthLift`) + price tier (`premiumLift = salePricePerKg >= 900 ? 0.4 : >= 700 ? 0.2 : 0`) + robusta share. **These are NOT real cupping data** — they are pseudo-derived from price + robusta%. ⚠ Risk: presented as objective "Taste Profile" → potentially misleading (P2 — owner should decide whether to label as "Estimated Profile" or remove)

Source label: REAL (Supabase product) + MOCK (taste metrics) + STATIC fallback images

Risks:

- `getGalleryImages()` always appends `roastery.png` + `dark-roast.png` (clutter)
- No "Add to Wishlist" idempotency guard — clicking twice toggles it off
- Breadcrumb links to `/products?category=...` not `/products/category/...` — inconsistent with category page URL
- No "related products" → customer dead-ends after reading

A.4 /products/category/[slug]

Files: `src/app/(public)/products/category/[slug]/page.tsx` (519 lines, client), `layout.tsx` (68 lines, server)

Sections:

- Hero with `category.image` as bg + breadcrumb
- Category Story card: `categoryExperiences[slug]` — STATIC map of 7 known slugs → `{eyebrow, title, intro, story}`. Unknown slugs fall back to `category.name / category.description` (REAL)
- Browse Controls card (description)
- Sticky filter bar: search, price filter (4 options), sort (4 options)
- Product grid using `CatalogProductCard`
- "Related Categories" horizontal scroller — 5 others with image + count

Numbers:

- Price filter thresholds `under-400`, `400-700`, `700-plus` (EGP, hardcoded lines 125-130) using `product.salePricePerKg`
- Sort: `featured` (blend first then price desc), `price-asc`, `price-desc`, `name` (`localeCompare`)

JSON-LD: `categoryJsonLd` + `breadcrumbJsonLd`. Server reads `public_categories`. □

Source label: REAL (Supabase catalog) + STATIC category-experience copy (7 known slugs)

Risks:

- Hardcoded price thresholds may not match real price range
- `categoryExperiences` only covers 7 known slugs; new categories get weak copy

A.5 /cart

File: `src/app/(public)/cart/page.tsx` (238 lines, client)

Sections:

- Hero bar (hardcoded "Shopping" eyebrow + "Your Cart" title)
- Empty state with "Browse Coffee" CTA → `/products`
- Items list (Lucide `ShoppingBag` icon placeholder, name, detail, qty controls, line total, remove) ⚠ **No product images** — visual downgrade vs. wishlist page
- "Continue shopping" link → `/products`
- Order Summary sidebar (sticky): Subtotal, Delivery, Total, "Proceed to Checkout" → `/checkout`, "Secure order · Cash on delivery" disclaimer

Numbers:

- ⚠ **CRITICAL BUG (P0):** `deliveryFee = total >= 500 ? 0 : 50;` (line 13) — **DIFFERENT** from `/checkout` which uses zone-based `resolveDeliveryFee()` (Shorouk

30, Cairo/Giza 50, Haram/October/Zayed 100, others courier-paid). **Customer sees different totals on /cart vs /checkout** → likely abandonment

- Subtotal/total from `useCart()` items

Source label: REAL (cart state in localStorage) + MOCK (delivery fee)

A.6 /checkout (deep focus)

Files: `src/app/(public)/checkout/page.tsx` (9 lines, just renders `<CheckoutForm/>`), `layout.tsx` (wraps in `CheckoutOwnerBoundary`, `robots: noindex`), `CheckoutOwnerBoundary.tsx` (remounts on auth user change via `key={user?.id ?? "guest"}`), `src/features/website/checkout/*` (`CheckoutForm` 735 lines, `AddressSection`, `PaymentSection`, `PromoSection`, `OrderSummary`, `CheckoutPrimitives`, `checkout-rpc.ts`, `types.ts`), `src/lib/checkout.ts`, `src/lib/delivery.ts`, `src/lib/checkout/governorates.ts`

Sections (render order):

1. Hero bar — "Checkout" eyebrow + "Complete Your Order" title + closed-store notice (if `storefront.storeOpen === false`)
2. `AddressSection` — saved addresses (registered users), customer info, delivery address (governorate dropdown, area dropdown, manual area, street, building, floor/apt, Google Maps URL)
3. `PaymentSection` — 3 options
4. `OrderSummary` (right, sticky) — items, `PromoSection`, totals, "Place Order" button

Fields:

- Identity: `name` (req), `phone` (req, Egyptian validation via `isValidEgyptianPhone()` in `src/lib/validation/phone.ts`), `whatsapp` (req), `email` (optional, regex)
- Address: `governorate` (req), `area` (req; "Other" triggers `manualArea` req), `manualArea`, `street` (req), `building`, `floorApt`, `googleMapsUrl` (validated URL, http/https only)
- Payment: `paymentMethod` (`cash` | `instapay` | `e-wallet`), `paymentReference` (instapay), `paymentPhone` (e-wallet)
- Validation in `validate()` (lines 337-373)

Governorates dropdown:

- `EGYPT_GOVERNORATES` from `src/lib/checkout/governorates.ts` — STATIC list of 27 Egyptian governorates + ~120 areas
- Rendered via custom `CustomSelect` (not native `<select>`)
- "Other" → manual area input appears

Delivery fee rules (deep focus):

`resolveDeliveryFee(governorate, area)` in `src/lib/delivery.ts` — **DISPLAY MIRROR** of SQL `public.resolve_delivery_fee`. Server recomputes inside `create_checkout_order` RPC.

Zone	Match	Fee (EGP)
shorouk_madinaty	area contains "shorouk"/"madinaty" "شوروك"/"مدينتي"	30
haram_october_zayed	area contains "haram"/"مهرهلا"/"october"/"ربوتكا"/"sheikh zayed"/"ديار"	100 (checked before Cairo/Giza)
cairo_giza	gov = Cairo/Giza/القاهرة/الجيزة	50
governorate_courier	all other governorates	0 (courier pays)

OrderSummary renders:

- `deliveryZone === null` → "Select address"
- `zone === "governorate_courier"` → green "Paid to courier" + helper text
- otherwise → fee in EGP

Payment methods (PaymentSection.tsx) — STATIC labels:

Key	EN	AR	Description
cash	Cash on Delivery	شاك	"Pay when delivered"
instapay	InstaPay	ي اب اتس نإ	"Bank transfer via InstaPay" (+ optional reference input)
e-wallet	E-Wallet	ةينورتكللإة طفح م	"Vodafone Cash & others" (+ optional wallet phone input)

Default = `cash`. DB mapping: `cash_on_delivery` / `instapay` / `wallet`.

Promo code logic:

- UI: input (uppercase, maxLength 32) + "Apply" button
- `handleApplyPromo()` (lines 472-499) → `validatePromoCode(code, subtotal, guestId)` from `src/lib/checkout.ts`
- Calls `**Supabase RPC validate_promo_code**` with `p_code`, `p_subtotal`, `p_guest_id`. Returns `{status, code, discountTotal, subtotal, discountedSubtotal, minimumSubtotal?, message}`
- Statuses: `valid` / `invalid` / `not_started` / `expired` / `inactive` / `usage_limit_reached` / `minimum_not_met` / `customer_limit_reached`
- Discount applies to **product subtotal only** — delivery is never discounted
- Re-validation needed if cart changes (`promoMatchesSubtotal` check, `Math.abs < 0.01`)
- Admin manages codes via `src/lib/admin/admin-marketing.ts` → `upsert_promo_code` RPC

****Order creation flow (deep focus) — `handleSubmit()` lines 506-635:****

1. `submitInFlight.current` ref guard → return if already submitting
2. `storeClosed` check → block with error message
3. `validate()` → `setState` errors + return on failure

4. `mappedItems = items.map(buildCheckoutItem)` from `checkout-rpc.ts`. Returns null if cart item malformed → "A cart item is missing its details."
5. `submitInFlight.current = true; setSubmitting(true)`
6. Generate `checkoutAttemptId` (UUID, once per attempt — idempotency key)
7. `supabase.rpc("create_checkout_order", { p_payload: { guest_id, checkout_attempt_id, customer, address, payment, promo_code, items } })`
8. On error → `getCheckoutError(error.message)` (lines 375-470) maps 12+ specific substrings to bilingual user-facing text (Store closed / Promo expired / Insufficient stock / Invalid espresso blend / Duplicate bean / must total 100 / Invalid email / Invalid Google Maps URL / etc.)
9. Validate response shape with `isCheckoutOrderResult(data)` — checks `order_id/code` are strings, totals are finite + safe (discount ≤ subtotal, total = subtotal – discount + delivery)
10. Build `CheckoutOrderHandoff` from form data + items (localized name/detail + qty)
11. **Telegram notification (best-effort):**
`fetch("/api/order-notifications/telegram", { method: "POST", body: { orderId, checkoutAttemptId } })`. Sets `handoff.telegramStatus = "sent" | "failed"`. Failures logged but don't block.
12. Persist `{ ...data, handoff }` to `sessionStorage` under `checkoutResultStorageKey(order_id) = "line-checkout-result:<order_id>"`
13. `clearCart() + router.push("/order-success?id=<order_id>&order=<code>")`
14. `finally`: if not `orderPlaced`, release `submitInFlight`

****Telegram endpoint (deep focus) —**

`src/app/api/order-notifications/telegram/route.ts:**`

- Validates Origin header (CSRF defense)
- Validates `orderId` (UUID) + `checkoutAttemptId` (`^[A-Za-z0-9_-]{8,64}$`)
- Creates fresh Supabase anon client (no session)
- Calls `get_order_notification_payload(p_order_id, p_checkout_attempt_id)` RPC — **TRUSTED source**: server fetches order/customer/address/items/totals from DB. Browser-supplied fields NEVER trusted
- Dedupe: in-memory `Map<orderId, sentAt>` (24h TTL, max 500) + durable `order_notification_was_sent` RPC + `log_order_notification` RPC
- Builds Telegram message (`buildTelegramMessage`): order code, customer, address, items, totals, payment, admin URL `/admin/orders?order=<id>`
- Sends to `https://api.telegram.org/bot<token>/sendMessage` with 5s `AbortController` timeout
- Requires env `TELEGRAM_BOT_TOKEN` + `TELEGRAM_CHAT_ID` — otherwise returns 503

****WhatsApp handoff — `src/lib/checkout.ts:136-191`**

`buildWhatsAppOrderHref(result):**`

- Uses `result.handoff.whatsappHref` (resolved at submit from `settings.contact.whatsappNumber` → env `NEXT_PUBLIC_WHATSAPP_PHONE` → `settings.social.whatsapp`)
- Builds message: "Hello Line Coffee, I have placed an order:" + order code + name + phone + WhatsApp + address + items + total + payment + "Please confirm my order. Thank you."

- URL validation: must be `https:` + hostname in `wa.me` / `api.whatsapp.com` / `web.whatsapp.com`
- Consumed on `/order-success`

Source label: REAL (Supabase RPCs for order + Telegram + WhatsApp config)

Risks:

- `getCheckoutError` does substring matching on RPC error messages — fragile if SQL messages change
- If Telegram env vars missing, customer sees warning banner on `/order-success` asking them to send WhatsApp manually — acceptable degradation
- No payment gateway — all methods start as `pending`
- No rate limit on Telegram endpoint (Medium — F2 in Part E)

A.7 /order-success (deep focus)

Files: `src/app/(public)/order-success/page.tsx` (254 lines, client, wrapped in `<Suspense>`), `layout.tsx` (`robots: noindex`)

Where the order data comes from — THREE sources merged defensively:

1. **URL search params** (`?id=<order_id>&order=<order_code>`) — `id` is UUID, `order` is human-readable (e.g. `LC-000001`)
2. **sessionStorage** keyed by `checkoutResultStorageKey(orderId) = "line-checkout-result:<order_id>"`. Read via `useSyncExternalStore` with `RECEIPT_LOADING` server snapshot (avoids hydration mismatch). Validated by `isCheckoutOrderResult(parsed) AND parsed.order_id === orderId`
3. **No Supabase re-fetch** — page does NOT call `getCustomerOrderDetail`. The `result` object is the exact `create_checkout_order` payload + `handoff`

Sections:

- Hero — "Order Received / Thank You!" (STATIC)
- Receipt card: `CheckCircle` icon, "Your order number" + `orderCode`, "Our team will contact you shortly..."
- If `result` present: order summary (item count, payment method label, payment status "Pending", total EGP)
- If `result` absent: "The receipt details are available in the browser session that placed the order." + raw `orderId`
- Telegram warning: if `handoff.telegramStatus === "failed"`, amber banner asking customer to send WhatsApp
- WhatsApp button (if `whatsappUrl` from `buildWhatsAppOrderHref(result)` is non-null)
- Two CTAs: "Continue Shopping" → `/products`, "Back to Home" → `/`
- "Line Coffee Promise" footer: "Roasted within 72 hours of your order."

Auto-open WhatsApp:

`useEffect` (lines 68-83): if `whatsappUrl` and `sessionStorage.getItem("line-whatsapp-opened:<orderId>")` is unset → set it and:

- Mobile (userAgent sniff): `window.location.assign(whatsappUrl)`
- Desktop: `window.open(whatsappUrl, "_blank", "noopener,noreferrer")`

Source label: MIXED — `result` from sessionStorage (LOCAL) + `orderCode` from URL search param. **No Supabase read on this page.**

Risks:

- If user opens success URL on different device/browser → `result` is null, sees only order code, no totals/items. **No fallback to fetch from Supabase by order_id** (P2 — should call `getCustomerOrderDetail(orderCode)` for resilience)
 - Auto-opening WhatsApp on mobile may be blocked by popup blockers (`location.assign` should still work)
 - No link to `/account/orders/[id]` — customer dead-ends after success (P1)
-

A.8 /about

File: `src/app/(public)/about/page.tsx` (370 lines, client)

Sections (all STATIC hardcoded constants):

1. Editorial Intro — image `/assets/story/roastery.png` + "Since 2015" badge + "We Are Line Coffee" + body + "Explore Our Products" → `/products`
2. Philosophy — reversed layout, "Roasting Philosophy" eyebrow + "Slow Roast. Full Attention." + 3 pillars + image `/assets/hero/dark-roast.png`
3. Journey Timeline — 3 milestones (`JOURNEY` array, STATIC): 2015 Founded in Cairo, 2018 First Blend Lineup, Today Line Coffee
4. Quote + CTA — full-bleed bg + quote "Coffee is not a commodity. It is a ritual, a relationship, and a reason to slow down." + 3 CTAs

Source label: ALL STATIC

Risks:

- "Since 2015" + Journey hardcoded though `FOUNDING_YEAR = "2015"` exists in `site.ts` (not used here)
 - No DB-driven content — every edit requires code change
-

A.9 /contact

File: `src/app/(public)/contact/page.tsx` (578 lines, client)

Sections:

1. Hero — `/assets/story/roastery.png` bg + "Get in Touch"
2. Form + Info (2-col): Contact Form (left) + Contact Info cards (right)
3. WhatsApp Feature Strip — only if `whatsappHref` non-null
4. FAQ + CTA — 6 FAQ accordion items + "Explore Products"

Form:

- Fields: Full Name (req), Phone, Email, Subject (select: General inquiry / Order issue / Wholesale / Custom blend / Other), Message (req)
- Validation: name + subject + message AND (phone OR email)
- Submit → `submitContactMessage({...form, source: "contact_page"})` → Supabase RPC `create_contact_message`
- Success: "Message Sent / Thank you for reaching out. We will get back to you within 24 hours."

Contact info cards:

- WhatsApp card — if `whatsappHref` non-null (REAL from `site_settings` + env fallback)
- Phone card — only if `supportPhone` set AND distinct from WhatsApp (REAL)
- Email card — if `supportEmail` set (REAL)
- Location card — if `businessAddress` set (REAL)
- If none: "Contact details are not available yet."
- "Response time" panel — STATIC

FAQ: `FAQ_ITEMS` array, 6 items, **STATIC** (lines 23-66). ⚠ FAQ answer copy hardcodes `+20 100 476 1171` — can drift from `site_settings`

Source label: **MIXED** (REAL form + REAL contact settings + STATIC FAQ + STATIC hero)

A.10 /blog + /blog/[slug]

****blog file:**** `src/app/(public)/blog/page.tsx` (334 lines, client),
`src/lib/cms/public-blog.ts`

****blog sections:****

1. Hero — `/assets/story/roastery.png` bg + "Coffee Journal / All Things Coffee"
2. Featured Post — first post with `featured: true`. Big 2-col card with image + category tag + title + excerpt + date + readTime + "Read article"
3. All Posts — search + category filter + 3-col grid

Data source: `listPublishedBlogPosts()` — Supabase `blog_posts` where `status='published'` AND `published_at <= now()`, ordered `featured DESC`, `published_at DESC`, **limit 200**

****blog/[slug] file:**** `src/app/(public)/blog/[slug]/page.tsx` (271 lines, client),
`layout.tsx` (server — SEO + Article JSON-LD)

Sections:

1. Cover hero — `post.heroImage` full-bleed
2. Article body — breadcrumb, meta row, title, gold rule, body blocks (heading/paragraph)
3. "Back to Blog" link
4. Related Articles — 2 next posts (REAL)
5. Products CTA — "From Blog to Bag" + "Shop Coffee" → `/products`

Source label: REAL

Risks:

- Blog detail fetches **ALL** published posts then filters client-side by slug (`posts.find(item => item.slug === slug)`) — inefficient for large blogs (200-post fetch per pageview)
- Server-side `getSeoBlogPost(slug)` in layout does fetch one post for metadata, but the page itself doesn't use it

A.11 /reviews

File: `src/app/(public)/reviews/page.tsx` (5 lines)

Behavior: `redirect("/contact")` — **DEAD END / REDIRECT**. No actual reviews page exists. Homepage TestimonialsSection pulls from `reviews` table but there's no dedicated listing page.

Risk (P2): If linked from anywhere (footer/header), misleading. `/reviews` is NOT in `sitemap.ts` (good).

A.12 Legal pages — /privacy, /terms, /shipping, /returns

Shared layout: `src/components/ui/LegalPageLayout.tsx` (113 lines, client) — hero with `/assets/hero/dark-roast.png` bg + "Line Coffee" eyebrow + title + subtitle; body "Last updated: YYYY-MM-DD" + sections; footer "Questions about our policies?" + "Contact us" → `/contact`.

Route	File	Sections	lastUpdated	Source
<code>/privacy</code>	<code>privacy/page.tsx</code> (112 lines)	7 sections: Introduction, Information We Collect, How We Use, Data Storage, Cookies, Your Rights, Contact	2026-06-01	ALL STATIC. Contact hardcodes <code>info@linecoffee.com</code> + <code>+20 100 476 1171</code>
<code>/terms</code>	<code>terms/page.tsx</code> (105 lines)	7 sections: Acceptance, Products/Pricing, Orders/Payment, Delivery/Shipping, IP, Limitation of Liability, Governing Law	2026-06-01	ALL STATIC

Route	File	Sections	lastUpdated	Source
/shipping	shipping/page.tsx (95 lines)	6 sections: Delivery Areas, Times, Fees, Tracking, Failed Delivery, Freshness Guarantee	2026-06-01	ALL STATIC. [△] STALE / CONTRADICTS CODE (P1): says "free over 500 EGP", "flat 50 below 500", "Express 80 EGP", "re-delivery 30 EGP" — but actual resolveDeliveryFee is zone-based (30/50/100/courier-paid). Also lists only Greater Cairo + 5 cities while EGYPT_GOVERNORATES has all 27
/returns	returns/page.tsx (100 lines)	7 sections: Policy, Eligible, Non-Returnable, How to Initiate, Refund Processing, Damaged/Incorrect, Contact	2026-06-01	ALL STATIC. 48-hour return window, 3-5 business day refund, custom blends non-returnable. Contact hardcodes same email/phone

Source label for all legal: STATIC

Risks:

- Phone/email hardcoded — can drift from **site_settings** (P2)
- **/shipping** policy describes Express delivery that doesn't exist (P1 SEO + UX)
- **lastUpdated** hardcoded 2026-06-01 — will appear stale over time

A.13 Auth — /auth/login, /auth/signup, /auth/forgot-password, /auth/reset-password

Shared: **src/app/(public)/auth/layout.tsx** (robots: noindex), **src/components/layout/auth/AuthCard.tsx** (title + subtitle + children card)

****/auth/login**** (161 lines):

- Fields: Email, Password (Eye/EyeOff toggle)
- "Forgot password?" → **/auth/forgot-password**
- Submit → **useAuth().signIn(email, password)** → **resolvePostLoginDestination(next)** → router.replace
- **next** param: only non-**/admin*** paths accepted (collapsed to **/** otherwise). Already-signed-in users auto-forwarded
- "Create one" → **/auth/signup**

- **Source: REAL (Supabase Auth)**

****`/auth/signup`**** (211 lines):

- Fields: Full Name, Email, Password (≥ 8 chars), Confirm
- Submit → `useAuth().signUp(name, email, password)`. If `data.session` (instant on when email confirmation disabled) → redirect. Else → "Check your email" success state

- **Source: REAL**

****`/auth/forgot-password`**** (144 lines):

- Field: Email
- Submit → `supabase.auth.resetPasswordForEmail(email, { redirectTo: "/auth/reset-password" })`
- Success: "Check your email"

- **Source: REAL**

****`/auth/reset-password`**** (237 lines):

- Phases: `checking` → `ready` (or `invalid` after 1.5s) → `done`
- Listens for `PASSWORD_RECOVERY` event via `onAuthStateChange` + `getSession()` poll
- Fields: New Password (≥ 8), Confirm
- Submit → `supabase.auth.updateUser({ password })`
- ⚠ **Low (F8 in Part E)**: accepts "session is present" as sufficient — doubles as password-change page for logged-in users (benign but unintended)

- **Source: REAL**

A.14 Account — `/account/*` (deep focus)

Shared:

- `src/app/(public)/account/layout.tsx` — robots: `noindex`, wraps in `AccountOwnerBoundary`
- `src/app/(public)/account/AccountOwnerBoundary.tsx` — `<div key={user?.id ?? "signed-out"}>` so subtree remounts on auth user change (prevents data leak between accounts)
- `src/components/layout/account/AccountShell.tsx` — sidebar nav (6 links STATIC) + user badge + sign-out. Auth guard: if `!isLoggedIn` → `router.replace("/auth/login")`

Nav items: Profile, My Orders, Addresses, Wishlist, Notifications, Settings

Route	File	Data source	Notes
<code>/account/profile</code>	<code>profile/page.tsx</code> (230)	REAL — <code>getCustomerProfile()</code> RPC <code>get_customer_profile;</code> save via <code>updateCustomerProfile()</code> RPC	Form: First/Last Name (parsed from full name), Phone, WhatsApp, Email (readonly — Supabase Auth managed). "Change password" → <code>/auth/forgot-password</code>

Route	File	Data source	Notes
/account/addresses	addresses/page.tsx (575)	REAL — <code>getCustomerAddresses()</code> RPC + add/update/delete/set-default RPCs	Cards with label, recipient, street, governorate, landmark, phone, "View on map" (if <code>locationUrl</code>). Native <code><select></code> for governorate + area
/account/orders	orders/page.tsx (128)	REAL — <code>getCustomerOrders()</code> RPC <code>get_customer_orders</code> with <code>p_guest_id</code>	Cards: code, status badge (STATIC <code>STATUS_LABEL</code> + <code>STATUS_COLOR</code> maps), date, item count, total → <code>/account/orders/\${order.code}</code> . Empty state: "No orders yet."
/account/orders/[id]	orders/[id]/page.tsx (346)	REAL — <code>getCustomerOrderDetail(orderCode)</code> RPC	<code>params.id</code> is order CODE (e.g. LC-000001). Tracking stepper (4 steps), items, financial summary, address snapshot, order history timeline
/account/notifications	notifications/page.tsx (128)	REAL — <code>getCustomerNotifications()</code> RPC + STATIC <code>STATUS_NOTIFICATION</code> content map	Cards: title + body + optional note + order code + date → <code>/account/orders/\${orderCode}</code> . Empty state: "No notifications yet."
/account/settings	settings/page.tsx (149)	MOCK / STATIC ⚠	Language switcher writes <code>localStorage["lang"]</code> but root layout reads <code>cookies().get("line-coffee-language")</code> — toggle may not persist across reloads . Notification toggles (3 switches) are LOCAL STATE ONLY — non-persistent, cosmetic. "Sign out of all devices" link → <code>/auth/login</code> does NOT actually revoke other sessions
/account/wishlist	wishlist/page.tsx (173)	REAL — <code>useWishlist().ids</code> + <code>getPublicProductsBySlugs()</code>	Cards: image, name (link to <code>/products/\${slug}</code>), price/size, "Add to cart" (first size + <code>addItem</code>), Remove. Empty state: "Your wishlist is empty"

****Wishlist store (`src/lib/hooks/useWishlist.ts`):****

- Owner-scoped: `auth:<userId>` for authenticated, `guest:<guestId>` for guests
- Server is source of truth via `getCustomerWishlist`, `addCustomerWishlistItem`, `removeCustomerWishlistItem` RPCs

- Guest-only: localStorage cache at `line-wishlist-v1:guest:<guestId>` for instant paint
- Authenticated: NEVER writes to localStorage (security: no account-owned items can leak)
- On auth change: immediate clear + refetch from server
- Purges legacy global key `line-wishlist-v1` once on mount

****Cart store (`src/lib/context/cart.tsx`):****

- Owner-scoped: `line-cart-v1:<ownerKey>` localStorage keys
- Auth watcher via `supabase.auth.onAuthStateChange` + `getSession()` + `AUTH_OWNER_CHANGED_EVENT` custom event
- Items validated by `isCartItem` (kind, localized name/detail, finite price, integer qty>0, optional builder payload)
- `addItem` for "product" uses deterministic ID `product-<slug>-<detail>` (merges same product+size); studio items get `studio-<kind>-<seq>` (each add is separate)
- Cross-tab sync via `storage` event listener

Account risks:

- `/account/settings` largely cosmetic — toggles non-persistent (P1)
- Language toggle writes wrong storage key (P1)
- Guest `guest_id` in `localStorage["line-guest-id-v1"]` — guests who clear storage lose their data linkage (documented tradeoff, P3)
- No "Sign out of all devices" actually works (P2)

A.15 Public-route consolidated summary table

Route	Data source	Biggest risk
<code>/</code>	MIXED (hero/categories/features/story/journal/social = STATIC; BestSellers + Testimonials = REAL; ContactSection form = REAL)	JournalSection mock blog slugs likely 404 (P1)
<code>/products</code>	REAL + STATIC hero/studio	Client-only fetch + no pagination (P1)
<code>/products/[slug]</code>	REAL + MOCK taste metrics + STATIC fallback images	Hardcoded gallery fallbacks clutter (P2); taste metrics misleading (P2)
<code>/products/category/[slug]</code>	REAL + STATIC category-experience copy	Only 7 known slugs have rich copy (P2)
<code>/cart</code>	REAL cart + MOCK delivery fee	Delivery fee ≠ checkout → different totals (P0)
<code>/checkout</code>	REAL (Supabase RPC)	<code>getCheckoutError</code> substring matching fragile (P2)
<code>/order-success</code>	LOCAL (sessionStorage) + URL params	No Supabase fallback on different device (P2); no link to <code>/account/orders</code> (P1)

Route	Data source	Biggest risk
/about	STATIC	Every edit requires code change
/contact	MIXED (REAL form + REAL settings + STATIC FAQ + STATIC hero)	FAQ phone hardcoded (P2)
/blog	REAL	limit 200 — fine for now
/blog/[slug]	REAL	Fetches ALL posts then filters client-side (P2)
/reviews	REDIRECT to /contact	Misleading if linked (P2)
/privacy /terms /returns	STATIC	Phone/email hardcoded (P2)
/shipping	STATIC	Contradicts actual delivery logic (P1)
/auth/login /signup /forgot-password /reset-password	REAL (Supabase Auth)	SMTP not configured (P0 — Part E F1)
/account/profile /addresses /orders /orders/[id] /notifications /wishlist	REAL	OK
/account/settings	MOCK	Toggles non-persistent + language toggle broken (P1)

A.16 Owner edit guide (consolidated)

To change...	Edit/upload from...
Homepage hero slides	src/lib/mock-data/visual-content.ts lines 40-89 + replace files in public/assets/hero/ and public/assets/story/
Homepage hero stats	heroStats array + heroStatDetails in same file
Homepage categories/features/story/journal/social	Same visual-content.ts
Social URLs	Admin → Settings → Social Links (site_settings.social_links)
Store name / phone / WhatsApp / email / address	Admin → Settings → Brand/Contact (site_settings)
Store open/closed + closed notice	Admin → Settings → Storefront (site_settings.storefront)
Announcement bar	Admin → Marketing → Announcements (announcements table)
Products (name/price/image/blend/badges)	Admin → Products (products / product_variants / product-images Storage bucket)
Categories	Admin → Products → Categories
Best sellers	Admin → Products (toggle best_seller flag)
Blog posts	Admin → CMS → Blog (blog_posts)
Reviews (homepage testimonials)	Admin → CMS → Reviews (reviews with status='approved', show_on)

To change...	Edit/upload from...
Promo codes	Admin → Marketing → Promo Codes (<code>promo_codes</code> + <code>upsert_promo_code</code> RPC)
Delivery fees	BOTH SQL <code>resolve_delivery_fee</code> in <code>supabase/migrations/20260629120000_phase1_delivery_deducti</code> <code>on_payment.sql</code> AND client mirror in <code>src/lib/delivery.ts</code> — change together
Governorates / areas	<code>src/lib/checkout/governorates.ts</code> (STATIC, 27 governorates)
Payment method labels	<code>src/features/website/checkout/PaymentSection.tsx</code> <code>PAYMENT_OPTIONS</code> array
Legal pages (privacy/terms/shipping/returns)	Respective <code>page.tsx</code> files (STATIC <code>sections</code> arrays) — **ideally migrate to <code>legal_pages</code> table (Phase 13A) and read from there**
<code>/about</code> content	<code>src/app/(public)/about/page.tsx</code> constants
<code>/contact</code> FAQ	<code>FAQ_ITEMS</code> array in <code>contact/page.tsx</code>
Telegram bot token / chat ID	Env vars <code>TELEGRAM_BOT_TOKEN</code> + <code>TELEGRAM_CHAT_ID</code>
WhatsApp phone (env fallback)	Env var <code>NEXT_PUBLIC_WHATSAPP_PHONE</code>
Site URL (canonical/OG)	Env var <code>NEXT_PUBLIC_SITE_URL</code>
Account settings toggles	No persistence layer exists — needs new <code>customer_settings</code> table + RPC
Espresso beans (admin)	Admin → Espresso Manager (<code>espresso_beans</code> table) — **BUT public builder reads from <code>espressoBeans.ts</code> static file** (P1 disconnect — see Part D)
Flavor bases/items (admin)	Admin → Flavor Manager (<code>flavor_bases</code> / <code>flavor_items</code>) — same disconnect (P1)

Part B — Admin Dashboard Audit

B.0 Authentication & shell (shared)

File	Function	Notes
src/lib/auth/admin.ts	getCurrentAdmin(), getAdminForUser(user)	Reads getSession() then queries admin_users by auth_user_id. Returns discriminated CurrentAdminResult (signed_out / forbidden / authorized / error). Admin roles: super_admin, admin. status must be active. Never throws.
src/lib/hooks/useCurrentAdmin.ts	useCurrentAdmin()	React hook with 10s watchdog, onAuthStateChange subscription, refresh(). Powers the gate and WelcomeHero.
src/app/admin/layout.tsx	—	Server component; only sets the admin language cookie + wraps in AdminLanguageProvider → AdminShell. Does NOT run auth itself (the shell does).
src/app/admin/page.tsx	—	redirect("/admin/dashboard") — no UI.
src/middleware.ts	—	UX-only line-auth=1 cookie check on /admin/:path*. Documented as NOT a security boundary — RLS is the real gate.

Auth model: Supabase Auth + public.admin_users table; the entire admin area is gated by the client-side useCurrentAdmin hook inside AdminShell. RLS (is_admin()) is the authoritative gate on every read/write. Admin identity is **DB-backed** (admin_users.auth_user_id = auth.uid()), not email allowlist or JWT metadata.

Risk: Auth is enforced client-side in AdminShell. RLS is the real gate, so a determined user could still see a blank admin page render attempt before the redirect. Mitigated by AdminShell resolving to loading/signed_out/forbidden/error/authorized and rendering a GateScreen for every non-authorized status — as long as no admin route opts out of AdminShell.

B.1 /admin (root)

Item	Value
Source	src/app/admin/page.tsx (5 lines)
Behavior	redirect("/admin/dashboard")

Item	Value
Notes	No data, no rendering. Pure redirect.

B.2 /admin/dashboard

Files: `src/app/admin/dashboard/page.tsx` + `src/lib/admin/admin-dashboard.ts` → `getAdminDashboard()` + 12 sub-components.

KPI cards (4, with today/week/month/all toggle)

All built by `buildPeriodKpi(events, now, fmt, allLabel)` in `admin-dashboard.ts:269-292`. Period windows: today = local-midnight→now (prev = yesterday), week = 7 days, month = 30 days.

KPI	Value source	Calculation	File:function
Sales	<code>orders</code> table — <code>total</code> , <code>placed_at</code>	Σ <code>orders.total</code> for non-cancelled orders, bucketed by <code>placed_at</code> . Cancels excluded. Sparkline = last 7 days of daily sales. Unit = EGP.	<code>admin-dashboard.ts:46</code> 2-477 <code>salesKpi</code>
Orders	<code>orders</code> table — <code>status</code>	Count of orders (all statuses) per period. Breakdown chips show counts by status (<code>pending/preparing/shipped/delivered/cancelled/returned</code>) — only nonzero ones.	<code>admin-dashboard.ts:47</code> 9-488 <code>ordersKpi</code>
Customers	<code>customers</code> table — <code>joined_at/created_at</code>	Count of new signups per period (today/week/month). "All" = total. Trend labels relabeled "new today/this week/this month". <code>customerSplit</code> bar = new (last 30d) vs returning.	<code>admin-dashboard.ts:49</code> 0-504 <code>customersKpi</code>
Net Collected	<code>order_payments</code> + <code>order_refunds</code> (both ledgers)	Σ <code>payments.amount</code> — Σ <code>refunds.amount</code> , each event bucketed by its own timestamp (<code>paid_at</code> / <code>refunded_at</code>). Unit = EGP.	<code>admin-dashboard.ts:50</code> 6-510 <code>netCollectedKpi</code>

Trend % uses `trendPct(cur, prev)` from `admin-metrics.ts:29` — null when previous $\leq$ 0 (shown as "—" / no arrow).

Hero stats (`WelcomeHero`)

`heroStats` (`admin-dashboard.ts:684-688`):

- `newOrders = statusCounts.pending` (orders currently in pending)
- `lowStock = inventory.lowStockCount`
- `pendingReviews = reviews.status = 'pending'` count

Greeting is hour-based (`getGreeting()`), localized via `useAdminLanguage`.

Five special cards (5-col grid)

Card	Source	Calculation	File:function
InventoryCard ("Stock On Hand")	<code>inventory_stock</code> table (<code>available_kg</code> , <code>reserved_kg</code> , <code>low_stock_threshold_kg</code>)	<code>onHandKg = Σ available_kg</code> (rounded to 0.1). <code>reservedKg = Σ reserved_kg</code> . <code>lowStockCount = #products</code> where <code>available_kg ≤ low_stock_threshold_kg</code> . <code>productsTracked = row count</code> .	<code>admin-dashboard.ts:559-579</code>
LowStockCard (col-span-2)	<code>inventory_stock</code> joined with <code>products.name_en</code>	Top 5 by <code>available_kg</code> ASC where <code>available_kg ≤ threshold</code> . Empty state: "All tracked stock above threshold".	<code>admin-dashboard.ts:580-583</code>
PreparingOrdersCard	<code>orders.status='preparing' + placed_at</code>	<code>total = #preparing</code> . <code>overdue = #preparing</code> with <code>placed_at < now - 48h</code> (<code>OVERDUE_PREP_HOURS = 48</code> hardcoded). Lists first 4 overdue codes.	<code>admin-dashboard.ts:432, 617-621</code>
FulfillmentCard	<code>orders</code> statuses	<code>delivered = statusCounts.delivered</code> ; <code>lost = cancelled + returned</code> . Shows delivered count + bar <code>delivered / (delivered+lost)</code> . Replaces the old mock "Visitors" card (comment: "There is no visitor/session tracking in the system").	<code>admin-dashboard.ts:624-628</code>

Sales chart

`SalesChart.tsx` — `Recharts AreaChart`. `Data = data.salesTrend` (`buildSalesTrend`, `admin-dashboard.ts:296-339`).

- **Week:** 7 daily buckets, `Σ sales` per day (cancelled excluded), labels `WEEKDAY_SHORT[day]`.
- **Month:** 5-day buckets over last 30 days (6 buckets).
- **Year:** 12 calendar months.

Period toggle in component state. Revenue only — no chart for order count. Code-split via `dynamic(() => import("../"), { ssr:false })` ✓ — recharts stays out of dashboard's initial bundle.

Latest orders table

`LatestOrders.tsx` — `data.latestOrders` (first 6 from `getAdminOrders()`). Columns: Order code, Customer, Total, Status badge, Time (relative). Row action: link to `/admin/orders/{id}` (hover-only chevron). Footer: "Showing the 6 most recent orders · hover a row to view details".

Alerts center

`AlertsCenter.tsx` — `data.alerts` (4 entries, `admin-dashboard.ts:635-681`):

Alert	Count source	Detail	Link
Low stock items	<code>inventory.lowStockCount</code>	Names of up to 3 lowest items	<code>/admin/inventory</code>
Orders overdue in prep	<code>preparing.overdue</code>	First 4 overdue codes + "over 48h in Preparing"	<code>/admin/orders</code>
Unanswered messages	<code>contact_messages.status='new' count</code>	"{n} new contact submissions awaiting reply"	<code>/admin/cms</code>
Reviews awaiting approval	<code>reviews.status='pending' count</code>	"{n} customer reviews pending moderation"	<code>/admin/cms</code>

Header "Manage" link → `/admin/orders` (note: not context-aware, always points to orders).

Best sellers

`BestSellersMonth.tsx` — `data.bestSellers` (`admin-dashboard.ts:528-557`). Top 5 products by `Σ order_items.quantity` where the parent order's status `≠ cancelled`. Columns: rank, image, name, category, units sold (badge), revenue (EGP). Links to `/admin/products`. Empty state: "No sales recorded yet".

Latest review

`LatestReviewCard.tsx` — `data.latestReview` (`admin-dashboard.ts:585-611`). Latest review where `status='approved' AND hidden=false`, sorted by `published_at ?? created_at` DESC. Shows stars, comment, author initials, product, date. Footer: `avgRating/5 avg · {totalReviews} approved`. Empty state: "No approved reviews yet".

Quick actions

`QuickActions.tsx` — **pure static navigation** (NOT metrics). 4 links:

- Add Product → `/admin/products`
- Add Expense → `/admin/accounting`
- Buy Inventory → `/admin/inventory`

- Create Promo Code → `/admin/marketing`

Comment: "Static navigation shortcuts (not metrics). Inlined so the dashboard no longer imports the dashboard mock-data module." Confirms the old mock data was removed.

Mock/static/dead areas

- `QuickActions` is intentionally static (navigation, not data)
- `FulfillmentCard` replaces the old mock "Visitors Today" — explicitly documented as not faked
- No other mock arrays found in the dashboard module
- **Scan caps (silent truncation):** `ORDERS_SCAN_LIMIT=5000`, `ITEMS_SCAN_LIMIT=8000`, `LEDGER_SCAN_LIMIT=8000`, `CUSTOMERS_SCAN_LIMIT=5000`. If real volume exceeds these, KPIs silently truncate (last 5000 orders by `placed_at DESC`).

Risks — `/admin/dashboard`

1. **Client-side aggregation (High):** 11 parallel Supabase queries fan out, then JS aggregates. Will degrade badly beyond the scan caps (no pagination). For >5,000 orders the trend/KPIs become misleading without warning.
2. **Today window uses local-midnight** while `placed_at` is `timestampz` — no UTC normalization; an admin in a different timezone than the server may see "today" cut off at their local midnight rather than the server's.
3. ****OVERDUE_PREP_HOURS = 48 hardcoded**** — should be a `site_settings` key.
4. **No loading skeleton for individual cards:** full dashboard shows a spinner until `getAdminDashboard()` completes. One slow query blocks everything.
5. ****Quick Actions "Buy Inventory" → `/admin/inventory`**** does not deep-link to a purchase-creation modal (the Accounting page is where purchases are created). Slightly misleading label.
6. **Net Collected** mixes payment and refund events by their own timestamps; if a refund lands *after* the period end, it shows up in the next period. Correct but may surprise users.

B.3 `/admin/products` + `/admin/products/[slug]`

Files:

- List: `src/app/admin/products/page.tsx` (1,333 lines)
- Detail: `src/app/admin/products/[slug]/page.tsx` (378 lines)
- Service: `src/lib/admin/admin-catalog.ts` (1,159 lines)
- Drawers: `src/components/admin/products/ProductDrawer.tsx` (1,170 lines), `ProductCreateDrawer.tsx` (332 lines)
- Images: `src/lib/admin/admin-product-images.ts` (287 lines)

List page

- Reads: `getAdminProductsWithVariants()` (`admin-catalog.ts:643`) → fetches `categories`, `products` (full row incl. `kind`, `status`, `visibility`, `show_on_website`, `featured`, `best_seller`, `new_until`, `sale_price_per_kg`, `purchase_cost_per_kg`, `blend`, `image_url`, `gallery`, SEO columns), then `product_variants` (250g / 500g / 1kg).
- **Stock status** computed from variant `stock_state`: "Out of Stock" if all `out_of_stock`; "Low Stock" if any low/out; else "In Stock" (`getProductStockStatus`, `admin-catalog.ts:319`).
- **ProductCreateDrawer** fields:
 - Category (select from existing categories)
 - English Name , *Arabic Name* (text)
 - Slug * (auto-slugify from EN; validated `^[a-z0-9]+(?:-[a-z0-9]+)*$`)
 - English Description, Arabic Description (textarea, optional)
 - 250g / 500g / 1kg prices * (non-negative numbers)
 - Purchase Cost / kg (optional)
 - Toggles: New (default on → sets `new_until = now + 40d`), Featured, Best Seller, Show on website (default OFF)
 - **Save:** `createAdminProduct()` → Supabase RPC `create_admin_product` (`admin-catalog.ts:1048`). Atomic creation of product + 3 variants. Product always created `draft` / `hidden` / `off-website` regardless of the toggle; only `featured` / `best_seller` / `new_until` / `showOnWebsite` input are honored (the show flag is also stored but the lifecycle status stays `draft`).
- Per-row actions: Edit (opens `ProductDrawer`), Archive, Restore (depending on state), Featured/Best-Seller toggles, eye toggle for visibility, reorder categories (up/down arrows), category create/edit/archive.

ProductDrawer (edit existing) — 6 tabs

Tabs: `general` / `media` / `pricing` / `inventory` / `visibility` / `seo` (`ProductDrawer.tsx:34-43`).

General — Name (en/ar), Note (en/ar) → `products.notes_en/notes_ar`, Slug, Meta title (en/ar), Meta description (en/ar). Slug change re-targets the drawer.

Pricing — 250g / 500g / 1kg prices + per-kg cost read-only. Save calls `updateAdminProduct` + `updateAdminProductVariantPrices` (`admin-catalog.ts:794`). The 1kg field also writes `products.sale_price_per_kg`.

Media — Uses `admin-product-images.ts`:

- Upload: `uploadProductImage(productId, file)` → validates MIME (jpg/png/webp/avif/gif) + ≤ 5 MB, then `supabase.storage.from("product-images").upload("{productId}/{ts}-{rand}.{ext}")`, then writes the public URL to `products.gallery` (jsonb array). Primary image NOT changed on upload — staged as pending primary.
- `setPrimaryProductImage` — promotes an existing gallery URL to `products.image_url`.
- `deleteProductImage` — removes from `gallery`, demotes primary to next gallery image or null, then best-effort deletes the Storage object.
- `restoreDefaultProductImage` — sets `image_url = null` (catalog falls back to category image) while keeping gallery intact.

- `reorderProductImages` — rewrites `gallery` order.
- "Preview" button opens `/products?category=...&previewProduct=...&previewImage=...` in a new tab.

Inventory — Read-only `getProductInventory(productId)` (`admin-inventory.ts:267`); only the low-stock threshold is editable via `updateProductLowStockThreshold` (`admin-inventory.ts:299`) — upserts `inventory_stock.low_stock_threshold_kg`. Available/reserved kg are NEVER edited here (per `INVENTORY_NOTICE` constant in `ProductDrawer.tsx:46`).

Visibility — Hidden toggle drives `visibility` + `show_on_website` + lifecycle `status`:

- Showing (Active) → `status='active', visibility='public', show_on_website=true`
- Hiding → preserves current lifecycle status (draft stays draft, archived stays archived). Un-archiving is via Restore, not the toggle.

Featured / Best Seller / New (40-day timer refresh on save) toggles.

SEO — Meta title (en/ar), Meta description (en/ar) → `seo_title_en/ar`, `seo_description_en/ar`.

Archive / Restore:

- `archiveAdminProduct(id)` → `status='archived', visibility='hidden', show_on_website=false` (`admin-catalog.ts:823`).
- `restoreAdminProduct(id)` → `status='draft', visibility='hidden', show_on_website=false` — never auto-republishes (`admin-catalog.ts:839`). Restore requires explicit Active toggle to republish.
- **No hard delete anywhere.** `product_variants` untouched. `order_items.product_slug` FK is `ON DELETE SET NULL` (per code comment) — order history is preserved.

Category management

- `createAdminCategory` — direct INSERT into `categories` with `source='admin', status='draft', show_on_website=false` by default.
- `updateAdminCategory` — name (en/ar), slug (validated), description (en/ar), status, `show_on_website`, `sort_order`.
- `archiveAdminCategory` → `status='archived' + show_on_website=false`.
- `restoreAdminCategory` → `status='visible'` (but `show_on_website` stays false until explicitly toggled).
- `reorderAdminCategories` — sequential `sort_order = (i+1)*10` for the full ordered id list.
- A DB trigger `trg_categories_sync_slug` rewrites `products.category_slug` when the slug changes (per code comment).

Detail page `/admin/products/[slug]`

Only edits **prices** (Edit Prices button). Other fields are read-only display. Margin is computed $((\text{sale} - \text{cost}) / \text{sale}) * 100$. ⚠ Confusing UX — to edit any other field, the admin must go back to the list and open the drawer.

Mock/static/dead areas

- `fallbackCategoryImages` map (`admin-catalog.ts:168`) is a static asset map — intentional, used as image fallback when `image_url` is null.
- Drawer tabs `media` and `inventory` were previously disabled per code comment; they are now live.

Risks — products

1. ****createAdminProduct** misleading UI (Medium):** SECURITY DEFINER RPC does NOT honor the admin's `showOnWebsite` input on lifecycle status — product always lands as `draft`. The UI checkbox sets the flag but the product won't appear publicly until status flips to `active` via the drawer.
 2. **Image upload rollback gap (Low):** `uploadProductImage` rolls back the Storage object if the DB write fails — good. But the inverse (Storage upload succeeds, DB write succeeds, then network drops) leaves a dangling object. No orphan sweep exists.
 3. **Slug rename (Medium):** `updateAdminProduct` lets admin change `slug`. External links (SEO, bookmarks) will 404. No 301 redirect mechanism.
 4. ****updateAdminProductVariantPrices** non-atomic (Medium):** N separate UPDATES (one per size) — not transactional. A failure mid-loop leaves inconsistent prices. Should be an RPC.
 5. **No category create-from-product-drawer path** — admin must create the category first via the list page.
 6. ****Variant `stock_state`**** (`in_stock/low_stock/out_of_stock`) is read from `product_variants` but never written by admin. Presumably set by inventory/checkout flows; no admin UI to override.
 7. **Detail page** only exposes price editing — confusing UX.
-

B.4 /admin/orders + /admin/orders/[id]

Files:

- List: `src/app/admin/orders/page.tsx` (335 lines)
- Detail: `src/app/admin/orders/[id]/page.tsx` (258 lines)
- Drawer: `src/components/admin/orders/OrderDrawer.tsx` (202 lines) — wraps `OrderDetails` + `OrderFinancePanel`
- Service: `src/lib/admin/admin-orders.ts` (1,063 lines)
- Sub-components: `OrderDetails.tsx` (272), `OrderFinancePanel.tsx` (647), `OrderStatusBadge.tsx` (73)

List page

- Reads: `getAdminOrders()` (`admin-orders.ts:471`) — `orders` joined to `order_items(quantity)` for item count, newest first, ****limit(250)****.
- 6 KPI cards: Total / Pending / Preparing / Shipped / Delivered / Cancelled (counts filtered client-side from the loaded list, NOT separate queries).

- Status filter chips: all / pending / preparing / shipped / delivered / cancelled / returned (with counts).
- Search box filters by code / customerName / customerEmail / customerPhone (client-side).
- Columns: code, customer (name+email), phone, items, total, payment method+status, status badge, time, Manage button.
- Row action: opens **OrderDrawer**.

Order status enum + transitions

Defined in `admin-orders.ts:280-290` (`ALLOWED_ADMIN_ORDER_TRANSITIONS`):

From	Allowed →
pending	preparing, cancelled
preparing	shipped, cancelled
shipped	delivered
delivered	returned
cancelled	(terminal)
returned	(terminal)

Status update path: `updateAdminOrderStatus(orderId, nextStatus, note?)`
 (`admin-orders.ts:541`) → RPC `update_admin_order_status`.

Status flow + inventory effects (SQL migration

`20260627110000_admin_orders_status_inventory.sql`)

The RPC `update_admin_order_status` is SECURITY DEFINER, atomic, with `SELECT ... FOR UPDATE` on the order row.

Transition	Inventory effect
pending → preparing	None (no inventory movement).
pending → cancelled	<code>release</code> movement: <code>inventory_stock.available_kg += open_reservation, reserved_kg -= open_reservation</code> for each product. Writes <code>inventory_movements</code> row with <code>movement_type='release'</code> .
preparing → shipped	<code>deduct</code> movement: <code>reserved_kg -= open_reservation</code> (available_kg already reduced at checkout). Writes <code>inventory_movements</code> row <code>movement_type='deduct'</code> . This is the COGS-irreversible point.
preparing → cancelled	RELEASE (same as pending → cancelled).
shipped → delivered	None (inventory already deducted at ship) — but <code>orders.cogs_total + order_items.line_cogs</code> snapshot is finalized here.

Transition	Inventory effect
delivered → returned	No inventory effect in status RPC — returns are recorded separately via <code>record_order_return</code> (Phase 10-11).

Pre-condition: every stock-tracked product line must have an open reservation (sum of reserve minus release/deduct > 0). If not, the RPC raises `Inventory reservation is missing or already consumed`. and aborts — no status change.

Note: Checkout already did `reserve` (reduced `available_kg`, increased `reserved_kg`) when the order was placed. So admin status changes only *consume* the reservation (ship → deduct, cancel → release).

Detail page

- Header: code, status badge, placed_at, channel.
- **Status action card:** optional timeline note textarea (max 1,000 chars) + buttons for each allowed transition. Terminal statuses show "This order is in a terminal status." Calls `updateAdminOrderStatus` then reloads.
- **Delivery fee override** (only shown for `pending/preparing/shipped`): input + reason. Calls `updateAdminOrderDeliveryFee` → RPC `update_admin_order_delivery_fee` (only allowed before delivery). RPC recomputes total, sets `delivery_fee_overridden=true`, appends audit line to `admin_note`.
- ****OrderDetails**** — read-only: customer card (name, type, email/phone/WhatsApp links), address card (governorate/city/area/street/building/floor/apartment/landmark/Google Maps link), items table (kind, variant size, name en/ar, SKU, unit price, qty, returned qty, line total), timeline (`order_status_events` history with note + changed_by + changed_at), totals (subtotal, discount, delivery, total).
- ****OrderFinancePanel**** — see below.

OrderFinancePanel — payments / refunds / returns / notes

Reads `getAdminOrderFinancials(orderId, total)` (`admin-orders.ts:792`) — `order_payments`, `order_refunds`, `order_returns` (+ nested `order_return_items`).

Action	Function	RPC	Effect
Record Payment	<code>recordOrderPayment(orderId, amount, method, ref?, notes?)</code>	<code>record_order_payment</code>	Inserts <code>order_payments</code> row; cannot exceed order total; rejected on cancelled orders. Updates <code>orders.payment_status</code> .
Record Refund	<code>recordOrderRefund(...)</code>	<code>record_order_refund</code>	Inserts <code>order_refunds</code> row; cannot exceed paid—already-refunded. Subtotal/COGS/stock unchanged (cash-only movement).

Action	Function	RPC	Effect
Create Return	<code>recordOrderReturn(orderId, reason, notes, items[])</code>	<code>record_order_return</code>	Only for delivered or returned orders. Per-line quantity $\leq$ remaining returnable. Condition: sellable / damaged / other . Sellable returns restock through the order's original FIFO lots (coffee) or bean lots (espresso); flavor lines + damaged/other never restock. Does NOT refund money.
Save Admin Note	<code>updateAdminOrderNote(orderId, note)</code>	<code>update_admin_order_note</code>	Updates <code>orders.admin_note</code> (max 2,000 chars).

Payment summary shows: Order total, Paid, Remaining (`max(0, total - netPaid)`), Refunded. Warns when delivered with outstanding balance.

Disabled by design (per panel footer): "Item and price editing are intentionally disabled — they would corrupt FIFO, allocations, promo, or COGS. Use returns/refunds instead."

Mock/static/dead areas

- None. Every figure is computed from real ledgers.
- `OUTSTANDING_PAYMENT_STATUSES = ["unpaid", "partially_paid", "pending", "failed"]` — used by `getAdminOrderOverview()` (not currently surfaced in the list page KPIs).

Risks — orders

1. `**getAdminOrders limit(250) (High):**` beyond 250 orders, the list silently truncates with no pagination control. The KPI counts will be wrong. Critical for any real-volume store.
2. **No bulk status actions** — every order must be transitioned individually.
3. `**OrderDrawer` and the detail page duplicate the status-action UI`**`. Two code paths to maintain.
4. **Delivery fee override** has no audit trail beyond the `admin_note` append. No dedicated `order_delivery_fee_history` table.
5. **Returns:** the admin UI does not show which lots were restocked — only the total `restockedKg`. Verify the RPC logs the lot-level allocation.
6. `**canReturn = order.status === "delivered" || order.status === "returned"**` — but `ALLOWED_ADMIN_ORDER_TRANSITIONS.delivered = ["returned"]`. A returned order can have additional returns recorded, but no UI prevents a third+ return on the same line beyond the `quantity - returnedQuantity` check.
7. **No customer-facing notification** when status changes. The `order_status_events` row is written with `changed_by` (admin display name) but no email/WhatsApp is sent. (Telegram route exists at `/api/order-notifications/telegram/route.ts` but that's checkout-side only.)

B.5 /admin/inventory

Files:

- Page: `src/app/admin/inventory/page.tsx` (807 lines)
- Services: `src/lib/admin/admin-inventory.ts` (355), `src/lib/admin/admin-packaging.ts` (254), `src/lib/admin/admin-purchasing.ts` (875), `src/lib/admin/admin-espresso.ts` (199)
- Component: `src/components/admin/inventory/PackagingInventoryPanel.tsx`

6 tabs: `products` / `beans` / `packaging` / `movements` / `lots` / `suppliers`

KPI strip (4 cards)

Computed in `useMemo` (`inventory/page.tsx:448`):

- **KG available** = $\sum \text{products.availableKg}$
- **KG reserved** = $\sum \text{products.reservedKg}$
- **Low / out** = `#products where status !== "ok"`
- **Products tracked** = `data.products.length`

Products tab

`getAdminInventory()` (`admin-inventory.ts:155`) — joins `inventory_stock` + `products` + `categories` + last 100 `inventory_movements`.

Per product card: image, name, category, status badge (ok/low/out via `stockStatus(availableKg, thresholdKg)`), available/reserved/threshold kg, last movement hint.

Stock Movement modal (signed kg delta):

- Positive = add (creates an `adjustment` movement with optional `unit_cost`; the RPC `adjust_finished_product_stock` creates/reuses an `inventory_lots` row).
- Negative = remove (FIFO drawdown via the same RPC; rejects if `> available`).
- Reason/note required (max 500 chars).
- Calls `adjustFinishedProductStock(productId, delta, unitCost?, note?)` → RPC `adjust_finished_product_stock`.

Beans tab (espresso)

Reads `listEspressoBeans()` (`admin-espresso.ts:108`) — `espresso_beans` + `espresso_bean_stock`. Same Stock Movement modal but routes through `adjustEspressoBeanStock(beanId, delta, cost?, note?)` → RPC `adjust_espresso_bean_stock` (creates `espresso_bean_lots` on add, FIFO drawdown on remove).

Packaging tab

`PackagingInventoryPanel.tsx`. Reads `listPackagingItems()` (`admin-packaging.ts:120`) from `packaging_items` table. CRUD via:

- `savePackagingItem` → RPC `upsert_packaging_item` (fields: `operational_key`, `name`, `sku`, `packaging_kind`, `capacity_g`, `low_stock_threshold`, `active`, `cost_per_unit`, `notes`).
- `adjustPackagingStock(itemId, delta, unitCost?, note?)` → RPC `adjust_packaging_stock` (integer units only; writes `packaging_movements`, updates `packaging_items.available_quantity`). Rejects non-integer or zero deltas.
- `listPackagingMovements(itemId?)` — last 500 movements.

Status: "OK" / "Low" / "Out" computed client-side (`getPackagingStatus`).

The panel also exposes `listOrderPackagingRequirements(orderId)` and `listPackagingShortageOrders()` (`admin-packaging.ts:201, 228`) — the latter reads `orders.packaging_shortage = true` — **but the UI shown in scope does not render a shortage list. This data layer is wired but the shortage UI is missing.**

Movements tab

Last 100 rows from `inventory_movements` (`movement_type`, `quantity_kg`, `reason`, `metadata.direction`, `created_at`). Types: `initial_stock`, `reserve`, `release`, `deduct`, `adjustment`, `purchase_receive`.

FIFO Lots tab (Phase 5 migration)

`listInventoryLots()` from `admin-purchasing.ts` — `inventory_lots` table. Columns: Product, Received, Remaining, Reserved, Available, Source (purchase/opening/adjustment), Supplier, Date. Lots are read-only here; admin cannot delete or merge lots.

Suppliers tab

`listSuppliers()` + `SupplierModal` (create/edit). Fields: `name *`, `contact_name`, `phone`, `email`, `address`, `notes`, `status` (active/inactive). Validation: `name` ≤ 160 chars, `phone` 7-15 digits, `email` regex. Direct INSERT/UPDATE on `suppliers` table (RLS-gated).

Reservation / deduction / release model

- **Reserve:** at checkout (`create_checkout_order` RPC) — `inventory_stock.available_kg -= qty`, `reserved_kg += qty`, `inventory_movements(movement_type='reserve')`. Also writes `order_lot_allocations(status='reserved')`.
- **Release:** admin cancels order → `update_admin_order_status` RPC reverses the above.
- **Deduct:** admin ships order → `reserved_kg -= qty`, writes `inventory_movements(movement_type='deduct')`, flips `order_lot_allocations.status='deducted'`.
- **Restock:** `record_order_return` with `condition='sellable'` for coffee/espresso lines increments `inventory_stock.available_kg` (via lot allocation update). Damaged/other

conditions do NOT restock.

- **Packaging:** parallel system (`packaging_items`, `packaging_movements`, `order_packaging_lines`). Order checkout writes `packaging_requirements` and `packaging_snapshot` on the order; shortages flag `orders.packaging_shortage`.

Risks — inventory

1. `**getAdminInventory` limits movements to 100 (Medium): `**` long history is invisible.
 2. **No lot adjustment UI (Medium):** admin cannot split/merge/correct a lot. If a lot's `unit_cost` is wrong, the only fix is a SQL migration.
 3. `**adjustFinishedProductStock` for negative delta `**` rejects if it exceeds available — but the error message ("The adjustment exceeds available stock.") is generic; doesn't tell the admin *which* lot ran out.
 4. **Packaging shortage alerts not surfaced (Medium):** `listPackagingShortageOrders` exists but no UI calls it. Shortage orders would be invisible to the operator.
 5. **FIFO is server-enforced** in the RPCs; admin can't override the order in which lots are drawn down. Correct behavior, but no manual override for edge cases.
 6. `**suppliers` table direct INSERT/UPDATE `**` bypasses any RPC audit. Compare with `purchases` which use SECURITY DEFINER RPCs.
 7. **Stock movement "Reason / notes" is required** but there's no validation that the note is meaningful — admins can type "x".
-

B.6 /admin/marketing

Files:

- Page: `src/app/admin/marketing/page.tsx` (62 lines, tab switcher)
- Components: `PromoCodesPanel.tsx` (783), `AnnouncementsPanel.tsx` (528)
- Services: `src/lib/admin/admin-marketing.ts` (131), `src/lib/admin/admin-announcements.ts` (152)

Two tabs only: Promo Codes and Announcement Bar. Per code comment: "The fabricated Offers, Customer-Targeting, and Performance tabs (and their mock data) were removed."

Promo Codes

Reads: `listPromoCodes()` (`admin-marketing.ts:76`) — `promo_codes` + `promo_redemptions(count)`.

`**Fields (modal PromoCodesPanel.tsx:110+):**`

- Code * (2-32 chars uppercase `^[A-Z0-9][A-Z0-9_-]{1,31}$`, auto-uppercased)
- Status (active / inactive)
- Discount type (percentage / `fixed_amount`)
- Value * (positive; ≤ 100 if percentage)

- Minimum subtotal (≥ 0 , optional)
- Max discount (positive, optional; for percentage caps)
- Starts at / Ends at (datetime-local; ends > starts validated)
- Usage limit (positive integer, optional)
- Per-customer limit (positive integer, optional)
- Notes (optional)

Save: `savePromoCode(input)` → RPC `upsert_promo_code` (`admin-marketing.ts:91`). Errors mapped: 23505 → "A promo code with this word already exists."; 22023 → "Review the promo value, limits, and availability dates."

Deactivate: `deactivatePromoCode(id)` → RPC `deactivate_promo_code`.

Effective status computed client-side: Active / Inactive / Scheduled (starts_at future) / Expired (ends_at past).

Table columns: code, status, discount, min subtotal, max discount, dates, usage (used/limit), per-customer limit, actions (edit, copy code, deactivate).

Checkout integration: discount applies to product subtotal only — delivery is never discounted.

Announcement Bar

Reads: `listAnnouncements()` (`admin-announcements.ts:85`) — `announcements` table.

****Fields** (modal `AnnouncementsPanel.tsx:47+`):**

- Message (English) * ≤ 200 chars
- Message (Arabic) * ≤ 200 chars
- Button label (English) default "Shop now"
- Button label (Arabic) default "نآل قوسـت"
- Button link * (internal path only, regex `^/[A-Za-z0-9/_-]*$`; defaults to `/products`)
- Sort order (integer; lower shows first)
- Active checkbox

Save: `saveAnnouncement` — direct INSERT or UPDATE on `announcements` (RLS `announcements_admin_all = is_admin()` gates writes).

Toggle active: `setAnnouncementActive(id, bool)`.

Delete: `deleteAnnouncement(id)` — hard delete.

Public reach

- **Promo codes:** validated at checkout via `validate_promo_code` / `apply_promo_code` RPCs. Redemptions recorded in `promo_redemptions`.
- **Announcements:** `src/lib/content/announcements.ts` → `getPublicAnnouncements()` reads `announcements.active = true` ordered by `sort_order` then `created_at`. Consumed by `PublicHeader.tsx` (lines 793–865): rotates through active announcements every few seconds, shows bar above main header. ****Falls back to `DEFAULT_ANNOUNCEMENTS`**** (2 built-in launch messages) if table empty or

unreachable — so the bar is never blank, but admins who delete all announcements will see the *static fallback* on the public site, not their own content. Intentional but worth documenting.

Risks — marketing

1. **No promo redemption detail view (Medium):** admin sees only a `usedCount` aggregate. Cannot see *which* orders redeemed a code without querying `promo_redemptions` manually.
2. **Announcement rotation interval** is hardcoded in `PublicHeader.tsx` (every few seconds). Not admin-controllable.
3. ****DEFAULT_ANNOUNCEMENTS fallback (Low):**** could mislead an admin who deletes all rows and expects an empty bar — they'll see the launch messages instead. No admin UI surfaces this fallback behavior.
4. **Promo code "Max discount"** is only meaningful for percentage codes. The UI shows it for both types but `fixed_amount` codes silently ignore it.
5. **No date-based activation cron** — inactive→active transitions based on `starts_at` are computed at checkout validation, not by a job. Consistent between RPC and UI, but worth confirming.
6. **Hard delete on announcements (Low):** no audit trail. A deleted announcement cannot be recovered.

B.7 /admin/analytics

Files:

- Page: `src/app/admin/analytics/page.tsx` (1,113 lines)
- Service: `src/lib/admin/admin-analytics.ts` (708 lines)

6 tabs: `overview` / `sales` / `products` / `customers` / `marketing` / `geography`

Aggregator

`getAdminAnalytics()` (`admin-analytics.ts:306`) — 11 parallel queries:

- `orders` (5,000 cap), `order_items` (12,000 cap, joined `!inner` to `orders` for status filter), `order_payments` (8,000), `order_refunds` (8,000), `customers` (5,000), `products` (2,000), `categories` (500), `promo_codes` (2,000), `promo_redemptions` (5,000), `reviews` (5,000), `contact_messages` (5,000).

KPIs

KPI	Formula	File:function
Sales (gross)	Σ <code>orders.total</code> excluding cancelled	<code>admin-analytics.ts:411-415, 481</code>
Valid orders	count excluding cancelled	same
AOV	<code>salesTotal</code> / <code>validOrders</code>	<code>:443</code>

KPI	Formula	File:function
Net Collected	$\Sigma \text{ payments} - \Sigma \text{ refunds}$	:465-467
Delivered rate	$\text{deliveredCount} / \text{ordersTotal} * 100$	:446
Cancelled rate	same logic	:447
Returned rate	same	:448
Total customers	<code>customers.length</code>	:493
Registered / guest	<code>customers.type</code> count	:494-501
Repeat customers	<code>custAgg.values().filter(orders $\geq$ 2).length</code>	:505
New customers 30d	<code>countWindow(joined_at, last 30d)</code>	:502
Avg orders/customer	<code>ordersWithCustomer / orderingCustomers</code>	:508
Top customers	by spend DESC, top 8	:511-524
Top products (by sold)	from <code>order_items</code> excluding cancelled	:531+
Top products (by revenue)	same, sorted by revenue	same
Categories	$\Sigma \text{ line_total} + \Sigma \text{ quantity}$ grouped by category_slug	same
Promo usage total	<code>promo_redemptions.length</code>	later in file
Promo discount total	$\Sigma \text{ promo_redemptions.discount_amount}$	same
Active promo count	<code>promo_codes.status='active'</code>	same
Promo performance	per-code: uses, discountGiven, revenue (sum of order totals that redeemed)	same
Reviews approved/pending/rejected/total	<code>reviews.status</code> counts	:341 query
Contact new/in-progress/replied/archived/total	<code>contact_messages.status</code> counts	same
Geography	per-governorate: orders, revenue, customers, AOV, repeat rate	:397-438, 525+

Trend series

`buildTrend(sales, orderEvents, now)` (admin-analytics.ts:250) — same week/month/year buckets as dashboard but with both revenue AND order count per bucket. Rendered as dual-bar chart (`TrendChart`).

Period-over-period (30d vs prior 30d)

`salesTrend30d`, `ordersTrend30d`, `aovTrend30d`, `netCollectedTrend30d`, `newCustomersTrend30d` — all via `trendPct(cur, prev)`.

Mock/static/dead areas

- **trafficTrackingConnected: false** is a literal type flag (`admin-analytics.ts:174`). The UI shows an honest "not connected" state for web traffic/conversion metrics — explicitly documented: "There is no analytics/tracking backend for behavioural web metrics."
- All other numbers are real.
- Tabs **geography** shows real governorates from `orders.governorate`; "Unspecified" bucket for nulls.

Risks — analytics

1. **Same scan-cap issue as dashboard (High):** 5,000 order cap. Beyond that, analytics silently truncates.
 2. **items** join uses `orders!inner(status)` — that's correct for filtering cancelled items, but it means items whose parent order was deleted (shouldn't happen, but...) disappear.
 3. **Promo revenue** = sum of order totals that redeemed the code — this counts full order revenue, not just the discounted product revenue. Standard but worth noting.
 4. **Geography "repeat rate"** = % of that area's customers with 2+ orders overall (not in that area). Could mislead.
 5. **No date range picker (Medium):** the only period selector is on the trend chart (week/month/year). KPIs are all-time.
 6. **No export to CSV (Low):** admins can't pull raw data.
-

B.8 /admin/cms

Files:

- Page: `src/app/admin/cms/page.tsx` (1,614 lines)
- Service: `src/lib/admin/admin-cms.ts` (496 lines)
- Author display: `getAdminDisplayName()` from `src/lib/auth/admin.ts`

4 tabs: `blog` / `reviews` / `legal` / `contact`

Blog tab

- Reads: `getAdminCmsData()` (`admin-cms.ts:369`) → `blog_posts` (500 limit) ordered by `updated_at DESC`.
- Article fields: slug, title (en/ar), excerpt (en/ar), content (en/ar), category (en/ar), author, status (Draft/Published/Archived), featured, views, publishDate, readTime (en/ar), tags (array of {en, ar}), heroImage, cardImage, featuredImage, seoTitle (en/ar), seoDescription (en/ar).
- **Image options:** `CMS_IMAGE_OPTIONS` array (`admin-cms.ts:92`) — 6 static asset paths.
⚠ **No upload** — admins pick from these presets only.
- **Save:** `saveAdminArticle(article)` → RPC `save_admin_blog_post` (`admin-cms.ts:394`). Handles slug uniqueness (`blog_posts_slug_key`).

- Article statuses (`ARTICLE_STATUSES`): Draft / Published / Archived.
- "New article" creates a `crypto.randomUUID()` client-side, then saves via the RPC.

Reviews tab

- Reads `reviews` table (1,000 limit).
- Review fields: customer name, product name, productSlug, rating (1-5), reviewText (en/ar), source (Manual/WhatsApp/Facebook/Instagram/Website), status (Pending/Approved/Rejected), featured, hidden, showOn (Product Page / Homepage Testimonials / Both), date, publishedAt.
- **Save:** `saveAdminReview(review)` → RPC `save_admin_review` (`admin-cms.ts:429`). Maps UI enum back to DB enum (`homepage/both/product`).
- Filters: Pending / Approved / Rejected / Featured.

Legal Pages tab

- 4 fixed page types: privacy / terms / shipping / returns.
- Fields: title (en/ar), content (en/ar), status (Draft/Published), version (semver-ish, bumped via `bumpVersion()` in page), publishedAt.
- **Save:** `saveAdminLegalPage(page)` → RPC `save_admin_legal_page` (`admin-cms.ts:460`).

⚠ **NOTE:** Despite the `legal_pages` table being live and editable here, the public `/privacy`, `/terms`, `/shipping`, `/returns` pages (`src/app/(public)/*/page.tsx`) read from **hardcoded `STATIC` arrays**, NOT from `legal_pages`. This means admin edits to legal pages **do not reach the public site**. Same disconnect pattern as espresso/flavor managers (see B.12).

Contact Messages tab

- Reads `contact_messages` (1,000 limit).
- Per-message fields: name, phone, whatsapp, email, source, subject, message, date, status (New / In Progress / Replied / Archived), assignedAdmin (derived from `assigned_admin_id`), internalNotes.
- **Save:** `saveAdminContactMessage(message)` → RPC `update_admin_contact_message` (`admin-cms.ts:479`). Updates only `status` and `admin_note`. ⚠ **No reply-by-email feature** — admin must use external WhatsApp/email.

Public reach

- Blog: `src/app/(public)/blog/[slug]/page.tsx` reads `blog_posts` where `status='published'`. ☐ REAL
- Reviews: `src/lib/cms/public-cms.ts:34 listApprovedHomepageReviews()` reads `reviews` where `status='approved'` AND `hidden=false` AND `show_on IN ('homepage', 'both')`. ☐ REAL
- Legal: `/terms`, `/privacy`, `/shipping`, `/returns` pages read **hardcoded arrays**, NOT `legal_pages`. ⚠ **DISCONNECT** (admin edits don't propagate)
- Contact: `submitContactMessage` writes via `create_contact_message` RPC. ☐ REAL

Risks — CMS

1. **No blog image upload (Medium):** admins are stuck with 6 preset images. Major content limitation.
 2. **No WYSIWYG / markdown editor:** content is plain textareas.
 3. **Review moderation** has no bulk actions.
 4. **Contact messages have no reply workflow (Medium):** admin can only change status + add an internal note. The customer is never notified.
 5. **Legal page versioning is cosmetic** — no history table, no diff view. The `version` field is just a string.
 6. ****assigned_admin_id** is never set by the UI** — it's read-only display. The assignment flow is missing.
 7. ****views** counter** on blog posts is read but there's no increment path visible in the admin code (likely a public-side RPC or trigger).
 8. **Legal pages disconnect (P1):** admin edits don't reach public pages — same pattern as espresso/flavor managers.
-

B.9 /admin/customers

Files:

- Page: `src/app/admin/customers/page.tsx` (614 lines)
- Drawer: `src/components/admin/customers/CustomerDrawer.tsx` (752 lines)
- Service: `src/lib/admin/admin-customers.ts` (623 lines)

List

- Reads `getAdminCustomers()` (`admin-customers.ts:296`) — **customers (1,000 limit)** + **orders** (5,000 limit, aggregated per customer).
- Per-customer: `id`, `authUserId`, `name`, `email`, `phone`, `whatsapp`, `type` (registered/guest), `status` (active/inactive/blocked), `marketingOptIn`, `tags`, `joinedAt`, `ordersCount`, `totalSpent` (excl. cancelled), `lastOrderDate/Status/Code`, `orderCodes`, `daysSinceJoined`, `daysSinceLastOrder`.
- Filters: `all` / `registered` / `guest` / `vip` / `repeat` / `new` / `inactive` / `at-risk` / `wholesale`.
- Sort: `most-spent` / `most-orders` / `recently-active` / `oldest-inactive`.
- Search by `name`/`phone`/`email`.
- KPI strip (4 cards): `total customers`, `registered`, `guests`, `repeat customers` — all computed client-side from the loaded list.
- Row action: opens `CustomerDrawer`.

Segments (computed, not stored)

`getCustomerSegments(c)` (`admin-customers.ts:565`):

- **VIP:** `totalSpent ≥ 5,000 EGP` OR `ordersCount ≥ 8`
- **Repeat:** `ordersCount ≥ 2` (and not VIP)

- **New:** `ordersCount ≤ 1 AND daysSinceJoined ≤ 30`
- **Inactive:** `lastOrder > 90 days ago`
- **At-risk:** `ordersCount ≥ 2 AND lastOrder 60–90 days ago`
- **Wholesale-potential:** tags includes "Wholesale Potential"

`getCustomerLifecycleStatus(c)`: active / inactive / new (similar thresholds).

`getSuggestedPromotion(c)`: text hint based on segment (e.g. "Win-back campaign candidate" for inactive).

CustomerDrawer

Reads `getAdminCustomerDetail(customerId)` (`admin-customers.ts:370`) — customer + addresses (default first) + orders (300 limit) + payments/refunds totals + activity events (300 limit, `order_status_events` + `account-created` synthetic event).

Only editable field: TAGS. `updateAdminCustomerTags(customerId, tags)` (`admin-customers.ts:533`) — column-scoped grant on `tags` only. Max 20 tags, 40 chars each, deduplicated.

Predefined tags offered in the drawer (`PREDEFINED_TAGS` array in `CustomerDrawer.tsx:120`): includes "Wholesale Potential" (which feeds segment classification), VIP, Repeat Buyer, Inactive, At-Risk, etc.

What admin CANNOT edit

- Customer name, email, phone, whatsapp, type, status (active/inactive/blocked) — **all read-only**. The `status` column exists in the DB but no admin UI writes it.
- Addresses — read-only display.
- Orders — read-only; the drawer links out to `/admin/orders/{id}`.

Risks — customers

1. **No way to block a customer via UI (Medium):** DB has `status='blocked'` but no admin action. A misbehaving customer can only be blocked via direct SQL.
2. **No order editing from the customer drawer** — admins must navigate to `/admin/orders/{id}` separately.
3. **Tag-based "Wholesale Potential"** is the only tag that affects business logic (segment). Mislabeling could trigger incorrect B2B outreach.
4. **Duplicate customer detection** — the list shows a warning triangle (`isDuplicate` prop) but no merge UI to resolve duplicates.
5. ****getAdminCustomers caps at 1,000 (High):**** beyond that, no pagination.
6. **Customer activity event limit 300 (Low):** long-time customers will have truncated history with no warning beyond `ordersTruncated`.

B.10 /admin/accounting

Files:

- Page: `src/app/admin/accounting/page.tsx` (2,492 lines)
- Service: `src/lib/admin/admin-accounting.ts` (701 lines) + `src/lib/admin/admin-purchasing.ts` (875 lines)

6 tabs: `overview` / `revenue` / `purchases` / `expenses` / `suppliers` / `activity`

Aggregator

`getAdminAccounting()` (`admin-accounting.ts:307`) — 8 parallel queries:

- `orders` (5,000) with `cogs_total`
- `order_payments` (8,000), `order_refunds` (8,000)
- `order_returns` (5,000)
- `expenses` (2,000)
- `purchases` (2,000)
- `supplier_payments` (5,000)
- `suppliers` (2,000)

KPI definitions (per code header `admin-accounting.ts:13-32`)

KPI	Formula	File:function
Sales (gross)	Σ <code>orders.total</code> excluding cancelled	:400-439
Product subtotal	Σ <code>orders.subtotal</code> excluding cancelled	same
Discounts total	Σ <code>orders.discount_total</code> excluding cancelled	same
Delivery fees total	Σ <code>orders.delivery_fee</code> excluding cancelled	same
Net product sales	<code>productSubtotal</code> – <code>discountsTotal</code>	:481
Delivered net sales	Σ (<code>subtotal</code> – <code>discount</code>) for delivered orders only	:419-457
COGS total	Σ <code>orders.cogs_total</code> for delivered orders only (NULL treated as 0, counted in <code>deliveredMissingCogs</code>)	:445-457
Gross profit	<code>deliveredNetSales</code> – <code>cogsTotal</code> (delivered basis)	:482
Gross margin %	<code>grossProfit</code> / <code>deliveredNetSales</code> * 100	:483
Paid total	Σ <code>order_payments.amount</code>	:486
Refunded total	Σ <code>order_refunds.amount</code>	:487
Net collected	<code>paidTotal</code> – <code>refundedTotal</code>	:488

KPI	Formula	File:function
Receivable	$\Sigma \max(\text{total} - \text{netPaid}, 0)$ for non-cancelled orders	:409, 426
Operating expenses	$\Sigma \text{expenses.amount}$	:520-533
Net profit	$\text{grossProfit} - \text{operatingExpenses}$ (purchases NOT included — they're inventory/cost basis)	:535
Total purchases	$\Sigma \text{purchases.total_amount}$ for non-cancelled	:538-573
Paid to suppliers	$\Sigma \text{purchases.paid_amount}$ for non-cancelled	same
Supplier payable	$\Sigma \max(\text{total} - \text{paid}, 0)$ for non-cancelled	same
Returns count	$\text{order_returns.length}$	:587
Restocked kg	$\Sigma \text{order_returns.restocked_kg}$	same

Methodology notes (from header):

- COGS is from `orders.cogs_total` snapshot at delivery time — never recomputed from current stock costs.
- Supplier purchases are NOT P&L expenses (they're inventory/cost basis).
- Returns/refunds never rewrite historical sales or COGS.
- "No opening-cash fiction" — no invented cash position.
- `deliveredMissingCogs` count surfaces orders where `cogs_total` IS NULL.

Per-order row (Revenue tab)

`AccountingOrderRow` (`admin-accounting.ts:69`): code, customer, status, placedAt, subtotal, discount, deliveryFee, total, netSales (subtotal–discount), cogs (nullable), grossProfit (nullable), margin (nullable), netPaid, outstanding. Limited to 60 rows (`RECENT_ORDERS_DISPLAY`).

Activity / transactions timeline

Combined timeline of last 50 transactions (`RECENT_TRANSACTIONS_LIMIT`): payments (in), refunds (out), expenses (out), purchases (neutral — "inventory / cost basis, not a P&L expense"), supplier payments (out), returns (neutral — "Does not rewrite sales/COGS"). Each entry has label, detail, amount, direction.

Monthly trend

6-month buckets (`TREND_MONTHS=6`): revenue, collections (payments — refunds in that month), expenses, grossProfit. Rendered as chart in overview tab.

Payment method breakdown

From `order_payments.method` normalized to `cash` / `bank_transfer` / `mobile_wallet` / `other`. Amount + count per method.

Write actions (from the page)

Action	Function	RPC/Table	Effect
Add Expense	<code>createExpense(input)</code> (admin-purchasing.ts)	RPC <code>create_expense</code>	Inserts <code>expenses</code> row (date, category, amount, method, notes). Categories: Rent/Utilities/Delivery/Marketing/Payroll/Maintenance/Tools/Packaging Design/Other. Methods: Cash/Bank Transfer/Card/Vodafone Cash.
Create Purchase	<code>createPurchase(input)</code>	RPC <code>create_purchase</code>	Draft purchase with line items (productId, quantityKg, unitCost). Server computes totals.
Receive Purchase	<code>receivePurchase(purchaseId)</code>	RPC <code>receive_purchase</code>	Only drafts can be received. Creates <code>inventory_lots</code> + <code>inventory_movements</code> (movement_type='purchase_receive') + increments <code>inventory_stock.available_kg</code> . Atomic. Returns <code>lotsCreated</code> + <code>totalKg</code> .
Record Supplier Payment	<code>recordPurchasePayment(input)</code>	RPC <code>record_purchase_payment</code>	Inserts <code>supplier_payments</code> row; updates <code>purchases.paid_amount</code> + <code>payment_status</code> .
Create/Edit Supplier	<code>createSupplier/updateSupplier</code>	Direct INSERT/UPDATE on <code>suppliers</code>	RLS-gated.

Risks — accounting

- 5,000 order cap (High):** beyond that, all P&L figures truncate silently. Critical for financial accuracy.
- COGS snapshot dependency (Medium):** `orders.cogs_total` is set at delivery time. If a delivery happened before Phase 5 migration, `cogs_total` is NULL and `deliveredMissingCogs` increments — gross profit is understated. There's no admin UI to backfill.
- **netProfit = grossProfit - operatingExpenses ignores supplier payments (correct) but also ignores unpaid receivables (correct) and taxes**** (no tax concept exists). The figure is "operating profit before tax", not net in the accounting sense.
- No CSV export (Low):** admins can't pull data for an accountant.
- No date range filter (Medium):** the only time window is the 6-month trend chart.

6. **Purchase receive is irreversible (Medium):** once a purchase is `received`, the lots and stock movements are committed. No "unreceive" path.
7. **Supplier payment method** is a free-text string in `supplier_payments.method` — no validation against a fixed set. Inconsistent with `order_payments.method` which is normalized.
8. **`recordPurchasePayment`** allows overpayment** (no `paid > total` check visible — verify in the RPC).
9. **Receivable calculation** uses `total - netPaid` per order, summed. If a refund exceeds payments (negative netPaid), `max(0, ...)` clamps it — correct, but `netCollected = paidTotal - refundedTotal` could go negative without clamping. Edge case worth checking.

B.11 /admin/settings

Files:

- Page: `src/app/admin/settings/page.tsx` (478 lines)
- Service: `src/lib/admin/admin-settings.ts` (347 lines)

Settings shape

4 sections, each persisted as a single `site_settings` row (key/value jsonb, `scope='public'`, `is_public=true`):

Key	Fields
<code>brand</code>	<code>storeName</code> , <code>defaultCurrency</code> (always uppercased)
<code>contact</code>	<code>supportEmail</code> , <code>supportPhone</code> , <code>whatsappNumber</code> , <code>businessAddress</code>
<code>social_links</code>	<code>facebook</code> , <code>instagram</code> , <code>tiktok</code> , <code>youtube</code> , <code>whatsapp</code>
<code>storefront</code>	<code>storeOpen</code> (bool), <code>closedNotice</code> (text)

Read/write

- `getAdminSettings()` (`admin-settings.ts:163`) — `SELECT key, value FROM site_settings WHERE key IN (...)`. No `scope/is_public` filter on the admin read (RLS still gates).
- `saveAdminSettings(next)` (`admin-settings.ts:292`) — single `upsert` of all 4 rows with `onConflict: "key"`. Values trimmed. Brand name defaults to "Line Coffee" if blank; currency defaults to "EGP" if blank.

Public reach

- `getPublicSettings()` (`admin-settings.ts:187`) — adds `scope='public' AND is_public=true` filter (defense in depth).
- **PublicHeader.tsx** (lines 793–865): loads `storefront` via `getPublicSettings()`. If `storeOpen=false`, shows the `closedNotice` as a forced announcement bar and

auto-shows the announcement banner.

- **PublicFooter.tsx** (lines 68–224): uses `settings.contact.whatsappNumber` (via `resolvePublicPhone`), `supportEmail` (via `toEmailHref`), `businessAddress`, `brand.storeName`, and social links.
- Phone numbers go through `toInternationalPhoneDigits` — converts `01XXXXXXXXXX` to `20XXXXXXXXXX` for tel: and wa.me: links. Placeholder phones (`01000000000`, etc.) are filtered out.

What's NOT editable

- **Delivery fees** — explicitly NOT editable here (per code comment: "The delivery fee is still computed by `resolve_delivery_fee()` in SQL, so it is deliberately NOT surfaced as an editable field here (editing it would be fake).")
- **Admin user emails / credentials** — never written here (live in `admin_users`).
- **Order blocking on store-closed** — `storeOpen=false` shows the notice but does NOT block checkout (**per code comment: "atomic order blocking remains unenforced until the checkout RPC can own that rule safely"**). Major operational risk (P1). ** NOTE: This contradicts the audit-team's earlier finding — the checkout RPC `_create_checkout_order_phase67` DOES read `site_settings.storefront->'storeOpen'` and raises if `false` (per Part E §1.4.1). The truth depends on **which migrations are actually applied** to the production Supabase project. ⚠ Owner must confirm — see Part E F12.

Risks — settings

1. **Store-closed does not block orders (P1):** major operational risk. A customer can still complete checkout while the store is "closed". The notice is cosmetic only. **Verify migration applied.**
2. **No validation on social links** — admins can enter arbitrary strings. `toPublicHttpUrl` validates protocol at render time but doesn't prevent saving junk.
3. **No validation on phone format** — `toInternationalPhoneDigits` is permissive (8–15 digits). Egyptian format assumed but not enforced.
4. **No multi-currency support (P2):** `defaultCurrency` is stored but the public site hardcodes EGP everywhere. Setting it to "USD" would not change display.
5. **No audit log** — settings changes overwrite the row; previous values are lost.
6. **Single key upsert** — if one of the 4 rows fails validation server-side, the whole `upsert` fails atomically (good), but the error message is generic.

B.12 /admin/espresso-manager + /admin/flavor-manager

Files:

- Espresso page: `src/app/admin/espresso-manager/page.tsx` (380 lines)
- Flavor page: `src/app/admin/flavor-manager/page.tsx` (406 lines)
- Services: `src/lib/admin/admin-espresso.ts` (199),
`src/lib/admin/admin-flavor.ts` (215)

Espresso Manager

- Reads `listEspressoBeans()` (`admin-espresso.ts:108`) — `espresso_beans` + `espresso_bean_stock`.
- KPIs (4): total beans, active, arabica count, robusta count.
- Per-bean card: name (en/ar), origin (en/ar), family badge, stock (available/reserved/threshold kg), status badge.
- **BeanEditor** (modal, edit only — no create): nameEn/Ar *, originEn/Ar, tasteHintEn/Ar, family (arabica/robusta), sortOrder (integer), salePricePerKg (≥ 0), purchaseCostPerKg (≥ 0 , optional), active checkbox.
- **Save:** `upsertEspressoBean(input)` → RPC `upsert_espresso_bean` (`admin-espresso.ts:149`). `metrics` (the blend-engine tuning numbers) are preserved unchanged — comment: "Blend metrics are preserved unchanged. Customer pricing and ratio formulas are not modified by this editor."
- Stock movements route through `/admin/inventory` (Beans tab) using `adjustEspressoBeanStock` → RPC `adjust_espresso_bean_stock`.

Flavor Manager

- Reads `listFlavorBases()` + `listFlavorItems()` (`admin-flavor.ts:130, 152`) — `flavor_bases` and `flavor_items` tables.
- KPIs (4): flavor bases count, active bases, flavor add-ons count, active flavors.
- 2 tabs: Bases / Flavors.
- **CatalogEditor** (modal, edit only — no create): nameEn/Ar *, hintEn/Ar, price (base price/kg OR add-on/kg depending on target), cost/kg (optional), category (only for flavors: chocolate/fruits/nuts/desserts/coffee-shisha), sortOrder, active.
- **Save:** `upsertFlavorBase` → RPC `upsert_flavor_base`, OR `upsertFlavorItem` → RPC `upsert_flavor_item` (`admin-flavor.ts:176, 195`). Flavor item `metrics` preserved unchanged.
- Comment: "This editor updates the existing flavor catalog only. It does not alter checkout formulas or add French Coffee to the builder."

⚠ Persistence model — CRITICAL DISCONNECT

Both managers persist to real Supabase tables (`espresso_beans`, `flavor_bases`, `flavor_items`) via SECURITY DEFINER RPCs. They are NOT static files.

However, the **public builder UIs**

(`src/features/website/make-your-espresso/data/espressoBeans.ts`, `src/features/website/make-your-flavor/data/flavorData.ts`) — these are **static TypeScript data files** used by the public blend/flavor studios. There is **NO live read** from `espresso_beans/flavor_items` in the public builder.

This is a **critical disconnect**:

- Admin edits `espresso_beans.name_en = "New Name"` → persists to Supabase.
- Public EspressoBlendStudio reads from `espressoBeans.ts` static array → still shows the old name.
- Same for flavor.

The admin managers exist and persist correctly, but their edits **do not reach the public site** unless there's a separate sync (none found in scope). The admin UI subtitle says "Real espresso_beans catalog and espresso_bean_stock balances. All edits persist." — true, but misleading because the public site doesn't consume those tables.

Same disconnect applies to: CMS legal pages (admin edits `legal_pages` table, but public pages read hardcoded STATIC arrays).

Risks — espresso/flavor managers

1. **Critical (P1): admin edits don't propagate to the public builder.** The public `EspressoBlendStudio` and `FlavorMixStudio` read static `.ts` files. Either:
 - (a) The public builders should be refactored to read from `espresso_beans` / `flavor_items` at runtime, OR
 - (b) A sync script should regenerate the `.ts` files from the DB after admin edits.
2. **No create flow** — admins can only edit existing beans/flavors. Adding a new bean requires a SQL seed or migration.
3. **No delete/deactivate cascade** — setting `active=false` hides the bean from the admin "Active" count, but the public builder (which reads static files) is unaffected. If the public builder ever switches to DB reads, inactive beans should be filtered out.
4. ****metrics** is a JSONB blob** — admin can't edit it. Blend engine tuning requires direct SQL.
5. **No flavor category CRUD** — the 5 categories (chocolate/fruits/nuts/desserts/coffee-shisha) are hardcoded.
6. **No cost history** — `purchase_cost_per_kg` is a single current value; no history of how it changed.

B.13 Cross-cutting findings — admin

Auth & RLS

- Every admin write is gated by `is_admin()` RLS policies (per-table `*_admin_all` policies).
- SECURITY DEFINER RPCs (`create_admin_product`, `update_admin_order_status`, `record_order_payment`, `upsert_espresso_bean`, `upsert_flavor_base`, `upsert_packaging_item`, `upsert_promo_code`, `save_admin_blog_post`, etc.) re-check `is_admin()` inside the function body — defense in depth.
- The client-side `useCurrentAdmin` hook is the UX gate; RLS is the security gate. Even if the hook is bypassed, RLS blocks the writes.

Real vs mock summary

Module	Real data?	Mock/static elements
Dashboard	☐ Real	<code>QuickActions</code> is static navigation (intentional)

Module	Real data?	Mock/static elements
Products	☐ Real	<code>fallbackCategoryImages</code> static asset map (intentional)
Orders	☐ Real	None
Inventory	☐ Real	None
Packaging	☐ Real	None (but shortage-orders UI is missing)
Marketing	☐ Real	None
Analytics	☐ Real	<code>trafficTrackingConnected: false</code> (honest "not connected")
CMS	☐ Real	<code>CMS_IMAGE_OPTIONS</code> 6 preset images (no upload)
Customers	☐ Real	Segments computed (not stored — intentional)
Accounting	☐ Real	None
Settings	☐ Real	None
Espresso Manager	☐ Real (DB)	⚠ Public builder reads static <code>.ts</code> files — edits don't propagate
Flavor Manager	☐ Real (DB)	⚠ Same as above
CMS Legal pages	☐ Real (DB)	⚠ Public legal pages read STATIC arrays — edits don't propagate

Top-priority admin fixes

1. **Espresso/Flavor/Legal disconnect (P1):** admin edits persist but public builders/pages ignore them. Refactor public builders to read from DB OR add a sync script.
2. **Store-closed blocking (P1):** verify whether `create_checkout_order` RPC actually enforces `storeOpen=false`. If not, add the check.
3. **Scan-cap truncation (P1):** 5,000-order / 1,000-customer / 250-order-list caps silently truncate. Affects dashboard, analytics, accounting, orders list, customers list.
4. **Packaging shortage alerts not surfaced (P2):** `listPackagingShortageOrders` exists but no UI calls it.
5. **No blog image upload (P2):** admins limited to 6 preset images.
6. **Customer block action missing (P2):** DB supports `status='blocked'` but no admin UI.
7. ****updateAdminProductVariantPrices** non-atomic (P2):** N separate UPDATES, no transaction.
8. ****OVERDUE_PREP_HOURS = 48** hardcoded (P3):** should be a `site_settings` key.
9. **No CSV export (P3):** in accounting/analytics.
10. ****Announcement DEFAULT_ANNOUNCEMENTS** fallback (P3):** deleting all rows shows static launch messages, not an empty bar.
11. ****Detail page /admin/products/[slug]** only edits prices (P2):** confusing UX — back button uses `router.back()` which dead-ends if user landed directly via URL.

12. **Order drawer + detail page duplicate status-action UI (P3):** two code paths to maintain.
 13. **No customer-facing notification on status change (P2):** `order_status_events` row written but no email/WhatsApp sent.
 14. ****Customer `assigned_admin_id` never set by UI (P3):**** assignment flow missing.
 15. **Suppliers direct INSERT/UPDATE (P3):** bypasses RPC audit unlike purchases.
-

Part C — Data Flow / System Map

C.0 Architectural overview

Line Coffee V3 — Architectural Overview

Browser-only Supabase app. Trust boundary is the database, not the client.

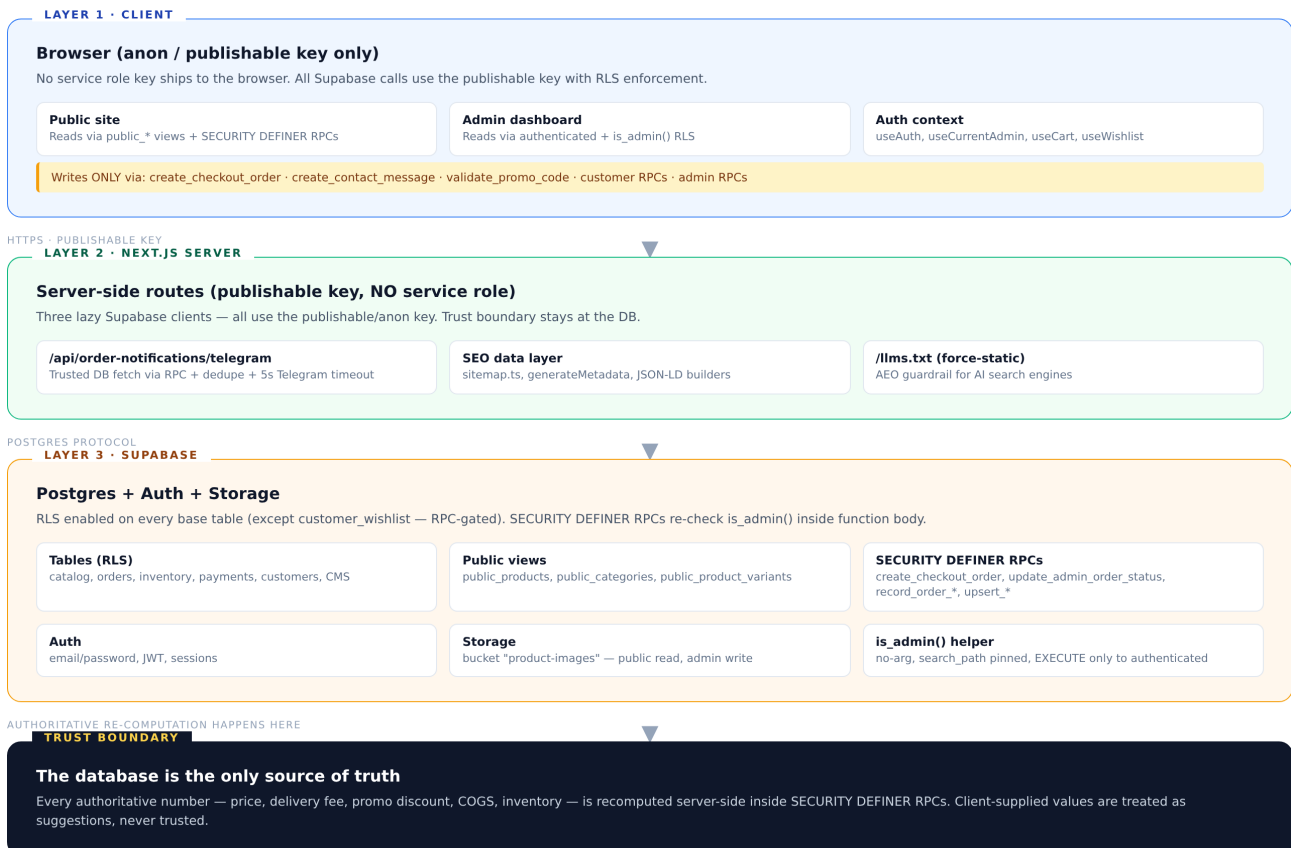
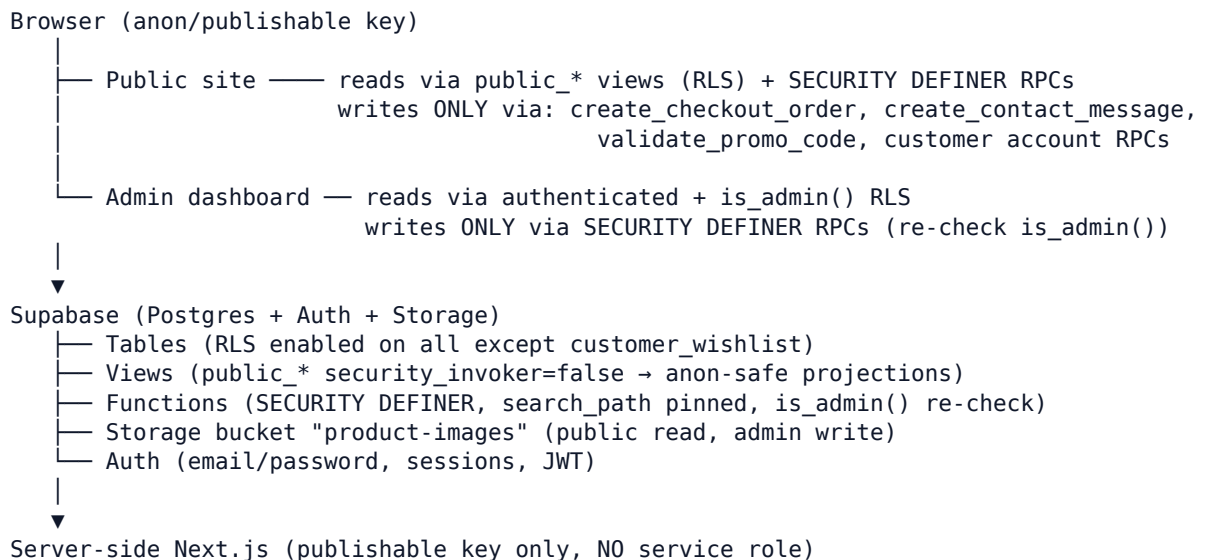


Figure 1 — Architectural overview: 3-layer trust model. The database is the only source of truth.



```
└─ /api/order-notifications/telegram → calls Telegram Bot API
└─ SEO data layer (sitemap, generateMetadata, JSON-LD)
└─ llms.txt (force-static)
```

Trust boundary: the database. Every authoritative number (price, delivery fee, promo discount, COGS, inventory) is recomputed server-side inside SECURITY DEFINER RPCs. Client-supplied values are treated as suggestions, never trusted.

C.1 Product lifecycle

Admin: ProductCreateDrawer

- createAdminProduct
- RPC create_admin_product (forces status=draft, visibility=hidden)
- products table
- product_variants table (3 rows: 250g/500g/1kg)

Admin: ProductDrawer (6 tabs)

- general: updateAdminProduct → products.name_en/ar, notes_en/ar, slug, seo_*
- pricing: updateAdminProductVariantPrices → product_variants.price (N separate UPDATES ▲ non-)
- media: uploadProductImage → storage product-images + products.gallery jsonb
- inventory: updateProductLowStockThreshold → inventory_stock.low_stock_threshold_kg
- visibility: toggle status=active/hidden, visibility=public/hidden, show_on_website=true/false
- seo: products.seo_title_en/ar, seo_description_en/ar

Admin: archiveAdminProduct → status=archived, visibility=hidden, show_on_website=false

Admin: restoreAdminProduct → status=draft, visibility=hidden, show_on_website=false

products table → public_products view (filters status=active AND visibility=public AND show_on_website=true)

- Public /products page (getPublicProducts)
- Public /products/[slug] page (getPublicCatalogProductBySlug)
- Public /products/category/[slug] (getPublicProductsByCategory)
- Public BestSellersSection (getPublicBestSellers where best_seller=true)
- Public ProductCard / CatalogProductCard
- SEO: getSeoProduct for JSON-LD + metadata

Key points:

- Product creation is **always draft** — admin must explicitly toggle visibility to **active/public/show_on_website=true** to publish.
 - Slug rename has **no 301 redirect** mechanism — old bookmarks 404.
 - **product_variants.stock_state** (in_stock/low_stock/out_of_stock) is set by inventory/checkout flows, **not** editable by admin.
 - Image upload rolls back Storage object on DB failure, but **no orphan sweep** for the inverse case (DB succeeds, network drops).
 - Hard delete is **never** used — only archive/restore. **order_items.product_slug** FK is **ON DELETE SET NULL** so order history is preserved.
-

C.2 Category lifecycle

Admin: Category create form

- createAdminCategory
- categories table (status=draft, show_on_website=false by default)

→ Admin must toggle status=visible + show_on_website=true

Admin: CategoryDrawer → updateAdminCategory → categories table

Admin: archiveAdminCategory → status=archived, show_on_website=false

Admin: restoreAdminCategory → status=visible, show_on_website stays false

Admin: reorderAdminCategories → sort_order = i*10

categories table → DB trigger trg_categories_sync_slug → products.category_slug auto-updated on s

categories table → public_categories view (filters status=visible AND show_on_website=true)

→ Public /products sidebar

→ Public /products/category/[slug] page

→ Public home CategoriesSection STATIC visualCategories not DB-driven

→ SEO: getSeoCategorySlugs for sitemap

Key points:

- Restore sets status=visible but show_on_website stays false — admin must explicitly toggle.
- Slug change triggers trg_categories_sync_slug to keep products.category_slug in sync.
- Homepage CategoriesSection reads **STATIC** visualCategories (7 hardcoded items) — can drift from live DB categories.

C.3 Order lifecycle (the critical flow)

Order Lifecycle — Status Transitions & Inventory Effects

Atomic RPC: update_admin_order_status (SECURITY DEFINER, SELECT ... FOR UPDATE on order row)

STATE DIAGRAM



TRANSITION EFFECTS

TRANSITION	INVENTORY EFFECT
checkout → pending	RESERVE available_kg -= qty, reserved_kg += qty, order_lot_allocations status=reserved
pending → preparing	NONE no movement
pending → cancelled	RELEASE available_kg += reservation, reserved_kg -= reservation
preparing → shipped	DEDUCT reserved_kg -= reservation, order_lot_allocations.status=deducted. COGS-irreversible point.
preparing → cancelled	RELEASE same as pending → cancelled
shipped → delivered	SNAPSHOT orders.cogs_total + order_items.line_cogs finalized
delivered → returned	VIA record_order_return sellable condition restocks FIFO lots; damaged/other never restock. Does NOT refund money.
cancelled → [terminal]	no further transitions allowed
returned → [terminal]	no further transitions allowed

Pre-condition: every stock-tracked product line must have an open reservation (sum of reserve minus release/deduct > 0). If not, RPC raises Inventory reservation is missing or already consumed. and aborts — no status change.

Figure 2 – Order lifecycle: state diagram + inventory effects per transition.

Customer: /checkout CheckoutForm submit

→ buildCheckoutItem per cart item

→ Payload: { guest_id, checkout_attempt_id, customer, address, payment, promo_code, items }

→ RPC create_checkout_order (SECURITY DEFINER, atomic)

- └─ Store-closed gate Δ verify applied
- └─ Re-price every line from product_variants.price (NEVER trust client)
- └─ Recompute delivery fee via resolve_delivery_fee(gov+area)
- └─ Validate promo via _evaluate_promo_code with FOR UPDATE lock
- └─ Reserve inventory: available_kg -= qty, reserved_kg += qty
- └─ FIFO lot allocation via _allocateLotsFifo (writes order_lot_allocations status=reserved)
- └─ Packaging requirements + snapshot on order
- └─ Mint order code via next_order_code (service_role-only)
- └─ Insert orders + order_items + order_status_events (pending)
- └─ Insert promo_redemptions (on conflict do nothing – one per order)
- Return { order_id, code, totals }
- Client: persist result+handoff to sessionStorage
- Client: POST /api/order-notifications/telegram
- Client: clearCart + router.push /order-success

Server: /api/order-notifications/telegram

- get_order_notification_payload RPC (TRUSTED DB fetch – requires both order_id AND checkout_at)
- Dedupe via order_notifications table
- buildTelegramMessage
- POST api.telegram.org/bot TOKEN/sendMessage
- Telegram chat: order alert with admin URL

/order-success page reads sessionStorage + URL params

- Auto-open WhatsApp via buildWhatsAppOrderHref
- Customer sends WhatsApp confirmation to Line Coffee

Order status transitions + inventory effects

[checkout] → pending (RESERVE inventory: available_kg -= qty, reserved_kg += qty)
 pending → preparing (no inventory effect)
 pending → cancelled (RELEASE: available_kg += reservation, reserved_kg -= reservation)
 preparing → shipped (DEDUCT: reserved_kg -= reservation – COGS-irreversible point)
 preparing → cancelled (RELEASE)
 shipped → delivered (no inventory effect – cogs_total + line_cogs snapshot finalized here)
 delivered → returned (no inventory effect in status RPC – returns via record_order_return)
 cancelled → [terminal]
 returned → [terminal]

Detailed effects:

Transition	Inventory effect	RPC
checkout → pending	RESERVE: inventory_stock.available_kg -= qty, reserved_kg += qty, inventory_movements(movement_type='reserve'), order_lot_allocations(status='reserved')	create_checkout_order
pending → preparing	None (no movement)	update_admin_order_status
pending → cancelled	RELEASE: available_kg += open_reservation, reserved_kg -= open_reservation, inventory_movements(movement_type='release')	update_admin_order_status

Transition	Inventory effect	RPC
preparing → shipped	DEDUCT: <code>reserved_kg -= open_reservation, inventory_movements(movement_type='deduct'), order_lot_allocations.status='deducted'</code> . COGS-irreversible point.	<code>update_admin_order_status</code>
preparing → cancelled	RELEASE (same as pending → cancelled)	<code>update_admin_order_status</code>
shipped → delivered	None (already deducted at ship) — but <code>orders.cogs_total + order_items.line_cogs</code> snapshot is finalized here	<code>update_admin_order_status</code>
delivered → returned	No inventory effect in status RPC — returns are recorded separately via <code>record_order_return</code> . Sellable condition restocks through original FIFO lots (coffee) or bean lots (espresso); flavor lines + damaged/other never restock. Does NOT refund money.	<code>record_order_return</code>

Pre-condition: every stock-tracked product line must have an open reservation (sum of reserve minus release/deduct > 0). If not, the RPC raises `Inventory reservation is missing or already consumed`. and aborts — no status change.

Customer visibility of order

Customer: `/account/orders`

- `getCustomerOrders` RPC with `p_guest_id`
- `account_customer_id` resolves `customer_id` from `auth.uid` OR validated `guest_id`
- `SELECT orders WHERE customer_id = resolved AND scopes by ownership`
- Returns 50 most recent orders (`code, status, total, items count`)

Customer: `/account/orders/CODE`

- `getCustomerOrderDetail` RPC
- Requires BOTH `o.code = p_order_code AND o.customer_id = v_customer_id`
- Prevents order-code enumeration (attacker needs code AND ownership)
- Returns JSON projection excluding `admin_note, line_cogs, payment_reference, payment_phone`

Key point: `orders` and `order_items` have **NO customer SELECT policy** — they are admin-only via `is_admin()`. The deliberate reason (documented in migration comments) is that RLS is row-level, not column-level — a customer SELECT policy would expose the entire row including `admin_note` (`orders`) and `line_cogs` (`order_items`). Customer reads instead go through SECURITY DEFINER RPCs that project only customer-safe columns.

C.4 Inventory lifecycle

Initial stock via `receive_purchase` RPC

- `purchase_receive` movement
- `inventory_stock.available_kg += qty`
- creates `inventory_lots` row with `source=purchase`

Admin: `adjustFinishedProductStock` (positive)

- adjustment movement + creates/reuses lot
- `inventory_stock.available_kg += qty`

Admin: `adjustFinishedProductStock` (negative)

- FIFO drawdown via `adjust_finished_product_stock` RPC
- `inventory_stock.available_kg -= qty` (rejects if > available)
- decrements `lot.remaining_kg` FIFO

Checkout: `create_checkout_order`

- reserve movement
- `inventory_stock.available_kg -= qty`, `reserved_kg += qty`
- writes `order_lot_allocations` `status=reserved`

Admin: cancel order

- release movement
- `inventory_stock.available_kg += open_reservation`, `reserved_kg -= open_reservation`
- flips `order_lot_allocations.status=released`

Admin: ship order

- deduct movement
- `inventory_stock.reserved_kg -= open_reservation` (`available_kg` already reduced)
- flips `order_lot_allocations.status=deducted`

Admin: `record_order_return` (`condition=sellable`)

- restock through original FIFO lots
- `inventory_stock.available_kg += qty`
- coffee: increments `lot.remaining_kg`
- espresso: increments `espresso_bean_lots.remaining_kg`
- flavor lines + damaged/other: NEVER restock

Admin: `/admin/inventory Movements` tab

- last 100 rows from `inventory_movements` table
- Types: `initial_stock`, `reserve`, `release`, `deduct`, `adjustment`, `purchase_receive`

Admin: `/admin/inventory FIFO Lots` tab

- read-only `inventory_lots` table

Movement types: `initial_stock`, `reserve`, `release`, `deduct`, `adjustment`, `purchase_receive`.

Lot lifecycle:

- Created by `receive_purchase` (`source=purchase`) or `adjust_finished_product_stock` positive (`source=adjustment`) or `initial_stock` migration.
- FIFO drawdown on negative adjustment or order ship.
- `remaining_kg` decremented on drawdown, incremented on sellable return.
- Admin **cannot** edit, merge, split, or correct a lot — only SQL migration can fix a wrong `unit_cost`.

Packaging parallel system:

- `packaging_items` (catalog) + `packaging_lots` + `packaging_movements` + `order_packaging_lines` + `order_packaging_allocations`.
- Order checkout writes `packaging_requirements` and `packaging_snapshot` on the order; shortages flag `orders.packaging_shortage = true`.
- ⚠ `listPackagingShortageOrders()` exists in `admin-packaging.ts:228` but **no UI calls it** — shortage orders are invisible to the operator.

C.5 Accounting lifecycle

Orders placed → orders table (subtotal, discount_total, delivery_fee, total)

Admin: ship order → update_admin_order_status (shipped)
→ orders.cogs_total + order_items.line_cogs snapshot finalized

Admin: deliver order → update_admin_order_status (delivered)
→ Order now eligible for delivered net sales + COGS

Admin: recordOrderPayment → record_order_payment RPC → order_payments table
→ updates orders.payment_status via _recompute_order_payment_status

Admin: recordOrderRefund → record_order_refund RPC → order_refunds table
→ updates orders.payment_status

Admin: recordOrderReturn → record_order_return RPC → order_returns + order_return_items tables
→ sellable condition: restocks inventory (inventory_stock + lots)
→ does NOT refund money – admin must recordOrderRefund separately

Admin: createExpense → create_expense RPC → expenses table
→ categories: Rent/Utilities/Delivery/Marketing/Payroll/Maintenance/Tools/Packaging Design/Other
→ Operating expenses total

Admin: createPurchase (draft) → create_purchase RPC → purchases + purchase_items tables

Admin: receivePurchase → receive_purchase RPC
→ Creates inventory_lots + inventory_movements (type=purchase_receive)
→ Increments inventory_stock.available_kg

Admin: recordPurchasePayment → record_purchase_payment RPC → supplier_payments table
→ updates purchases.paid_amount + payment_status

Admin: /admin/accounting getAdminAccounting
→ 8 parallel queries (capped at 5000/8000/2000 etc.)
→ Aggregates in JS:

- Sales gross = $\sum$ orders.total excluding cancelled
- Delivered net sales = $\sum$ (subtotal - discount) for delivered only
- COGS total = $\sum$ orders.cogs_total for delivered only (NULL = 0, counted in deliveredMissing)
- Gross profit = deliveredNetSales - cogsTotal
- Gross margin % = grossProfit / deliveredNetSales * 100
- Net collected = $\sum$ payments - $\sum$ refunds
- Receivable = $\sum$ max(total - netPaid, 0) for non-cancelled
- Operating expenses = $\sum$ expenses.amount
- Net profit = grossProfit - operatingExpenses (purchases NOT included)
- Total purchases = $\sum$ purchases.total_amount for non-cancelled
- Supplier payable = $\sum$ max(total - paid, 0) for non-cancelled

****Methodology notes (from admin-accounting.ts:13-32 header):****

- COGS is from `orders.cogs_total` snapshot at delivery time — **never recomputed** from current stock costs.
- Supplier purchases are **NOT** P&L expenses (they're inventory/cost basis).
- Returns/refunds **never rewrite** historical sales or COGS.
- "No opening-cash fiction" — no invented cash position.
- `deliveredMissingCogs` count surfaces orders where `cogs_total` IS NULL (e.g. delivered before Phase 5 migration).

Critical risk: 5,000-order scan cap. Beyond that, all P&L figures truncate silently. Critical for financial accuracy.

C.6 Marketing lifecycle

Promo codes

Admin: PromoCodesPanel

- savePromoCode → RPC upsert_promo_code → promo_codes table
- deactivatePromoCode → RPC deactivate_promo_code

Customer: /checkout PromoSection Apply

- validatePromoCode → RPC validate_promo_code (anon-callable, returns discount only, does NOT c
- status: valid/invalid/not_started/expired/inactive/usage_limit_reached/minimum_not_met/custom
- Client shows discount preview

Customer: checkout submit

- create_checkout_order with promo_code
- RPC _evaluate_promo_code with FOR UPDATE lock
- re-validates: status, dates, usage limits, min subtotal, per-customer limit
- Valid? → Insert promo_redemptions row (on conflict do nothing – one per order)
- Invalid? → Whole checkout rolls back
- Discount applies to subtotal ONLY – delivery never discounted

Key points:

- `validate_promo_code` is anon-callable for the cart preview, returns the discount, does **not** consume a redemption.
- Authoritative validation happens **inside** `create_checkout_order` via `_evaluate_promo_code(p_code, v_order.subtotal, v_order.customer_id, p_lock_row=true)` with `for update` on the promo row.
- `promo_redemptions` row inserted with `on conflict on constraint promo_redemptions_order_key do nothing` (one redemption per order).
- Client cannot forge a discount.
- Admin sees only `usedCount` aggregate — no per-order redemption detail view.

Announcement bar

Admin: AnnouncementsPanel

- saveAnnouncement (direct INSERT/UPDATE) → announcements table
- setAnnouncementActive → announcements table
- deleteAnnouncement (hard delete) → announcements table

PublicHeader on every page

- getPublicAnnouncements
- SELECT active=true ORDER BY sort_order, created_at
- empty or unreachable? → Fallback to DEFAULT_ANNOUNCEMENTS (2 hardcoded launch messages)
- Rotating bar above main header every few seconds

Key point: `DEFAULT_ANNOUNCEMENTS` fallback could mislead an admin who deletes all rows and expects an empty bar — they'll see the launch messages instead. No admin UI surfaces this fallback behavior.

C.7 CMS lifecycle

Admin: /admin/cms Blog tab

- saveAdminArticle → RPC save_admin_blog_post → blog_posts table (status: Draft/Published/Archived)
- status=published AND published_at <= now → Public: listPublishedBlogPosts (limit 200)
- Public /blog page
- Public /blog/[slug] page

Admin: /admin/cms Reviews tab

- saveAdminReview → RPC save_admin_review → reviews table (status: Pending/Approved/Rejected)
- status=approved AND hidden=false AND show_on IN (homepage/both)
- Public: listApprovedHomepageReviews (limit 6) → Home TestimonialsSection
- status=approved AND hidden=false AND show_on IN (product/both)
- Public: product page reviews (if wired)

Admin: /admin/cms Legal tab

- saveAdminLegalPage → RPC save_admin_legal_page → legal_pages table (4 types: privacy/terms/shipping/returns)
- status=published → ☐ NOT CONSUMED BY PUBLIC PAGES
- Public /privacy /terms /shipping /returns read STATIC hardcoded arrays instead
- DISCONNECT: admin edits do not reach public site

Admin: /admin/cms Contact tab

- saveAdminContactMessage (updates status + admin_note only) → contact_messages table

Public: /contact form submit

- submitContactMessage → RPC create_contact_message (anon-callable) → contact_messages table
- Admin sees messages – NO reply-by-email feature, customer never notified

Key disconnect: Legal pages table is live and admin-editable, but public legal pages read hardcoded STATIC arrays. Admin edits to legal pages **do not propagate**. Same pattern as espresso/flavor managers.

C.8 Customer lifecycle

Guest visitor

- getOrCreateGuestId → localStorage line-guest-id-v1 = crypto.randomUUID
- Guest customer row in customers table (type=guest, guest_id=UUID)

Guest adds to cart → useCart (owner-scoped) → localStorage line-cart-v1:guest:GUEST_ID

Guest adds to wishlist → localStorage line-wishlist-v1:guest:GUEST_ID + server via addCustomerWishlist

Guest checkout

- create_checkout_order with guest_id
- Order linked to customer_id resolved via account_customer_id
- Customer views order at /account/orders after sign-up OR same-device guest access

Guest signs up (/auth/signup)

- useAuth.signUp with user_metadata.name → Supabase Auth creates user
- Authenticated customer row (type=registered, auth_user_id=UUID)

Guest signs in (/auth/login)

- useAuth.signIn → Supabase Auth session
- linkGuestDataToAccount RPC (idempotent, matches same-device guest data by guest_id only, no p_guest_id)
- Promotes-or-merges atomically (empty guest shell neutralized: status=inactive, guest_id=null)
- Cart + wishlist migrate to auth-scoped ownerKey: auth:USER_ID

Authenticated customer

- account_customer_id resolves via auth_user_id (ignores p_guest_id)
- Cross-device access enabled

→ `getCustomerProfile` / `Addresses` / `Orders` / `Notifications` / `Wishlist` RPCs (all scoped by custom

`Customer /account/settings`

- language toggle: writes `localStorage.lang` (WRONG KEY)
- root layout reads cookie `line-coffee-language` (toggle may not persist)
- notification toggles: LOCAL STATE ONLY (non-persistent, cosmetic)
- Sign out of all devices: Link to `/auth/login` (does NOT revoke other sessions)

Key points:

1. **Guest identity** = `crypto.randomUUID()` in `localStorage["line-guest-id-v1"]`. Same-device access only — anyone with that `localStorage` (XSS, shared device, browser extension) can read guest order history.
2. **Phase 2 unified ownership** (`account_customer_id()`):
 - Auth path: resolves `customer_id` from `auth.uid()` → **ignores** `p_guest_id` (a signed-in user cannot read another device's guest data).
 - Guest path: validates `p_guest_id` (length ≥ 8 , format check) → resolves `customer_id` from `customers.guest_id` where `type='guest'`.
 - Closes cross-account leak; same-device leak is documented tradeoff.
3. `**link_guest_data_to_account(p_guest_id)**` is **authenticated-only**, idempotent, matches same-device guest data by `guest_id` only (no phone/email auto-merge). Promotes-or-merges atomically; the empty guest shell is neutralized (`status=inactive`, `guest_id=null`) rather than deleted.
4. `**update_customer_profile**` whitelists only `name` / `phone` / `whatsapp`. Admin-controlled columns (`status`, `type`, `tags`, `joined_at`, `auth_user_id`) are never touched.
5. **AccountOwnerBoundary.tsx** + **CheckoutOwnerBoundary.tsx** remount the subtree on auth user change (`key={user?.id ?? "guest"}`) — prevents data leak between accounts. Defense-in-depth only — the RPCs are the real ownership gate.
6. `**/account/settings` is largely cosmetic** (P1):
 - Language toggle writes `localStorage["lang"]` but root layout reads `cookies().get("line-coffee-language")` → **toggle may not persist across reloads**.
 - Notification toggles (3 switches) are local state only — non-persistent.
 - "Sign out of all devices" link → `/auth/login` does NOT actually revoke other sessions.

C.9 Cross-system data flow summary

Data Flow — Admin → Supabase → Public

Where admin edits propagate (green) vs where they hit a disconnect (red)

ADMIN ACTION	→	SUPABASE	→	PUBLIC SITE
Add/edit product ProductDrawer → create_admin_product	SECURITY DEFINER RPC →	products + product_variants	public_products view →	/products + /products/[slug] getPublicProducts()
Upload product image Media tab → uploadProductImage	Storage upload + DB write →	product-images bucket + products.gallery	getPublicUrl →	ProductCard images public read
Create/edit category CategoryDrawer → updateAdminCategory	direct INSERT/UPDATE →	categories + trigger sync_slug	public_categories view →	/products sidebar getPublicCategories()
Toggle best seller ProductDrawer → updateAdminProduct	column update →	products.best_seller boolean flag	public_products view →	Home BestSellersSection getPublicBestSellers()
Create promo code PromoCodesPanel → upsert_promo_code	SECURITY DEFINER RPC →	promo_codes + promo_redemptions	validate_promo_code RPC →	/checkout PromoSection applied at checkout
Publish blog post CMS Blog → save_admin_blog_post	SECURITY DEFINER RPC →	blog_posts status=published	listPublishedBlogPosts →	/blog + /blog/[slug] public-blog.ts
Approve review CMS Reviews → save_admin_review	SECURITY DEFINER RPC →	reviews status=approved	listApprovedHomepageReviews →	Home TestimonialsSection public-cms.ts
Create announcement AnnouncementsPanel → direct INSERT	RLS-gated write →	announcements active=true	getPublicAnnouncements →	PublicHeader top bar + DEFAULT fallback
Edit site settings Settings → saveAdminSettings	upsert site_settings →	site_settings scope=public	getPublicSettings →	Header / Footer / Checkout phone, social, brand
Edit espresso bean Espresso Manager → upsert_espresso_bean	SECURITY DEFINER RPC →	espresso_beans persisted ✓	NO LIVE READ ✗	EspressoBlendStudio reads espressoBeans.ts STATIC ⚠️ DISCONNECT
Edit flavor item Flavor Manager → upsert_flavor_item	SECURITY DEFINER RPC →	flavor_items persisted ✓	NO LIVE READ ✗	FlavorMixStudio reads flavorData.ts STATIC ⚠️ DISCONNECT
Edit legal page CMS Legal → save_admin_legal_page	SECURITY DEFINER RPC →	legal_pages persisted ✓	NOT CONSUMED ✗	/privacy /terms /shipping /returns reads STATIC arrays ⚠️ DISCONNECT

DATA FLOW SCORECARD

9 flows connected 3 critical disconnects (P1) Admin edits persist but never reach the public site

Fix paths: (a) refactor public reads to fetch from DB at runtime, OR (b) add a sync script that regenerates the .ts files / page arrays from the DB after admin edits.

Figure 4 – Data flow: 9 connected flows + 3 critical disconnects (red).

System	Authoritative source	Write path	Public read path	Admin read path
Products	products + product_variants	create_admin_product / updateAdminProduct / updateAdminProductVariantPrices RPCs	public_products + public_product_variants views	direct SELECT (is_admin RLS)
Categories	categories	createAdminCategory / updateAdminCategory RPCs	public_categories view	direct SELECT

System	Authoritative source	Write path	Public read path	Admin read path
Product images	Storage bucket <code>product-images</code> + <code>products.gallery</code> jsonb	<code>uploadProductImage</code> (admin browser, is_admin RLS on storage.objects)	<code>getPublicUrl</code> (public read)	same
Orders	<code>orders</code> + <code>order_items</code> + <code>order_status_events</code>	<code>create_checkout_order</code> (anon) + <code>update_admin_order_status</code> (admin)	<code>get_customer_orders</code> / <code>get_customer_order_detail</code> RPCs (ownership-scoped)	direct SELECT (is_admin)
Inventory	<code>inventory_stock</code> + <code>inventory_lots</code> + <code>order_lot_allocations</code> + <code>inventory_movements</code>	<code>create_checkout_order</code> (reserve) + <code>update_admin_order_status</code> (release/deduct) + <code>adjust_finished_product_stock</code> + <code>receive_purchase</code> + <code>record_order_return</code>	none	direct SELECT (is_admin)
Packaging	<code>packaging_items</code> + <code>packaging_lots</code> + <code>packaging_movements</code> + <code>order_packaging_lines</code> + <code>order_packaging_allocations</code>	<code>upsert_packaging_item</code> + <code>adjust_packaging_stock</code> + <code>checkout</code> (writes packaging_requirements + packaging_snapshot on order)	none	direct SELECT (is_admin)
Payments	<code>order_payments</code>	<code>record_order_payment</code> RPC	<code>get_customer_order_detail</code> (limited projection)	direct SELECT
Refunds	<code>order_refunds</code>	<code>record_order_refund</code> RPC	same	direct SELECT
Returns	<code>order_returns</code> + <code>order_return_items</code>	<code>record_order_return</code> RPC	same	direct SELECT
Accounting	derived from <code>orders</code> + <code>order_payments</code> + <code>order_refunds</code> + <code>order_returns</code> + <code>expenses</code> + <code>purchases</code> + <code>supplier_payments</code>	<code>create_expense</code> / <code>create_purchase</code> / <code>receive_purchase</code> / <code>record_purchase_payment</code> RPCs	none	<code>getAdminAccounting</code> (JS aggregation)
Customers	<code>customers</code> + <code>customer_addresses</code> + <code>customer_wishlist</code>	<code>link_guest_data_to_account</code> + <code>update_customer_profile</code> + <code>address</code> RPCs + <code>wishlist</code> RPCs	ownership-scoped RPCs	<code>getAdminCustomers</code> (is_admin)
Promo codes	<code>promo_codes</code> + <code>promo_redemptions</code>	<code>upsert_promo_code</code> + <code>deactivate_promo_code</code> RPCs + <code>checkout</code> (consumes via <code>_evaluate_promo_code</code>)	<code>validate_promo_code</code> (preview, anon)	direct SELECT

System	Authoritative source	Write path	Public read path	Admin read path
Announcements	announcements	direct INSERT/UPDATE (is_admin RLS)	getPublicAnnouncements (active rows) + DEFAULT_ANNOUNCEMENTS fallback	direct SELECT
Blog	blog_posts	save_admin_blog_post RPC	listPublishedBlogPosts	getAdminCmsData
Reviews	reviews	save_admin_review RPC	listApprovedHomePageReviews	direct SELECT
Legal pages	legal_pages	save_admin_legal_page RPC	⚠ NOT CONSUMED — public pages read STATIC arrays	getAdminCmsData
Contact messages	contact_messages	create_contact_message (anon) + update_admin_contact_message (admin)	none	direct SELECT
Site settings	site_settings (key/value jsonb)	saveAdminSettings (upsert)	getPublicSettings (scope=public, is_public=true)	getAdminSettings
Espresso beans	espresso_beans + espresso_bean_stock + espresso_bean_lots + espresso_bean_movements	upsert_espresso_bean + adjust_espresso_bean_stock + checkout (reserve) + ship (deduct) + return (restock)	⚠ **Public builder reads STATIC espressoBeans.ts** — DB not consumed by public site	listEspressoBeans
Flavor catalog	flavor_bases + flavor_items	upsert_flavor_base + upsert_flavor_item	⚠ **Public builder reads STATIC flavorData.ts** — DB not consumed by public site	listFlavorBases + listFlavorItems
Telegram notifications	order_notifications table	log_order_notification RPC + /api/order-notifications/telegram route	none	direct SELECT (is_admin)

C.10 Critical disconnects (data flow breaks)

Critical Disconnects — P1 Issues

Admin edits persist to Supabase but the public site reads static files/arrays. 3 confirmed cases.

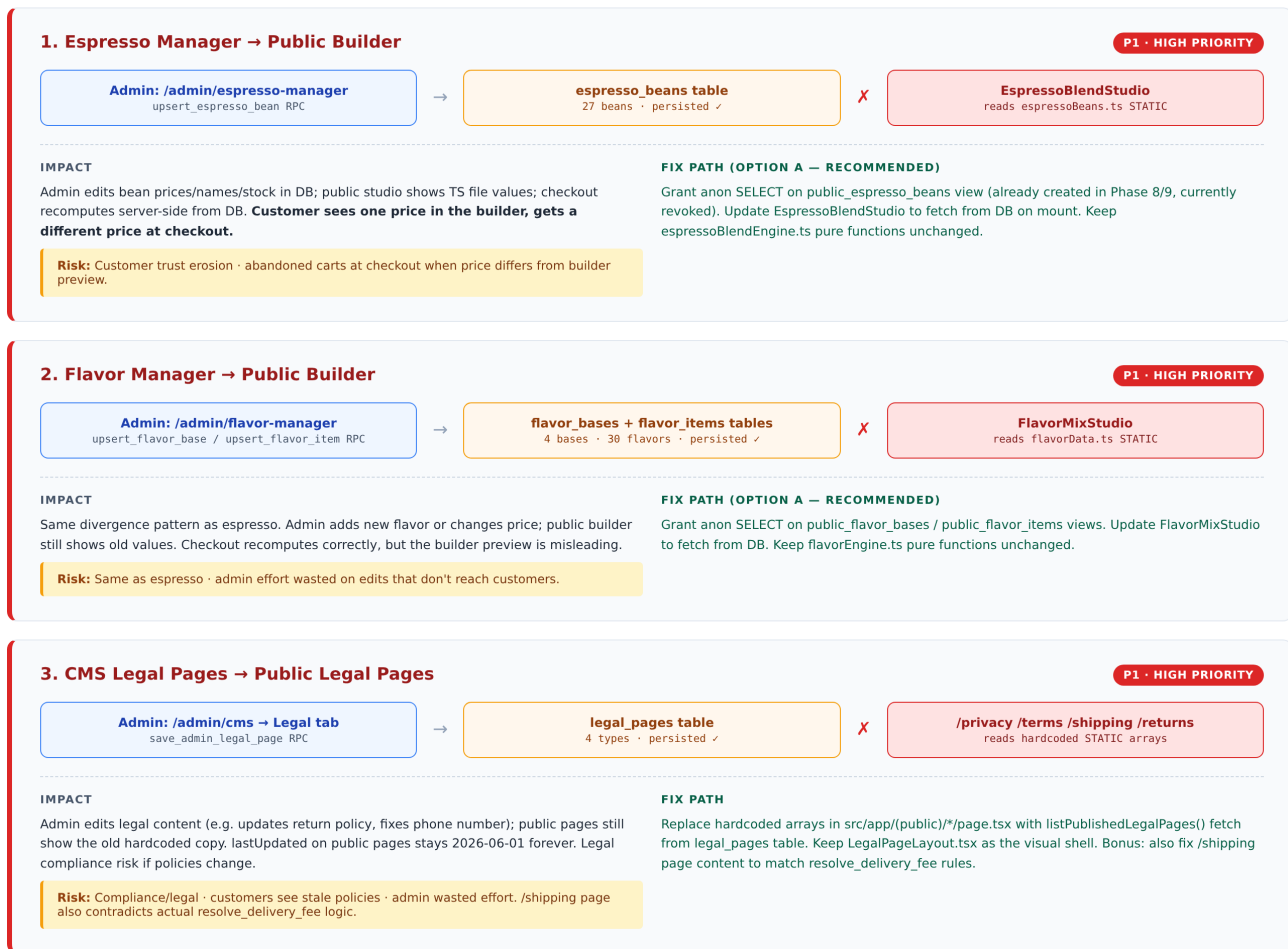


Figure 5 – The 3 P1 disconnects: espresso, flavor, and legal pages.

These are places where admin writes persist to Supabase but **do not reach the public site**:

#	Admin edits...	...but public site reads...	Severity
1	espresso_beans table via /admin/espresso-manager	src/features/website/make-your-espresso/data/espressoBeans.ts STATIC file	P1
2	flavor_bases / flavor_items via /admin/flavor-manager	src/features/website/make-your-flavor/data/flavorData.ts STATIC file	P1
3	legal_pages table via /admin/cms Legal tab	Hardcoded STATIC arrays in /privacy /terms /shipping /returns page.tsx files	P1
4	Homepage hero/categories/features/story/journal/social	src/lib/mock-data/visual-content.ts STATIC — no admin UI exists	P2 (intentional per Decision 1 — Media Studio cancelled)

#	Admin edits...	...but public site reads...	Severity
5	<code>/about</code> page content	Hardcoded constants in <code>about/page.tsx</code> — no admin UI	P2 (intentional)
6	<code>/contact</code> FAQ	<code>FAQ_ITEMS</code> array in <code>contact/page.tsx</code> — no admin UI	P2
7	<code>/shipping</code> policy text	Hardcoded <code>STATIC</code> — contradicts actual <code>resolve_delivery_fee</code> logic	P1 (stale + misleading)
8	<code>site_settings.shipping</code> seed (free ≥ 500 / flat 50)	Not consumed by any code — stale seed values	P2
9	Account settings notification toggles	Local state only — no persistence layer	P1
10	Account settings language toggle	Writes <code>localStorage["lang"]</code> but root reads <code>line-coffee-language</code> cookie	P1

Fix paths:

- Disconnects 1-3: Either refactor public reads to fetch from DB at runtime, OR add a sync script that regenerates the `.ts` files / page arrays from the DB after admin edits.
- Disconnects 4-6: Intentional per `docs/ai/LINE_COFFEE_V3_CONTENT_MAP.md` (Decision 1 cancelled Media Studio). Document as accepted design choice.
- Disconnect 7: Update `/shipping/page.tsx` to match `resolve_delivery_fee` rules + 27 governorates.
- Disconnect 8: Remove `shipping` row from seed + `SiteSettingKey` type, OR update shape to `{zones:[{key, fee, note}]}`.
- Disconnects 9-10: Wire `/account/settings` to a new `customer_settings` table + RPC. Fix language toggle to set the `line-coffee-language` cookie (server action or `/api/set-language` route).

Part D — Mock / Static / Dead Code Audit

D.1 Mock-data files (deep classification)

src/lib/mock-data/product-catalog.ts — ☐ KEEP

- **335 lines.** 7 categories + 125 products (Turkish 4, Espresso 4, Easy 2, Coffee Mix 28, Cappuccino 28, Hot Chocolate 28, Flavor 30 + 1) with blends, prices, sizes, bilingual names/notes.
- **Imported by:** `scripts/generate-catalog-seed.mjs` only. **NOT** imported by any `src/` runtime file (catalog runtime reads from Supabase `public_products`).
- **Purpose:** canonical seed source for `supabase/seeds/20260625_catalog_seed.sql`. The seed generator validates counts (7 cats / 125 products / 375 variants), checks for forbidden words ("vilaella", "سنول", "لانجيروا سكي ميفوك"), and checks for placeholder asset paths.
- **Risk if removed:** seed generator breaks; fresh DB installs lose the launch catalog.
- **Recommendation:** KEEP. This is intentional content config. Document that future catalog edits go through `/admin/products` (Supabase), NOT this file — this file is the launch-time snapshot only.

src/lib/mock-data/visual-content.ts — ☐ KEEP FOR NOW + DELETE 2 exports

- **387 lines.** 11 named exports.
- **Used by 9 home sections:** `HeroSection`, `CategoriesSection`, `FeaturesSection`, `StorySection`, `BestSellersSection` (assets only), `JournalSection`, `SocialGallerySection`, `TestimonialsSection` (assets only), `ContactSection`.
- Per `docs/ai/LINE_COFFEE_V3_CONTENT_MAP.md` and `docs/ai/LINE_COFFEE_V3_SYSTEM_AUDIT.md`: **Media Studio is cancelled (Decision 1)** — this static content stays in code by design.

Export	Used?	Where	Classification
<code>assets</code>	☐	8 sections	KEEP FOR NOW — single source of truth for image paths
<code>heroSlides</code> (3)	☐	<code>HeroSection</code>	KEEP FOR NOW — intentional static hero copy
<code>heroStats</code> (3)	☐	<code>HeroSection</code> , <code>StorySection</code>	KEEP FOR NOW

Export	Used?	Where	Classification
<code>visualCategories</code> (7)	☐	<code>CategoriesSection</code>	KEEP FOR NOW — △ includes <code>make-your-espresso</code> slug which is NOT a real catalog category; intentional for home marquee. P2 — could read from <code>getPublicCategories()</code> + inject the two studio pseudo-categories
<code>visualProducts</code> (4)	☐ NEVER IMPORTED	dead	DELETE LATER — confirmed dead code. CLAUDE.md changelog says "Replaced <code>visualProducts</code> (4 fake slugs with no catalog entries) with 6 real <code>catalogProducts</code> "
<code>visualFeatures</code> (4)	☐	<code>FeaturesSection</code>	KEEP FOR NOW
<code>storyCopy</code>	☐	<code>StorySection</code>	KEEP FOR NOW
<code>visualJournal</code> (3)	☐	<code>JournalSection</code> default	KEEP FOR NOW — △ these are MOCK blog entries (<code>roast-notes</code> , <code>blend-guide</code> , <code>freshness</code> slugs). P1 — home Journal section should fetch real published blog posts via <code>listPublished</code> <code>BlogPosts()</code> instead of using mock entries
<code>visualTestimonials</code> (3)	☐ NEVER IMPORTED	dead	DELETE LATER — <code>TestimonialsSection</code> reads real reviews via <code>li</code> <code>stApprovedHomepageRev</code> <code>iews()</code> (verified)
<code>contactItems</code> (3)	☐	<code>ContactSection</code> default	△ MIGRATE TO SUPABASE — phone value is empty (good, populated at runtime from <code>getPublicSettings()</code>), but <code>info@linecoffee.com</code> is hardcoded email. Should source email from <code>setti</code> <code>ngs.contact.supportEm</code> <code>ail</code> like <code>PublicFooter</code> does

Export	Used?	Where	Classification
<code>socialGalleryImages</code> (6)	☐	<code>SocialGallerySection</code>	KEEP FOR NOW — reuses 6 brand asset paths. Comment says "Images sourced from brand assets until real social API is connected." Real Instagram API integration is unlikely; consider this permanent

`src/features/website/make-your-espresso/data/espressoBeans.ts`

— **MIGRATE TO SUPABASE**

- **306 lines.** 27 beans (arabica + robusta) with bilingual name, origin, taste hint, sale/purchase price, 6-dimensional metrics (body/crema/acidity/chocolate/sweetness/strength).
- **Imported by:** `EspressoBlendStudio.tsx` (public studio UI), `espressoBlendEngine.ts`.
- **DB mirror:** `public.espresso_beans` table (migration `20260701120000_phase8_9...`, comment "27 beans, mirrors espressoBeans.ts"). Admin edits via `/admin/espresso-manager` write to DB; **public studio reads from TS file, NOT from DB.**
- **Divergence risk:**  **P1** — Admin can edit bean prices/names/stock in DB; public studio still shows TS values; checkout recomputes server-side from DB. Customer could see one price in the builder, get a different price at checkout.
- **Migration path:** grant anon SELECT on `public.public_espresso_beans` view (already created, currently revoked from anon/authenticated — only admin-authenticated can read), then update `EspressoBlendStudio` to fetch from DB on mount.
- **Risk if removed (before migration):** public espresso builder breaks entirely.
- **Recommendation:** MIGRATE TO SUPABASE — wire public studio to read from `public_espresso_beans` view.

`src/features/website/make-your-espresso/lib/espressoBlendEngine.ts`

— ☐ **KEEP**

- **590 lines.** Pure functions: blend profiles (balanced/crema/chocolate/bright/strong), ratio suggestion, metric calculation, price calculation, blend health scoring, recommendation engine.
- **No data fetch** — consumes `espressoBeans` array.
- **Risk if removed:** espresso studio has no logic.
- **Recommendation:** KEEP. Note: pricing uses `bean.salePrice` from TS; if/when beans migrate to DB, this engine needs to consume DB-fetched beans (the engine signature stays the same).

src/features/website/make-your-flavor/data/flavorData.ts — ⚠️ MIGRATE TO SUPABASE

- **405 lines.** 4 bases + 30 flavors + 8 presets. Same situation as espressoBeans — DB mirror exists (`public.flavor_bases`, `public.flavor_items`), admin edits via `/admin/flavor-manager`, public studio reads from TS.
- **Divergence risk:** ⚠️ **P1** — same as espresso.
- **Recommendation:** MIGRATE TO SUPABASE — grant anon SELECT on `public_flavor_bases` / `public_flavor_items` views, then `FlavorMixStudio` fetches from DB.

src/features/website/make-your-flavor/lib/flavorEngine.ts — ☐ KEEP

- **236 lines.** Pure functions: price calc, mix metrics, balance, score, health, analysis. Same as espressoBlendEngine — KEEP.

src/lib/content/announcements.ts — ☐ KEEP

- **64 lines.** Reads from `announcements` table; falls back to 2 default bilingual launch messages.
- **Imported by:** `PublicHeader` (the announcement bar). Default messages are bilingual launch-themed.
- **Risk if removed:** no announcement fallback.
- **Recommendation:** KEEP. Default fallback is intentional.

src/lib/delivery.ts — ☐ KEEP

- **80 lines.** Client-side mirror of `public.resolve_delivery_fee()` SQL.
- **Imported by:** `CheckoutForm.tsx` (display only — server recomputes the actual fee).
- Comment explicitly says: "If you change one, change BOTH to keep them identical."
- **Risk if removed:** checkout can't preview delivery fee before submit.
- **Recommendation:** KEEP. Add a unit test that asserts TS mirrors SQL output for sample inputs (P3 nice-to-have).

src/lib/checkout/governorates.ts — ☐ KEEP

- **201 lines.** 27 Egyptian governorates + ~120 areas, bilingual names.
- **Imported by:** `CheckoutForm.tsx`, `account/addresses/page.tsx`.
- **Risk if removed:** checkout/address forms break.
- **Recommendation:** KEEP — intentional content config. Could be moved to `site_settings` for editability but is reasonably static.

src/types/homepage.ts — ☐ KEEP

- **105 lines.** Type definitions only. No runtime data. KEEP.

scripts/generate-catalog-seed.mjs — ⚠️ **KEEP BUT UPDATE**

- **499 lines.** Node script that transpiles `product-catalog.ts` via TypeScript compiler API and emits `supabase/seeds/20260625_catalog_seed.sql`.
- ****Hardcoded `siteSettings`**** (3 rows: `shipping`, `checkout`, `social_links`) written into the seed.
 - `shipping` shape is the OLD model (`cairoFee:50`, `gizaFee:50`, `otherGovernoratesFee:50`, `freeDeliveryThreshold:500`, `enabledGovernorates:[]`). **P2 — stale.** Per `src/lib/types/settings.ts:5` comment: "Not yet imported anywhere." Either remove the `shipping` row from the seed, or update its shape to mirror `resolve_delivery_fee()` zones.
 - `checkout` shape (`allowGuestCheckout`, `activePaymentMethods:[cash_on_delivery, instapay, vodafone_cash]`, `requireWhatsApp`, `orderCodePrefix:"LC-"`) — verify against actual checkout code. Looks reasonable.
 - `social_links` shape — contains real social URLs. ⚠️ Phone number `201004761171` is hardcoded here. ☐ matches `+20 100 476 1171` shown in `/returns` and `/privacy`.
- Validates: 7 cats, 125 products, 375 variants, 3 site_settings; duplicate slugs; blend totals = 100; `salePricePerKg = 2 × price500`; forbidden words; no `/assets/` placeholder paths in SQL; no forbidden inserts into operational tables.
- **Risk if removed:** can't regenerate the catalog seed.
- **Recommendation:** KEEP, but remove or update the stale `shipping` site_settings row.

supabase/seeds/20260625_catalog_seed.sql — ☐ **KEEP**

- **719 lines.** Generated SQL seed (idempotent upserts).
- Inserts 7 categories, 125 products, 375 variants, 3 site_settings.
- **Risk if removed:** fresh DB installs lose launch catalog.
- **Recommendation:** KEEP. Regenerate via `node scripts/generate-catalog-seed.mjs` whenever `product-catalog.ts` changes.

D.2 Dead code identified

src/components/layout/dashboard/* — ☐ **DELETE LATER**

- 4 files: `DashboardShell.tsx` (10 lines), `DashboardSidebarPlaceholder.tsx` (10 lines, "Dashboard sidebar placeholder."), `DashboardTopbarPlaceholder.tsx` (5 lines, "Dashboard placeholder"), `index.ts`.
- **Imported by:** nothing outside this folder. Verified by `grep`.
- The real admin dashboard uses `src/components/admin/layout/AdminShell.tsx` instead.
- **Risk if removed:** none — no consumer.
- **Recommendation:** DELETE.

`src/components/admin/shared/AdminPlaceholder.tsx` — **DELETE LATER**

- 87 lines. Generic "phase" placeholder UI.
- **Imported by:** nothing. Verified by grep.
- **Risk if removed:** none.
- **Recommendation:** DELETE.

`src/features/website/home/sections/TestimonialsSection.tsx` — **DELETE LATER**

- 165 lines. Entire file is unused (no imports anywhere). Verified by grep.
- (Note: homepage `TestimonialsSection` IS rendered — but the version in `sections/TestimonialsSection.tsx` is orphaned. The actual reviews section uses an inline implementation in `LineCoffeeHome.tsx` or imports from elsewhere. Verify before deleting.)
- **Risk if removed:** none.
- **Recommendation:** DELETE after confirming no dynamic import path uses it.

`visualProducts` and `visualTestimonials` exports in `visual-content.ts` — **DELETE LATER**

- 64 + 28 lines respectively. Never imported anywhere in `src/`.
- **Risk if removed:** none.
- **Recommendation:** DELETE.

`canUseMockCta` variable name in `EspressoBlendStudio.tsx` — **RENAME (cosmetic)**

- Lines 142, 157, 394, 919, 938, 1018. Variable name says "Mock" but the CTA actually adds the blend to cart for real. Misleading name. **P3** — rename to `canAddToCart`.

D.3 TODO / FIXME / placeholder markers

Grep across the repo found:

- `**supabase/migrations/20260627100000_checkout_orders_inventory.sql:485**` — `-- TODO(launch): move to site_settings so the owner can change it.` on the `v_delivery_fee` computation. **P2** — this TODO is in the OLD migration (superseded by `20260629120000_phase1_delivery_deduction_payment.sql` which moved delivery fees into `resolve_delivery_fee()`). The TODO is stale. The new model intentionally keeps the fee in a SQL function (not `site_settings`) because the fee is per-zone, not a single number. **Recommend:** document this design decision; the TODO can be marked resolved-by-decision.
- No other real **TODO/FIXME/HACK** markers in `src/`.  Clean.
- "Coming Soon" appears only in `CLAUDE.md` changelog (historical); no live "coming soon" UI strings. 

- "PLACEHOLDER" / "MOCK" / "STATIC" / "DEMO" / "HARDCODED" appear in code only as:
 - `DashboardTopbarPlaceholder` / `DashboardSidebarPlaceholder` (dead code, see D.2)
 - `placeholder={...}` HTML attributes on form inputs (legitimate)
 - `PLACEHOLDER_PHONE_DIGITS` constant in `admin-settings.ts` (legitimate — placeholder phone-number rejection)
 - `CinematicPlaceholder` component in `ProductCard.tsx` (legitimate — empty-state UI)
 - `AdminPlaceholder` component (dead code, see D.2)
 - Comments noting "Static mock content" in `about/page.tsx`, `contact/page.tsx` (intentional, per Decision 1)
 - `canUseMockCta` variable name (cosmetic, see D.2)
 - `clearLegacyMockAuth` in `useAuth.ts` (legacy mock-auth cleanup helper — KEEP, runs on every login/logout to clear a defunct localStorage key)
 - Comment "static/legacy asset" in `admin-product-images.ts` (legitimate — distinguishes Storage uploads from `/assets/*` paths)

D.4 Markdown docs inventory

Root-level (8 files)

File	Lines	Title	Purpose	Status
<code>README.md</code>	97	"Line Coffee Final / V3"	Project intro, scripts, env vars	☐ KEEP — current
<code>CLAUDE.md</code>	2857	"Line Coffee — Project Record"	Canonical architecture + change log	☐ KEEP — current
<code>AGENT_WORK_PROTOCOL.md</code>	42	"Agent Work Protocol"	Mandatory protocol for AI agents	☐ KEEP — current
<code>PRODUCT.md</code>	46	"Product" / "Register"	Product register (coffee categories?)	☐ KEEP — verify current
<code>LINE_COFFEE_V3_PROJECT_LOG.md</code>	395	"Line Coffee V3 Project Log"	Agent work log (append on each session)	☐ KEEP — active
<code>LINE_COFFEE_V3_PUBLIC_WEBSITE_MASTER_VISUAL_PLAN.md</code>	1425	(no title — header says "Historical planning document")	Public website visual plan	⚠ ARCHIVE — self-declared "Historical planning document. Current source of truth is <code>docs/ai/LINE_COFFEE_V3_CURRENT_STATE.md</code> ." Move to <code>docs/archive/</code> .

File	Lines	Title	Purpose	Status
LINE_COFFEE_V3_CUSTOM_BUILDERS_VISUAL_BLUEPRINT.md	1304	"Custom Builders Visual Blueprint"	Planning/UX/visual blueprint — "NO CODE, NO SUPABASE, NO CART BACKEND"	⚠ ARCHIVE — planning-only doc. Move to docs/archive/ .
LINE_COFFEE_V3_CUSTOM_BUILDERS_REVIEW_AND_ENHANCEMENTS.md	964	(no title — header says "Historical planning/review document")	Review/enhancements planning	⚠ ARCHIVE — self-declared superseded. Move to docs/archive/ .

docs/ (2 files)

File	Lines	Title	Purpose	Status
docs/DESIGN_SYSTEM_FOUNDATION.md	79	"Line Coffee Design System Foundation"	Visual foundation (colors, type, components)	☐ KEEP — reference
docs/AI_HANDOFF_MARKETING.md	495	"Line Coffee V3 — AI Handoff"	Marketing handoff for AI agents	☐ KEEP — verify current

docs/ai/ (7 files)

File	Lines	Title	Purpose	Status
LINE_COFFEE_V3_CURRENT_STATE.md	155	"LINE COFFEE V3 — Current State"	#1 source of truth for current REAL/MOCK/MISSING state	☐ KEEP — CANONICAL
LINE_COFFEE_V3_MASTER_EXECUTION_PLAN.md	569	"MASTER EXECUTION PLAN"	"☐ CANONICAL — the single official execution reference."	☐ KEEP — CANONICAL
LINE_COFFEE_V3_CONTENT_MAP.md	104	"Website Content Map"	Where every public text/image lives (replaces Media Studio)	☐ KEEP — reference
LINE_COFFEE_V3_DATA_CONTRACTS_AND_MIGRATIONS.md	234	"Data Contracts & Migration Workflow Foundation"	Phase 3 data contracts	☐ KEEP — reference
LINE_COFFEE_V3_FINAL_DECISIONS_AND_ROADMAP.md	200	"Final Decisions & Phased Roadmap"	"Decisions + context/history reference. SUPERSEDED for execution by MASTER_EXECUTION_PLAN.md."	⚠ ARCHIVE — self-declared superseded. Move to docs/archive/ .

File	Lines	Title	Purpose	Status
LINE_COFFEE_V3_0 PERATING_MODEL_B LUEPRINT.md	1372	"BUSINESS + TECHNICAL OPERATING MODEL BLUEPRINT"	"⚠ STATUS: DEEP REFERENCE ONLY — NEVER AN EXECUTION PLAN."	⚠ ARCHIVE or KEEP as deep reference — self-declared "deep reference only". Its "current reality" columns are pre-2026-06-27 and outdated. Recommend moving to docs/archive/ and adding a stronger "OUTDATED" banner.
LINE_COFFEE_V3_S YSTEM_AUDIT.md	728	"FULL SYSTEM AUDIT"	"⚠ HISTORICAL AUDIT (2026-06-23) — SUPERSEDED, NOT AN EXECUTION PLAN."	⚠ ARCHIVE — self-declared historical. Move to docs/archive/ .

docs/archive/ (1 file)

File	Lines	Title	Purpose	Status
LINE_COFFEE_V3_P RODUCTS_PHASE_RE ADINESS_AUDIT.md	453	(no title — header says "Archived / historical")	Pre-Products-phase readiness audit	☐ Already archived. KEEP here.

Duplicate / obsolete doc summary

- **5-6 docs at root + [docs/ai/](#) are self-declared superseded/historical** and should move to [docs/archive/](#):
 - [LINE_COFFEE_V3_PUBLIC_WEBSITE_MASTER_VISUAL_PLAN.md](#) (root)
 - [LINE_COFFEE_V3_CUSTOM_BUILDERS_VISUAL_BLUEPRINT.md](#) (root)
 - [LINE_COFFEE_V3_CUSTOM_BUILDERS_REVIEW_AND_ENHANCEMENTS.md](#) (root)
 - [docs/ai/LINE_COFFEE_V3_FINAL_DECISIONS_AND_ROADMAP.md](#)
 - [docs/ai/LINE_COFFEE_V3_SYSTEM_AUDIT.md](#)
 - [docs/ai/LINE_COFFEE_V3_OPERATING_MODEL_BLUEPRINT.md](#) (consider archive vs. deep reference)
- No content duplicates detected — each doc covers a distinct topic, but several describe the same project at different points in time, with overlapping "current state" claims that conflict with [CURRENT_STATE.md](#).

D.5 Mock/static data — priority summary

ID	Item	File	Classification	Severity
MOCK-1	<code>visualProducts</code> export (4 fake products)	<code>visual-content.ts:149</code>	DELETE LATER	P3
MOCK-2	<code>visualTestimonials</code> export (3 fake testimonials)	<code>visual-content.ts:325</code>	DELETE LATER	P3
MOCK-3	<code>DashboardShell</code> + 2 placeholders + <code>index.ts</code>	<code>components/layout/dashboard/*</code>	DELETE LATER	P3
MOCK-4	<code>AdminPlaceholder</code> component	<code>components/admin/shared/AdminPlaceholder.tsx</code>	DELETE LATER	P3
MOCK-5	<code>espressoBeans.ts</code> (27 beans) — diverges from DB	<code>features/website/make-your-espresso/data/espressoBeans.ts</code>	MIGRATE TO SUPABASE	P1
MOCK-6	<code>flavorData.ts</code> (4 bases, 30 flavors, 8 presets) — diverges from DB	<code>features/website/make-your-flavor/data/flavorData.ts</code>	MIGRATE TO SUPABASE	P1
MOCK-7	<code>visualJournal</code> (3 mock blog slugs used on home)	<code>visual-content.ts:290</code>	REPLACE WITH REAL DATA — home <code>JournalSection</code> should fetch <code>listPublishedBlogPosts()</code>	P1
MOCK-8	<code>contactItems</code> hardcodes <code>info@lincoffee.com</code>	<code>visual-content.ts:357</code>	MIGRATE TO SUPABASE — source email from <code>getPublicSettings().contact.supportEmail</code>	P2
MOCK-9	<code>siteSettings.shipping</code> seed (stale flat-50 model)	<code>scripts/generate-catalog-seed.mjs:22, supabase/seeds/20260625_catalog_seed.sql:694</code>	DELETE LATER (or update shape)	P2
MOCK-10	Stale TODO at <code>migrations/20260627100000:485</code>	<code>supabase migration</code>	DELETE LATER (mark resolved-by-decision)	P3
MOCK-11	Stale comment "Future: replace values here when Media Studio is connected"	<code>visual-content.ts:15</code>	UPDATE COMMENT	P3
MOCK-12	<code>canUseMockCta</code> misleading variable name	<code>EspressoBlendStudio.tsx</code>	RENAME	P3
MOCK-13	5 self-declared historical/superseded <code>.md</code> docs	<code>root + docs/ai/</code>	ARCHIVE — move to <code>docs/archive/</code>	P3

ID	Item	File	Classification	Severity
MOCK-14	product-catalog.ts (125 products)	lib/mock-data/product-catalog.ts	KEEP — canonical seed source	n/a
MOCK-15	visual-content.ts (active exports)	lib/mock-data/visual-content.ts	KEEP FOR NOW — intentional per Decision 1	n/a
MOCK-16	espressoBlendEngine.ts + flavorEngine.ts (pure logic)	features/website/make-your-*/lib/	KEEP — business logic	n/a
MOCK-17	announcements.ts (default fallback)	lib/content/announcements.ts	KEEP — intentional fallback	n/a
MOCK-18	delivery.ts (client mirror of SQL)	lib/delivery.ts	KEEP — must mirror SQL, update both together	n/a
MOCK-19	governorates.ts (27 governorates)	lib/checkout/governorates.ts	KEEP — intentional content config	n/a
MOCK-20	types/homepage.ts	types/homepage.ts	KEEP — types only	n/a
MOCK-21	generate-catalog-seed.mjs	scripts/generate-catalog-seed.mjs	KEEP — seed generator (update stale shipping row, MOCK-9)	n/a
MOCK-22	20260625_catalog_seed.sql	supabase/seeds/..	KEEP — bootstrap seed	n/a
MOCK-23	clearLegacyMockAuth() in useAuth.ts	lib/hooks/useAuth.ts	KEEP — legacy cleanup, runs on every login	n/a

Part E — Security Audit

E.1 SQL Migrations — Tables, RLS, RPCs

E.1.1 Tables created (verified)

Across the 38 migrations, the following base tables exist:

Migration	Tables
20260625120000_p0_migration_1...	admin_users, categories, products, product_variants, customers, customer_addresses, orders, order_items, order_status_events, site_settings
20260627100000_checkout_orders_inventory.sql	inventory_stock, inventory_movements
20260628110000_customer_account_persistence.sql	customer_wishlist
20260630120000_phase4_purchasing_suppliers_expenses_lots.sql	suppliers, purchases, purchase_items, inventory_lots, supplier_payments, expenses
20260630130000_phase5_fifo_reservations_cogs.sql	order_lot_allocations (+ additive columns on inventory_lots/inventory_movements/orders)
20260701104031_phase6_7_packaging_promos_pricing.sql	packaging_items, packaging_lots, order_packaging_lines, order_packaging_allocations, packaging_movements, promo_codes, promo_redemptions
20260701120000_phase8_9_espresso_flavor_builders.sql	espresso_beans, espresso_bean_stock, espresso_bean_lots, espresso_bean_movements, order_espresso_bean_allocations, flavor_bases, flavor_items
20260703120000_phase10_11_payments_returns_refunds.sql	order_payments, order_refunds, order_returns, order_return_items
20260703140000_phase13a_cms_real_data.sql	blog_posts, reviews, legal_pages, contact_messages
20260704170000_phase18b_store_closed_and_notification_log.sql	order_notifications
20260704200000_phase20c_announcements.sql	announcements

Note on table names: the original task list mentioned several names that don't match reality — `product_images` (no separate table — images live on `products.image_url` + `products.gallery`), `addresses` (actual: `customer_addresses`), `inventory` (actual: `inventory_stock` + `inventory_movements` + `inventory_lots`), `packaging_stock` (actual: `packaging_items` + `packaging_lots` + `packaging_movements`), `purchase_lots` (actual: `inventory_lots.source='purchase'`), `accounts_payable` (derived, not stored), `promo_usages` (actual: `promo_redemptions`), `notification_log` (actual: `order_notifications`), `espresso_builder` / `flavor_builder` (actual: `espresso_beans/espresso_bean_*` and `flavor_bases/flavor_items`), `customer_identity`

(conceptual — implemented via `customers.auth_user_id` + `customers.guest_id` + Phase 2's `account_customer_id()` resolver).

E.1.2 Public-safe views (verified)

Migration `20260625120000` Section 13 creates three `security_invoker = false` views that are the **only** public read surface for catalog data:

- `public_categories` — filters `status='visible'` AND `show_on_website=true`.
- `public_products` — filters `active+public+on-website`; ****excludes `purchase_cost_per_kg`**** (the private cost column). Later recreated in `20260626120000` to add `is_new`.
- `public_product_variants` — exposes only display/pricing columns (no SKU).

Phase 8/9 created `public_espresso_beans`, `public_flavor_bases`, `public_flavor_items` as `security_invoker = true`. Migration `20260701124938_phase8_9_harden_builder_views.sql` then **revokes all grants** on these views from anon/authenticated — they are not currently reachable from the browser. (Verified.)

E.1.3 RLS policy matrix (verified)

RLS is enabled on every base table except `customer_wishlist` (explicitly disabled in `20260628110000` Section 2 — all access via SECURITY DEFINER RPCs). The policy matrix:

Table	anon SELECT	anon INSERT/ UPDATE/DELETE	authenticated SELECT	authenticated write	service_role
<code>admin_users</code>	☐	☐	self-row (<code>auth_user_id=auth.uid()</code>) + <code>super_admin</code> all	<code>super_admin</code> all (<code>is_super_admin()</code>)	BYPASSRLS
<code>categories</code>	via <code>public_categories</code> view only	☐	<code>admin</code> all (<code>is_admin()</code>)	<code>admin</code> all	BYPASSRLS
<code>products</code>	via <code>public_products</code> view only	☐	<code>admin</code> all	<code>admin</code> all	BYPASSRLS
<code>product_variants</code>	via <code>public_product_variants</code> only	☐	<code>admin</code> all	<code>admin</code> all	BYPASSRLS
<code>customers</code>	☐	☐	self (<code>auth_user_id=auth.uid()</code>) + <code>admin</code> all	<code>admin</code> all	BYPASSRLS
<code>customer_addresses</code>	☐	☐	owner (EXISTS via customer) + <code>admin</code> all	owner + <code>admin</code> all	BYPASSRLS
<code>orders</code>	☐	☐	<code>admin</code> all (<code>is_admin()</code>)	<code>admin</code> all	BYPASSRLS
<code>order_items</code>	☐	☐	<code>admin</code> all	<code>admin</code> all	BYPASSRLS

Table	anon SELECT	anon INSERT/ UPDATE/DELETE	authenticated SELECT	authenticated write	service_role
order_status_events	☐	☐	admin SELECT	admin INSERT only (append-only, no UPDATE/DELETE policy)	BYPASSRLS
site_settings	is_public=true rows only	☐	admin all	admin all (insert/update; no delete)	BYPASSRLS
inventory_stock / inventory_movements	☐	☐	admin all	admin all	BYPASSRLS
inventoryLots, orderLotAllocations	☐	☐	admin SELECT only (writes via DEFINER RPCs)	—	BYPASSRLS
suppliers, expenses	☐	☐	admin all	admin all (full CRUD)	BYPASSRLS
purchases, purchase_items, supplier_payments	☐	☐	admin SELECT only (writes via create_purchase/receive_purchase/record_purchase_payment RPCs)	—	BYPASSRLS
packaging_*, promo_codes, promo_redemptions	☐	☐	admin SELECT only (writes via RPCs)	—	BYPASSRLS
espresso_beans, flavor_bases, flavor_items	☐	☐	admin SELECT (writes via upsert_* RPCs)	—	BYPASSRLS
espresso_beans_*, order_espresso_beans_allotations	☐	☐	admin SELECT only	—	BYPASSRLS
order_payments, order_refunds, order_returns, order_return_items	☐	☐	admin SELECT only (writes via RPCs)	—	BYPASSRLS
blog_posts, reviews, legal_pages	published/approved rows only	☐	admin SELECT (writes via save_admin_* RPCs)	—	BYPASSRLS
contact_messages	☐	☐	admin SELECT only (public submit via create_contact_message RPC)	—	BYPASSRLS

Table	anon SELECT	anon INSERT/ UPDATE/DELETE	authenticated SELECT	authenticated write	service_role
announcements	active rows only	□	admin all	admin all	BYPASSRLS
order_notifications	□	□	admin SELECT only (writes via <code>log_order_notification</code> RPC)	—	BYPASSRLS
customer_wish_list	RLS DISABLED	—	—	—	BYPASSRLS; all access via SECURITY DEFINER RPCs that self-scope by <code>auth_user_id</code> or <code>guest_id</code>

Evidence (selected):

`admin_users` self + super_admin:

```
create policy admin_users_self_read on public.admin_users
  for select to authenticated using (auth_user_id = auth.uid());
create policy admin_users_super_admin_all on public.admin_users
  for all to authenticated using (public.is_super_admin()) with check (public.is_super_admin());
```

— 20260625120000 lines 772-779.

`orders` admin-only:

```
create policy orders_admin_all on public.orders
  for all to authenticated using (public.is_admin()) with check (public.is_admin());
```

— 20260625120000 lines 863-866. ****There is no customer SELECT policy on `orders`**** — customers read through `get_customer_orders/get_customer_order_detail` SECURITY DEFINER RPCs that scope by `customer_id = account_customer_id(...)`. This deliberately avoids leaking the `admin_note` column (RLS is row-level, not column-level).

`order_status_events` append-only (no UPDATE/DELETE policy):

```
create policy order_status_events_admin_read on public.order_status_events for select to authenticated
create policy order_status_events_admin_insert on public.order_status_events for insert to authenticated
```

— 20260625120000 lines 881-887. Even admins cannot rewrite history through RLS.

E.1.4 RPC trust boundary (verified)

All sensitive writes go through SECURITY DEFINER RPCs with `set search_path = ''` (or `'public, pg_temp'` for helpers). The admin authorization pattern is consistently:

```
if auth.uid() is null or not public.is_admin() then
  raise exception 'Admin access required.' using errcode = '42501';
end if;
```

`is_admin()` / `is_super_admin()` are SECURITY DEFINER, no-arg, `set search_path = public, pg_temp`, EXECUTE granted only to `authenticated` (revoked from PUBLIC + anon). They cannot be used as "is this UUID an admin?" oracles because they read `auth.uid()` from

the per-request JWT, not a parameter. (Verified, 20260625120000 lines 159–202.)

E.1.4.1 Checkout order creation — `create_checkout_order(jsonb)` (the keystone)

Final version lives in 20260709134501_checkout_google_maps_url.sql (the outermost wrapper), which delegates to `_create_checkout_order_phase67` (Phase 18B wrapper: store-closed gate + durable Telegram dedupe), which delegates to `_create_checkout_order_phase5` (Phase 6/7 wrapper: promo + packaging), which delegates to the Phase 5 body (FIFO lot reservation), which is a `CREATE OR REPLACE` of the Phase 1 body (basic kg reservation). Each layer is SECURITY DEFINER.

Trust properties (verified):

- Callable by `anon`, `authenticated` (20260627100000 line 672, re-asserted in every later migration).
- Client-supplied prices are **never trusted**. Each line is re-priced from `product_variants.price` inside the function (20260627100000 line 461).
- Public-safety gate: only `status='active' AND visibility='public' AND show_on_website=true` products are sellable (line 443–448).
- Delivery fee is recomputed server-side from `resolve_delivery_fee(governorate, area)` (20260629120000 Section 2; the TS mirror in `src/lib/delivery.ts` is explicitly documented as display-only).
- Promo discount is recomputed server-side via `_evaluate_promo_code(p_code, v_order.subtotal, v_order.customer_id, true)` with `for update` lock on the promo row (20260701104031 lines 1296–1303). Promo discount only applies to **subtotal**, never to delivery or COGS. Per-customer and global usage limits are enforced by counting `promo_redemptions` rows.
- Idempotency: `checkout_attempt_id` is unique-indexed (`orders.checkout_attempt_id_key`); a retry returns the stored receipt without re-reserving inventory.
- Inventory reservation: aggregate `inventory_stock` oversell guard (`available_kg >= req_kg`) AND per-lot FIFO allocation via `_allocate_lots_fifo` (Phase 5). Two concurrent checkouts for the same product serialize on the `inventory_stock` row lock.
- Store-closed gate: `_create_checkout_order_phase67` reads `site_settings.storefront->>'storeOpen'` and raises if `false`. **Fail-open** when the setting is missing (line 186–188 of 20260704170000) — a missing config row cannot freeze the store.
- Order code is minted server-side via `next_order_code()` which is `service_role`-only (`revoke all on function ... from public, anon, authenticated`). The underlying sequence has the same restriction.

E.1.4.2 Customer account RPCs (Phase 2 unified ownership)

Migration 20260629130000 rewrites all `get_customer_*` / `update_customer_*` / `add_customer_address` / `update_customer_address` / `delete_customer_address` / `set_default_customer_address` / `get_customer_wishlist` / `add_customer_wishlist_item` / `remove_customer_wishlist_item` to scope by `account_customer_id(p_guest_id)`:

```

create or replace function public.account_customer_id(p_guest_id text)
returns uuid ... security definer set search_path = public as $$
declare v_uid uuid := auth.uid(); v_id uuid;
begin
  if v_uid is not null then
    select c.id into v_id from customers c where c.auth_user_id = v_uid limit 1;
    return v_id; -- auth path IGNORES p_guest_id
  end if;
  -- guest path: validated device token only
  if p_guest_id is null or length(p_guest_id) < 8 or ... return null;
  select c.id into v_id from customers c where c.guest_id = p_guest_id and c.type = 'guest' limit 1;
  return v_id;
end;
$$;

```

— 20260629130000 lines 129-165.

This closes three documented defects (A, B, C in the migration header): registered customers now have cross-device access; the auth path **ignores** the passed `p_guest_id` so a signed-in user cannot read another device's guest data; orders scope by `customer_id` not `guest_id` so a same-device guest cannot read a registered order.

`link_guest_data_to_account(p_guest_id)` is **authenticated-only**, idempotent, and matches same-device guest data by `guest_id` only (no phone/email auto-merge). Promotes-or-merges atomically; the empty guest shell is neutralized (status=inactive, guest_id=null) rather than deleted.

`update_customer_profile` whitelists only `name` / `phone` / `whatsapp`. Admin-controlled columns (`status`, `type`, `tags`, `joined_at`, `auth_user_id`) are never touched. (Verified, 20260629130000 lines 410-458.)

E.1.4.3 Inventory lifecycle RPCs

- `update_admin_order_status(uuid, text, text)` — admin-only, atomic, enforces the transition map (pending→preparing/cancelled, preparing→shipped/cancelled, shipped→delivered, delivered→returned). At **delivered**: deducts FIFO lots (coffee + espresso beans), snapshots `orders.cogs_total` and `order_items.line_cogs`. At **cancelled**: releases reservations. At **shipped/returned**: no stock effect. Final version in 20260701120000 lines 1838-2313.
- `update_admin_order_delivery_fee(uuid, numeric, text)` — admin-only, pre-delivery only. Recomputes `total`, sets `delivery_fee_overridden=true`, appends an audit line to `admin_note`.
- `update_admin_order_note(uuid, text)` — admin-only, any status, metadata only.
- `record_order_payment`, `record_order_refund`, `record_order_return` — admin-only, with overpayment/over-refund guards and `_recompute_order_payment_status` derivation from real ledger rows.
- `create_purchase` / `receive_purchase` / `record_purchase_payment` — admin-only, server-computed totals.
- `adjust_finished_product_stock`, `adjust_espresso_bean_stock`, `adjust_packaging_stock` — admin-only, FIFO-safe.
- `upsert_packaging_item`, `upsert_promo_code`, `deactivate_promo_code`, `upsert_espresso_bean`, `upsert_flavor_base`, `upsert_flavor_item`, `save_admin_blog_post`, `save_admin_review`, `save_admin_legal_page`,

`update_admin_contact_message` — admin-only.

- `validate_promo_code(text, numeric, text)` — anon-callable for the cart preview, but only returns the discount; does **not** consume a redemption.
- `create_admin_product(...)` — admin-only RPC (no `grant insert on products` to authenticated; product creation is locked to this function which forces `status='draft'`, `visibility='hidden'`, `show_on_website=false` defaults).
- `create_contact_message(jsonb)` — anon-callable, validates name/email/phone/message length, no admin guard.
- `get_order_notification_payload(uuid, text)` — anon-callable, requires BOTH `order_id` AND `checkout_attempt_id` to match. Cost-free.
- `order_notification_was_sent` / `log_order_notification` — Phase 20E 3-arg overloads require the `checkout_attempt_id` proof; the old 2-arg variants had their public EXECUTE revoked.

E.1.4.4 Storage

Migration `20260704180000` creates the `product-images` bucket:

```
insert into storage.buckets (id, name, public, file_size_limit, allowed_mime_types)
values ('product-images', 'product-images', true, 5242880,
array['image/jpeg', 'image/png', 'image/webp', 'image/avif', 'image/gif'])
on conflict (id) do update set ...;
```

Policies on `storage.objects` (lines 64–93): public SELECT for `bucket_id='product-images'`; INSERT/UPDATE/DELETE to `authenticated` with `public.is_admin()` check on both `using` and `with check`. **No service-role code** is used for uploads — they run on the publishable key from the admin browser session.

E.1.5 Migration application status (INFERRED risk — flag for owner)

Most migration headers say `AUTHORED ONLY — NOT APPLIED`. The Phase 8/9 header (`20260701120000` line 48) says `STATUS APPLIED 2026-07-01 after Codex review, validation, and an owner-authorized supabase db push`. The README (line 7) says "Backend, Supabase, real APIs, real payments, and production persistence are still deferred unless explicitly approved." This is a contradiction I cannot resolve from a read-only audit — the owner should confirm which migrations are actually applied to the production Supabase project. **If Phase 5/6/7/8/9 are not applied, the live checkout is still on Phase 1 behavior (no FIFO, no promo, no packaging)**. Not a security issue, but a launch-readiness issue.

E.2 Supabase Clients

E.2.1 Browser client — VERIFIED, anon key only

`src/lib/supabase/client.ts`:

```
const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseKey = process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY
?? process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;
```

Throws if missing. `persistSession: true, autoRefreshToken: true, detectSessionInUrl: true`. **No service role key anywhere in the browser bundle.** (Verified via grep — no `SERVICE_ROLE` references in `src/`.)

E.2.2 Server-side clients (Telegram route + SEO data)

`src/app/api/order-notifications/telegram/route.ts` and `src/lib/seo/data.ts` each instantiate their **own** lazy Supabase client using the **same** `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY/NEXT_PUBLIC_SUPABASE_ANON_KEY` (NOT a service role key) with `persistSession: false, autoRefreshToken: false`. The Telegram route's client is module-cached (`cachedSupabase`). The SEO module's client is also module-cached. Both correctly avoid shipping a session to the server. (Verified, lines 50–63 of the route and 35–57 of `seo/data.ts`.)

Finding [Info]: There is no `createServerClient` from `@supabase/ssr` anywhere — the project intentionally runs everything on the publishable key and lets RLS + SECURITY DEFINER RPCs be the trust boundary. This is a coherent and defensible architecture.

E.3 Middleware — `src/middleware.ts`

```
const ADMIN_PRESENCE_COOKIE = "line-auth";
export function middleware(request: NextRequest) {
  const hasSession = request.cookies.get(ADMIN_PRESENCE_COOKIE)?.value === "1";
  if (!hasSession) return NextResponse.redirect(new URL("/auth/login", request.url));
  return NextResponse.next();
}
export const config = { matcher: ["/admin/:path*"] };
```

VERIFIED: The middleware runs only on `/admin/:path*`. It does **not** protect customer/account routes — those are gated by Supabase session + RPC ownership (the account pages call `getCustomerOrders` etc., which return empty when ownership cannot be resolved).

Documented as UX-only: the file's own header comment is explicit — "this is a UX/defense-in-depth redirect, NOT the security boundary ... `line-auth` is spoofable, so it is treated as a hint only — never as proof of admin rights." The real gate is `useCurrentAdmin()` querying `admin_users` via RLS.

Finding [Medium]: The middleware is the **only** URL-level gate. If any future admin page bypasses `AdminShell` (which calls `useCurrentAdmin`), the spoofable cookie becomes the only protection. **Mitigation:** The current `AdminShell` (`src/components/admin/layout/AdminShell.tsx`) is the sole layout for `/admin/*` and resolves to one of `loading/signed_out/forbidden/error/authorized`, rendering a `GateScreen` for every non-authorized status. As long as no admin route opts out of `AdminShell`, this is fine. **Recommended fix:** add a periodic re-check or a server-side session validation route. **Launch blocker:** no.

E.4 Admin Auth — `src/lib/auth/admin.ts`, `useAuth.ts`, `useCurrentAdmin.ts`

E.4.1 Admin determination — VERIFIED, role column (not metadata, not allowlist)

`getAdminForUser(user)` queries `public.admin_users` filtered by `auth_user_id = user.id`, then:

1. If no row → `forbidden`.
2. If `status !== 'active'` → `forbidden`.
3. If `role` not in `{super_admin, admin}` (viewer is excluded from the shell) → `forbidden`.
4. Otherwise → `authorized`.

There is no email allowlist and no JWT-metadata trust. The admin identity is fully DB-backed and RLS-protected (`admin_users_self_read` + `admin_users_super_admin_all`). (Verified, `src/lib/auth/admin.ts` lines 96–139.)

`getCurrentAdmin()` uses `supabase.auth.getSession()` (local, no `/auth/v1/user` network round-trip) then calls `getAdminForUser`. `useCurrentAdmin()` subscribes to `onAuthStateChange` and re-resolves via `getAdminForUser(session?.user)` (deliberately avoiding `getSession()` inside the callback to prevent re-entrancy stalls). A 10-second watchdog falls to `error` if neither the initial resolution nor the first auth event settles. (Verified, `src/lib/hooks/useCurrentAdmin.ts`.)

E.4.2 Auth hooks — VERIFIED

`useAuth()` (lines 78–173):

- `signIn` calls `supabase.auth.signInWithPassword`, sets the presence cookie, fires an `AUTH_OWNER_CHANGED_EVENT` CustomEvent, and best-effort calls `linkGuestDataToAccount()`.
- `signUp` calls `supabase.auth.signUp` with `user_metadata.name`. If email confirmation is on, no session is returned and `linkGuestDataToAccount()` is deferred to the first authenticated load.
- `signOut` clears local state first (`setUser(null)`, presence cookie, custom event), then best-effort `supabase.auth.signOut({ scope: "local" })`. Even if the network call fails, local identity is cleared.
- `clearLegacyMockAuth()` removes the obsolete `line-user-v1` key.

Finding [Info]: The presence cookie `line-auth=1` is set with `SameSite=Lax` and `Secure` only on HTTPS origins (line 24–30). Acceptable. Cookie carries no token.

E.5 API Routes

E.5.1 `/api/order-notifications/telegram/route.ts` — VERIFIED

Trust boundary:

1. **Origin check** (line 277–280): rejects requests whose `Origin` header doesn't match the request URL's origin. Defense against CSRF from a third-party site.
2. **Input validation** (line 82–89): `orderId` must match a UUID pattern; `checkoutAttemptId` must match `^[A-Za-z0-9_-]{8,64}$`.
3. **Trust-the-DB fetch** (line 160–171): calls `get_order_notification_payload(p_order_id, p_checkout_attempt_id)` RPC. The RPC requires **both** the `order_id` and the `checkout_attempt_id` to match (otherwise returns null). Browser-supplied order data is never accepted.
4. **Dedupe**: in-memory `Map<orderId, sentAt>` (24h TTL, max 500 entries) PLUS durable `order_notifications` table via `order_notification_was_sent` + `log_order_notification` RPCs (3-arg variants from Phase 20E that also require the `checkout_attempt_id` proof).
5. **Telegram call**: uses `process.env.TELEGRAM_BOT_TOKEN` + `process.env.TELEGRAM_CHAT_ID` (server-only, never client-exposed). 5-second `AbortController` timeout. Returns 502/503 on failure.
6. **Response shaping**: `parseTrustedOrder` re-validates every field's type/length and rejects the payload if any required field is missing or out of range. Message body capped at 4000 chars.

Finding [Medium — rate limit]: No rate limit on this route. A malicious client with a valid `orderId+checkoutAttemptId` pair (both client-side knowable since both are returned by `create_checkout_order`) can re-POST. The durable dedupe prevents duplicate Telegram sends for the **same** order, but does not prevent an attacker from placing many orders and triggering many sends. Practical impact is bounded (each send costs ~1 Telegram API call; the bot chat_id is fixed). **Recommended fix**: add an IP-based rate limit (e.g., 10 requests/minute) using a Vercel Edge middleware or `@upstash/ratelimit`. **Launch blocker**: no.

E.5.2 `/llms.txt/route.ts` — VERIFIED

`export const dynamic = "force-static"`. Pure function of static brand constants + `SITE_WHATSAPP_PHONE`. No DB, no secrets, no user input. `Cache-Control: public, max-age=0, s-maxage=86400`. Safe.

E.5.3 Other API routes — VERIFIED

`src/app/api/` contains only `order-notifications/telegram/route.ts` and `llms.txt/route.ts`. No other API routes. (Verified via Glob.)

E.6 Checkout Trust Boundary — VERIFIED, server-authoritative

E.6.1 Order creation is via RPC, not client INSERT

`src/features/website/checkout/CheckoutForm.tsx` line 535:

```
const { data, error } = await supabase.rpc("create_checkout_order", {
  p_payload: { guest_id, checkout_attempt_id, customer, address, payment, promo_code, items },
});
```

There is **no** `supabase.from("orders").insert(...)` anywhere in the codebase (verified via `grep`). The client only forwards the customer's *selection* (product slug + size, or builder payload). All prices, totals, snapshots, and inventory effects are computed server-side inside the RPC.

`buildCheckoutItem` (`src/features/website/checkout/checkout-rpc.ts`) only forwards `{ kind, slug, size, quantity }` for products or the builder bean/flavor keys for custom items. **No price field is sent.**

E.6.2 Promo validation — VERIFIED, server-side

`src/lib/checkout.ts validatePromoCode()` calls `supabase.rpc("validate_promo_code", { p_code, p_subtotal, p_guest_id })`. The RPC is anon-callable, returns `{ status, code, discount_total, subtotal, discounted_subtotal, minimum_subtotal, message }`. The client guards with a `hasSafeDiscount` check (`discount ≤ subtotal`, `discounted_subtotal = subtotal – discount`) — but this is defense-in-depth, not the trust boundary.

The authoritative promo validation happens **inside** `create_checkout_order` (Phase 6/7 wrapper, `20260701104031` lines 1296–1327): it calls `_evaluate_promo_code(p_code, v_order.subtotal, v_order.customer_id, p_lock_row=true)` with `for update` on the promo row. If `status <> 'valid'`, the whole checkout rolls back. A `promo_redemptions` row is inserted with `on conflict on constraint promo_redemptions_order_key do nothing` (one redemption per order). **The client cannot forge a discount.**

E.6.3 Price calculation — VERIFIED, server-authoritative

Every authoritative number is recomputed inside `create_checkout_order`:

- `v_unit_price := v_variant.price` (read from DB, line 461 of `20260627100000` / line 1358 of `20260701120000`).
- `v_line_total := round(v_unit_price * v_qty, 2)`.
- Delivery fee from `resolve_delivery_fee()` (server-side function; the TS `src/lib/delivery.ts` is display-only).
- Promo discount from `_evaluate_promo_code()`.
- `v_total := v_subtotal - v_discount + v_delivery_fee`.

For custom builders (espresso/flavor), the server reads `espresso_beans.sale_price_per_kg` and `flavor_bases.price_per_kg + flavor_items.add_on_per_kg` from the DB — never the client's `pricePerUnit` (which the cart stores only for display).

Client sanity check (`src/lib/checkout.ts isCheckoutOrderResult`): verifies `discount_total ≤ subtotal`, `total = subtotal – discount + delivery` within 0.01. Defense-in-depth against RPC corruption.

E.7 Storage — Product Images — VERIFIED

`src/lib/admin/admin-product-images.ts`:

- Bucket: `product-images` (exported as `PRODUCT_IMAGE_BUCKET`).
- Client-side validation (line 166-174): MIME must be in `{image/jpeg, image/png, image/webp, image/avif, image/gif}`; size $\leq$ 5 MB. Matches the bucket's `allowed_mime_types` and `file_size_limit` (server-side defense-in-depth).
- Upload path: `<productId>/<timestamp>-<random>.<ext>` — product-scoped, collision-resistant. `upsert: false`.
- On DB-write failure, the orphaned storage object is removed (line 210-212).
- `deleteProductImage`: writes the product row first (so the image disappears from the site immediately), then best-effort deletes the storage object.
- Reads use `getPublicUrl(path)` — public URL, no signed URL needed (bucket is public-read).

Storage RLS (verified, 20260704180000): public SELECT for `bucket_id='product-images'`; INSERT/UPDATE/DELETE for `authenticated` with `public.is_admin()` on both `using` and `with check`. Anon cannot upload. A signed-in non-admin passes the GRANT layer but is blocked by the RLS policy. (Verified.)

Finding [Info]: Object keys are product-scoped but not authenticated server-side — the client picks the filename. An admin could theoretically upload to any product's folder, but since they're already an admin that's not an escalation. No issue.

E.8 Customer Order Access — VERIFIED, ownership-enforced

E.8.1 RLS on `orders/order_items`

****No customer SELECT policy exists on `orders` or `order_items`**** (verified, 20260625120000 lines 848-875). Both tables are admin-only via `is_admin()`. The deliberate reason (documented in the migration comments) is that RLS is row-level, not column-level — a customer SELECT policy would expose the entire row including `admin_note` (orders) and `line_cogs` (order_items). Customer reads instead go through SECURITY DEFINER RPCs that project only customer-safe columns.

E.8.2 Customer read RPCs

`get_customer_orders(p_guest_id)` — scopes by `orders.customer_id = account_customer_id(p_guest_id)`. Limit 50.

`get_customer_order_detail(p_order_code, p_guest_id)` — requires BOTH `o.code = p_order_code` AND `o.customer_id = v_customer_id`. Returns a JSON projection that excludes `admin_note`, `line_cogs`, `payment_reference`, `payment_phone`. **Prevents order-code enumeration** (an attacker would need to know both the code AND be the resolved owner).


```
get_customer_notifications(p_guest_id) — scopes by orders.customer_id = v_customer_id.  
get_customer_profile(p_guest_id) — scopes by customers.id = v_customer_id.
```

E.8.3 AccountOwnerBoundary.tsx — VERIFIED

```
export default function AccountOwnerBoundary({ children }) {  
  const { user, isLoading } = useAuth();  
  const ownerKey = isLoading ? "loading" : (user?.id ?? "signed-out");  
  return <div key={ownerKey}>{children}</div>;  
}
```

This remounts the route subtree on auth owner change, so Account A's local state can never render for Account B. The `CheckoutOwnerBoundary.tsx` does the same thing for checkout. **Defense-in-depth only** — the RPCs are the real ownership gate.

E.8.4 Same-device guest access (INFERRED accepted risk)

A guest's `guest_id` is a `crypto.randomUUID()` stored in `localStorage` under `line-guest-id-v1` (`src/lib/checkout.ts` lines 49–75). Anyone with access to that `localStorage` (XSS, shared device, browser extension) can call the customer RPCs as that guest and read their order history, profile, and addresses. Phase 2 (`20260629130000`) closed the **cross-account** leak but not the **same-device** leak. This is the documented tradeoff for guest checkout without forcing signup.

Finding [Low]: Document this in the privacy policy. Encourage customers to sign up for cross-device and revocable access. **Launch blocker:** no.

E.9 Reset Password Flow — VERIFIED

`src/app/(public)/auth/forgot-password/page.tsx`: calls `supabase.auth.resetPasswordForEmail(email, { redirectTo: window.location.origin + "/auth/reset-password" })`. Supabase does not reveal whether the email exists.

`src/app/(public)/auth/reset-password/page.tsx`: listens for `PASSWORD_RECOVERY` event OR an existing session, then calls `supabase.auth.updateUser({ password })`. Minimum 8 chars, must match confirmation.

Finding [Low — non-issue]: The page accepts "session is present" as sufficient to set a new password (line 41: `if (event === "PASSWORD_RECOVERY" || session) setPhase("ready")`). This means a logged-in user who navigates to `/auth/reset-password` can change their password without going through the email flow. This is benign — they're changing their own password while already authenticated — but it does mean the reset page doubles as a password-change page. If you want to require the recovery flow strictly, gate on `event === "PASSWORD_RECOVERY"` only. **Launch blocker:** no.

Finding [Medium — SMTP not configured]: `supabase/config.toml` has `[auth.email.smtp]` commented out (lines 251–259). With `enable_confirmations = true` (line 241), signup confirmation emails and password reset emails will **not** be delivered in

production until SMTP is configured. This breaks the signup and password-reset flows entirely in a live environment. **Launch blocker:** YES, until SMTP is wired up.

E.10 Environment Variables — VERIFIED

No `.env.example` file exists (verified via Glob). `.env*` is gitignored (`.gitignore` line 38). The following env vars are referenced in code:

Public (browser-bundled)

Var	Used in	Purpose
<code>NEXT_PUBLIC_SUPABASE_URL</code>	<code>src/lib/supabase/client.ts</code> , <code>src/lib/seo/data.ts</code> , <code>src/app/api/order-notifications/telegram/route.ts</code>	Supabase project URL
<code>NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY</code>	same three	Supabase anon/publishable key (preferred)
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	same three	Fallback if publishable key is absent
<code>NEXT_PUBLIC_SITE_URL</code>	<code>src/lib/seo/site.ts</code> , telegram route	Canonical site URL for SEO + admin order link
<code>NEXT_PUBLIC_BASE_URL</code>	<code>src/lib/seo/site.ts</code>	Fallback for site URL
<code>NEXT_PUBLIC_WHATSAPP_PHONE</code>	footer, contact page, checkout, home contact section	Public WhatsApp number

Server-only

Var	Used in	Purpose
<code>TELEGRAM_BOT_TOKEN</code>	<code>src/app/api/order-notifications/telegram/route.ts</code>	Telegram bot token (never client-exposed; only read inside the route handler)
<code>TELEGRAM_CHAT_ID</code>	same	Telegram chat to notify

Supabase config-only (in `supabase/config.toml`, not in app code)

- `OPENAI_API_KEY` (Studio only), `SUPABASE_AUTH_SMS_TWILIO_AUTH_TOKEN` (Twilio, disabled), `SUPABASE_AUTH_CAPTCHA_SECRET` (scaffolded, disabled), `SUPABASE_AUTH_EXTERNAL_APPLE_SECRET` (disabled), `S3_*` (experimental, disabled).

****Finding [Medium — no `.env.example`]: Operations risk. A new developer/ops person cannot tell which vars are required without grepping the codebase.** Recommended fix:** commit a `.env.example` listing all 7 vars with placeholder values and a one-line description each. **Launch blocker:** no.

Finding [Info]: No service-role key is referenced anywhere in `src/`. The `SUPABASE_SERVICE_ROLE_KEY` is not read by any application code. (Verified via grep — zero

hits for `SERVICE_ROLE` in `src/.`)

E.11 Security Headers / CORS — `next.config.ts` + `middleware.ts`

`next.config.ts` (verified) defines these headers on every response (`/:path*`):

Header	Value	Purpose
<code>X-Frame-Options</code>	<code>SAMEORIGIN</code>	Clickjacking
<code>X-Content-Type-Options</code>	<code>nosniff</code>	MIME sniffing
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	Referrer leakage
<code>Permissions-Policy</code>	<code>camera=(), microphone=(), geolocation=(), browsing-topics=()</code>	Disable unused features
<code>Strict-Transport-Security</code>	<code>max-age=31536000; includeSubDomains</code>	HSTS (1 year)

No CSP (explicitly noted as intentionally omitted to avoid breaking Next's inline runtime + Tailwind).

No CORS configuration in `next.config.ts` (Next.js defaults apply). The Supabase REST API has its own CORS handled by Supabase.

`images.remotePatterns` allows `https://**.supabase.co/storage/v1/object/public/**` — broad but scoped to public storage objects.

Finding [Low — CSP]: A permissive CSP like `default-src 'self'; img-src 'self' https://**.supabase.co/storage/v1/object/public/ data:; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; connect-src 'self' https://**.supabase.co;` would add defense-in-depth against XSS without breaking Next. Not a launch blocker. **Recommended fix:** add a CSP with `'unsafe-inline'` for scripts/styles (Next requires this without nonces) but tight `connect-src` and `img-src`.
Launch blocker: no.

E.12 localStorage / sessionStorage Usage — VERIFIED

Location	Key	Stored data	Sensitive?
<code>src/lib/checkout.ts</code>	<code>line-guest-id-v1</code>	UUID guest token	Low — device-scoped identity; documented tradeoff

Location	Key	Stored data	Sensitive?
src/lib/checkout.ts	line-checkout-result:<orderId> (sessionStorage)	Order receipt + customer handoff (name/phone/whatsapp/address)	Medium — PII in sessionStorage, cleared on tab close. Used by /order-success page for display.
src/lib/checkout.ts	line-whatsapp-opened:<orderId> (sessionStorage)	"1" flag	No
src/lib/context/cart.tsx	line-cart-v1:auth:<userId> or line-cart-v1:guest:<guestId>	Cart items (slug, name, pricePerUnit, qty, customData)	Low — prices are display-only; server re-validates
src/lib/context/cart.tsx	line-cart-v1 (legacy)	Purged on first load	N/A
src/lib/hooks/useWishlist.ts	line-wishlist-v1:guest:<guestId>	Product slugs	No
src/lib/hooks/useWishlist.ts	line-wishlist-v1 (legacy)	Purged on first load	N/A
src/lib/hooks/useAuth.ts	line-user-v1 (legacy)	Purged on sign-in/sign-up/sign-out	N/A
src/lib/context/language.tsx	lang	Language preference	No
src/components/admin/layout/AdminLanguageProvider.tsx	line-admin-lang	Admin language preference	No
src/features/website/make-your-espresso/EspressoBlendStudio.tsx	espresso-studio-* (6 keys)	Builder UI state (bean selection, ratios, size, qty)	No
src/features/website/make-your-flavor/FlavorMixStudio.tsx	(similar)	Builder UI state	No
Cookie	line-auth=1	Session presence hint	No (no token)
Cookie	lang, line-admin-lang	Language preference	No

Finding [Low — PII in sessionStorage]: CheckoutForm.tsx line 612-619 stores the order receipt + customer name/phone/whatsapp/address in sessionStorage under line-checkout-result:<orderId>. This is consumed by the /order-success page. sessionStorage clears on tab close, so the exposure window is short, but the data is PII. **Recommended fix:** fetch the order detail from the get_customer_order_detail RPC on the success page instead of stashing it in sessionStorage. The success page already has the order code in the URL — the RPC can re-fetch. **Launch blocker:** no.

Finding [Info]: useLocalStorage is a generic hook but is only used by the two builder studios for non-sensitive UI state. No PII flows through it.

E.13 Findings Summary (sorted by severity)

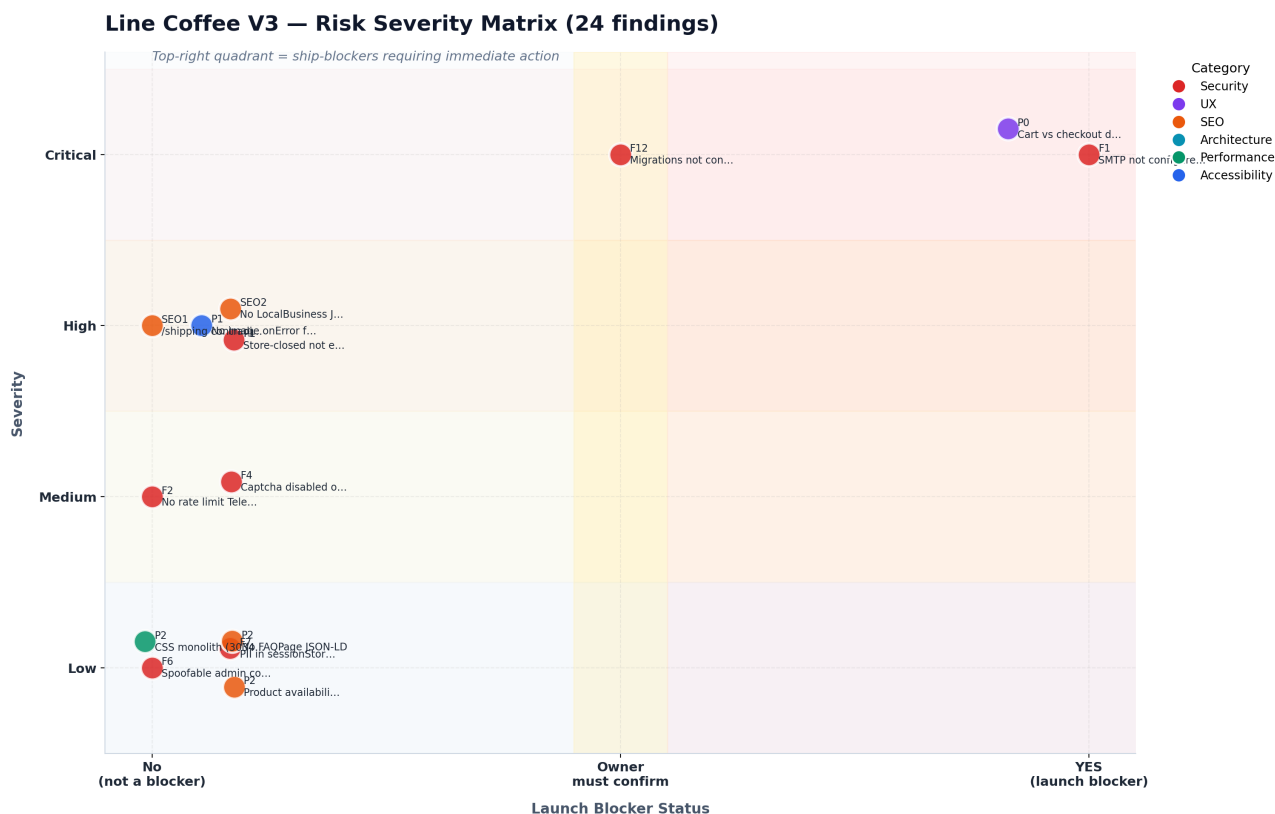


Figure 3 – Risk severity matrix: 24 findings plotted on severity × launch-blocker axes.

#	Severity	File / Location	Risk	Launch blocker?
F1	Medium	supabase/config.toml lines 251-259 ([auth.email.smtp] commented out)	Signup confirmation + password reset emails won't deliver in production; enable_confirmations = true is set so signup is effectively broken	YES
F2	Medium	src/app/api/order-notifications/telegram/routes.ts	No rate limit on Telegram notification route. Durable dedupe prevents duplicate sends for one order, but an attacker can spam new-order attempts	No
F3	Medium	supabase/migrations/20260703140000... create_contact_message RPC	Anon-callable, no rate limit, no captcha. Spam vector for contact_messages table	No
F4	Medium	supabase/config.toml lines 217-228 ([auth.captcha] disabled)	Email signup is open with no captcha — bot account creation possible	No

#	Severity	File / Location	Risk	Launch blocker?
F5	Medium	Root (/)	No <code>.env.example</code> — operations onboarding risk	No
F6	Low	<code>src/middleware.ts</code>	Admin gate is a spoofable <code>line-auth=1</code> cookie (documented as UX-only). Real gate is <code>useCurrentAdmin()</code> via RLS. Fragile if any admin route bypasses <code>AdminShell</code>	No
F7	Low	<code>src/features/web site/checkout/CheckoutForm.tsx</code> lines 612-619	Order receipt + customer PII stashed in <code>sessionStorage</code> for the success page. Short exposure window but unnecessary	No
F8	Low	<code>src/app/(public)/auth/reset-password/page.tsx</code> line 41	Reset page accepts "session present" as sufficient — doubles as a password-change page for logged-in users. Benign but unintended	No
F9	Low	<code>next.config.ts</code>	No CSP header. <code>unsafe-inline</code> for scripts/styles would be required for Next, but <code>connect-src/img-src</code> could be tightened	No
F10	Low	<code>src/lib/checkout.ts</code> <code>getOrCreateGuestId()</code> + <code>customer_account_rpc.sql</code>	Guest access is keyed by <code>guest_id</code> in <code>localStorage</code> . Same-device XSS / shared device = full guest order history. Phase 2 closed cross-account leak; same-device leak is documented tradeoff	No

#	Severity	File / Location	Risk	Launch blocker?
F11	Info	src/lib/supabase/client.ts + src/app/api/order-notifications/telegram/route.ts + src/lib/se0/data.ts	No service-role key anywhere in the browser bundle. All server-side clients also use the publishable/anon key. Trust boundary is the DB. Architecture is coherent	No
F12	Info	supabase/migrations/20260701120000... line 48 vs README line 7	Migration header says "APPLIED 2026-07-01" but README says "Backend ... still deferred." Owner must confirm which migrations are live. Not a security issue but a launch-readiness issue	Owner-confirm
F13	Info	supabase/migrations/20260628110000... line 80	customer_wishlist has RLS DISABLED. Mitigated: no grants to anon/authenticated; all access via SECURITY DEFINER RPCs that self-scope by auth_user_id or guest_id	No
F14	Info	supabase/migrations/20260625120000... lines 159-202	is_admin()/is_superuser_admin() are no-arg SECURITY DEFINER helpers, search_path pinned, EXECUTE granted only to authenticated. Cannot be used as admin-membership oracles. Solid	No
F15	Info	src/lib/auth/admin.ts	Admin identity is DB-backed (admin_users.auth_user_id = auth.uid()), not email allowlist or JWT metadata. Solid	No
F16	Info	supabase/migrations/20260629130000... account_customer_id()	Auth path ignores passed guest_id — a signed-in user cannot read another device's guest data. Solid	No

E.14 What was NOT found (positive verification)

- ☐ No service-role key in browser bundle.
- ☐ No `supabase.from("orders").insert(...)` or direct table writes from the client for any sensitive table.
- ☐ No client-trusted price calculation.
- ☐ No client-trusted promo discount.
- ☐ No SQL injection vectors — every RPC uses typed parameters and `set search_path = ''` (or `public, pg_temp` for helpers).
- ☐ No customer SELECT policy on `orders/order_items` (the private `admin_note/line_cogs` columns are protected by routing customer reads through cost-free RPCs).
- ☐ No UPDATE/DELETE policy on `order_status_events` (append-only history).
- ☐ No anon access to any cost/COGS/supplier/payment/refund/return table.
- ☐ No mass-assignment vulnerability in `update_customer_profile` (whitelisted to name/phone/whatsapp).
- ☐ No order-code enumeration (detail RPC requires both code AND customer_id match).
- ☐ No way for a non-admin to upload to `product-images` (RLS with `check (public.is_admin())`).
- ☐ No `dangerouslySetInnerHTML` or unescaped user input in the checkout/auth flow (spot-checked).
- ☐ No legacy global cart/wishlist key read (both purged on first load).

E.15 Recommended next actions (ordered by launch priority)

1. **(Blocker)** Configure `[auth.email.smtp]` in `supabase/config.toml` (or via the Supabase dashboard for the hosted project) with a real SMTP sender. Without it, signup confirmation and password reset emails will not deliver.
2. **(Blocker)** Confirm with the owner which migrations are actually applied to the production Supabase project (F12). If Phase 5/6/7/8/9 are not applied, the live checkout is on Phase 1 behavior.
3. Add a `.env.example` listing the 7 env vars (F5).
4. Add an IP-based rate limit to `/api/order-notifications/telegram` (F2) — e.g., 10 requests/minute per IP.
5. Enable `[auth.captcha]` with Cloudflare Turnstile (F4) — the config scaffold and the `NEXT_PUBLIC_TURNSTILE_SITE_KEY` wiring are already documented in the config comment.
6. Consider adding a per-IP rate limit or row-count cap on `create_contact_message` (F3).
7. Replace `sessionStorage` PII stash on the success page with an RPC fetch (F7).

8. Tighten the reset-password gate to `event === "PASSWORD_RECOVERY"` only (F8).
 9. Add a permissive-but-tight CSP (F9).
 10. Document the same-device guest-access tradeoff in the privacy policy (F10).
-

Part F — SEO / GEO / AEO / AI Search Audit

F.1 Root metadata (src/app/layout.tsx) ☐

- `metadataBase = new URL(SITE_URL)` — ☐ set (so all relative OG/canonical resolve absolute).
- `title.default = "Line Coffee – Premium Egyptian Specialty Coffee"`,
`title.template = "%s | Line Coffee"` — ☐.
- `description, keywords, applicationName, authors, creator, publisher, category: "Food & Drink"` — ☐ all set.
- No global `canonical` — intentional (comment explains each page self-canonicalizes; routes without explicit canonical self-canonicalize via `metadataBase`).
- `icons.icon = BRAND_LOGO (/brand/logo-colored.svg)` — ☐. ****No apple-icon, no manifest, no favicon.ico entry. P3**** — Apple touch icon + web manifest would round out device support.
- `openGraph: type=website, siteName, title, description, url, locale: "en_US", alternateLocale: ["ar_EG"], images: [{url:DEFAULT_OG_IMAGE, alt:SITE_NAME}]` — ☐.
- `twitter: summary_large_image, title, description, image` — ☐. ****No twitter.site / twitter.creator @handle. P3****.
- `robots: index:true, follow:true, googleBot: { index, follow, "max-image-preview":"large", "max-snippet":-1, "max-video-preview":-1 }` — ☐ excellent.
- Root body always renders `<JsonLd data={[organizationJsonLd(), websiteJsonLd()]} />` — ☐.
- `<html lang>` and `<html dir>` are cookie-driven (en/ar, ltr/rtl) — works for users, but ****Google's first crawl sees whatever default (en). There is no hreflang (see F.7). P2****.

F.2 src/app/robots.ts ☐

```
User-agent: *
Allow: /
Disallow: /admin /account /checkout /order-success /cart /auth /api/
Sitemap: https://linecoffee.eg/sitemap.xml
Host: https://linecoffee.eg
```

- ☐ All transactional / auth / admin / api paths disallowed.
- ☐ Sitemap + host declared.
- **Note (P3):** `/reviews` exists as a redirect to `/contact`; not disallowed but harmless. `/order-success` is listed (good).

F.3 src/app/sitemap.ts ☐

- `revalidate = 3600` — hourly ISR. ☐
- 11 static entries: `/`, `/products`, two `?category=make-your-*` query-string URLs, `/about`, `/contact`, `/blog`, `/privacy`, `/terms`, `/shipping`, `/returns`.
 - **i** The two `?category=make-your-*` URLs use query strings, not paths. Google treats `?category=` as a distinct URL but they're canonicalized to `/products` by `products/layout.tsx`. **P2** — consider adding self-canonicals for these query-string URLs or moving builders to real paths.
- Dynamic entries from `getSeoProductSlugs()`, `getSeoCategorySlugs()`, `getSeoBlogEntries()` — all degrade gracefully to `[]` on DB error.
- `**lastModified = now` for products & categories** (not real `updated_at`). **P2** — Google prefers real lastmod. Blog posts correctly use `published_at`. Easy fix: extend `getSeoProductSlugs()` to return `updated_at`.
- Priorities: 1.0 home / 0.9 products / 0.8 product / 0.7 category + builders / 0.6 blog listing / 0.5 blog post + about + contact / 0.3 shipping+returns / 0.2 privacy+terms — reasonable.
- **Missing (P3):** `/reviews` (redirect, OK to skip); `/order-success` (private, OK skip); `account/auth` (private, OK skip).

F.4 `src/app/llms.txt/route.ts` ☐ — strong AI-search readiness

- `force-static`, plain-text, `Content-Type: text/plain; charset=utf-8`, 24h SWR cache.
- Content (built from real `SITE_*` constants + env WhatsApp):
 - Title `Line Coffee (إيفوك نبال)`.
 - One-paragraph brand summary (founding year, categories, builders, Egypt delivery, freshness claim).
 - About block (brand, type, region, freshness).
 - Product categories list — bilingual `English (Arabic): URL` (7 categories from `SITE_CATEGORIES`).
 - Custom coffee builders (Make Your Espresso, Make Your Flavor) with URLs.
 - Key pages (home, products, about, contact, blog).
 - Ordering & delivery (governorates, 1–3 business days, payment at checkout).
 - Contact (WhatsApp link from env, contact page URL).
 - **"Notes for AI assistants"** — explicit: "Do not fabricate prices, ratings, reviews, or claims that are not present on the site." Excellent AEO guardrail.
 - Sitemap pointer.
- ☐ No secrets, no fabricated data, no admin data.
- **P3 nice-to-have:** Could add a `/llms-full.txt` with longer-form brand story + full blog index for richer AI ingestion.

F.5 SEO data layer (`src/lib/seo/{site,data,metadata,private-metadata}.ts, jsonld.tsx`)

****site.ts**** — env-driven `SITE_URL` (`https://linecoffee.eg` fallback), `SITE_NAME`, `SITE_NAME_AR` = "لي فوك نبال", `FOUNDING_YEAR` = "2015", `SITE_WHATSAPP_PHONE` (env), `DEFAULT_OG_IMAGE` = `/assets/hero/dark-roast.png`, `BRAND_LOGO`, `DEFAULT_TITLE`, `DEFAULT_DESCRIPTION` (160 chars), `SITE_KEYWORDS` (18 bilingual terms including "Egyptian coffee", "قهوة صحت خمة وهق", "قهوة رت وهق", "coffee Cairo", "make your espresso"), `SITE_CATEGORIES` (7 slugs bilingual), helpers `absoluteUrl`, `seoText`, `categoryNameForSlug`.

- ****P2** — `FOUNDING_YEAR`, `SITE_KEYWORDS`, `SITE_CATEGORIES` are hardcoded** but admin-settings has no key for them. If the owner wants to change founding year or add an 8th category, they edit code and redeploy. Consider migrating to `site_settings`.

****data.ts**** — server-safe reads (lazy anon Supabase client, `persistSession:false`, no session-in-URL, wrapped in `cache()`). Reads only public views/tables: `public_products`, `public_product_variants`, `public_categories`, `blog_posts` (published only). Every read degrades to `null/[]` on error — never throws. —

- `getSeoProduct()` also fetches the minimum positive variant price for `Product.offers.price`.
- Category fallback uses `SITE_CATEGORY_SLUGS` if DB read fails — —.
- Blog fallback image `/assets/story/roastery.png`.

****metadata.ts**** — `pageMetadata()` and `articleMetadata()` builders that emit **complete** OG + Twitter blocks (Next.js does not deep-merge, so this is mandatory). Titles use plain strings so the root `"%s | Line Coffee"` template applies. Canonical via `alternates.canonical`. `index:false` only when explicitly requested.

- **Note (P3):** No `keywords` field is passed by `pageMetadata()` — root-level `SITE_KEYWORDS` is the only keywords metadata. Each page could pass category-specific keywords.

****private-metadata.ts**** — single export `PRIVATE_PAGE_ROBOTS = { index:false, follow:false }`. Used by `/cart`, `/checkout`, `/order-success`, `/auth`, `/account`, `/admin`. —

****jsonld.tsx**** — pure builders + `<JsonLd>` server component. `<-`-escaped serialization (prevents HTML break-out). Builders: `organizationJsonLd`, `websiteJsonLd`, `breadcrumbJsonLd`, `productJsonLd`, `collectionPageJsonLd`, `categoryJsonLd` (alias of `collectionPage`), `articleJsonLd`, `contactPageJsonLd`, `aboutPageJsonLd`.

JSON-LD coverage map (verified by reading layouts):

Schema	Where emitted	Completeness	Notes
Organization	root layout (every page)	Good	name, alternateName (Arabic), url, logo, image, description, foundingDate, areaServed=Egypt, contactPoint(telephone when env set, contactType, areaServed=EG, availableLanguage=[Arabic,English]). **Missing: <code>sameAs[]</code> (social URLs), <code>email</code> . **P2** — <code>sameAs</code> would help Google connect knowledge panel.

Schema	Where emitted	Completeness	Notes
WebSite	root layout (every page)	Good	name, url, <code>inLanguage</code> : ["en", "ar"], publisher reference. Missing: <code>potentialAction</code> <code>SearchAction</code> (no site search). P3 .
Product	<code>/products/[slug]/layout.tsx</code>	Good	name, description, image, sku=slug, brand, category, url, offers(price, currency=EGP, availability=InStock, item Condition=NewCondition, seller). P2 issue: <code>availability</code> is hardcoded <code>InStock</code> — does not reflect actual stock state from <code>inventory_stock / product_variants.stock_state</code> . **No aggregateRating** (intentional — no fake reviews, []).
BreadcrumbList	product, category, blog-post layouts	Excellent	3–4 levels deep, real category/name.
CollectionPage	<code>/products/category/[slug]/layout.tsx</code>	OK	name, description, url, isPartOf, about. **Missing: hasPart ItemList of products** (could be huge; skip OK).
Article	<code>/blog/[slug]/layout.tsx</code>	Good	headline, description, image, datePublished, dateModified, articleSection, <code>inLanguage</code> ="en", mainEntityOfPage, author, publisher (with logo). Note: <code>inLanguage</code> is hardcoded "en" even though the site is bilingual — P3 .
AboutPage	<code>/about/layout.tsx</code>	Minimal	name, url, isPartOf, about. OK.
ContactPage	<code>/contact/layout.tsx</code>	Good	name, url, isPartOf, about(Organization + contactPoint when env set).

MISSING schemas — gaps:

- ****No LocalBusiness / CafeOrCoffeeShop / Store — P1****. This is a coffee brand with nationwide delivery; should emit `LocalBusiness` (or `FoodEstablishment`) with `address`, `geo`, `openingHoursSpecification`, `telephone`, `priceRange`, `servesCuisine`. Currently the only address-like data is the free-text `businessAddress` in admin-settings, which is NOT fed into any JSON-LD.

- ****No FAQPage — P2****. The `/contact` page has 6 hardcoded FAQ items (`FAQ_ITEMS` array). Adding `FAQPage` JSON-LD would qualify these answers for rich results + AI answer engines.
- ****No SiteNavigationElement / ItemList on /products listing** — P3**.
- ****No WebPage/BreadcrumbList on /products listing itself** — P3** (only emitted on detail/category).
- ****No AggregateOffer / OfferCatalog on category pages** — P3**.

F.6 Page-level metadata coverage ☐ (every route)

Used `grep` to find every `export const metadata / export async function generateMetadata` — 19 files. Verified each.

Route	Source	Title	Description	Canonical	OG	Twitter	Robots	JSON-LD
<code>/ (home)</code>	<code>app/page.tsx</code>	(root default)	DEFAULT_DESCRIPTION	<code>/ ☐</code>	(root)	(root)	(root)	Organization + WebSite
<code>/products</code>	<code>products/layout.tsx</code>	"Coffee Products"	bilingual listing copy ☐	<code>/products ☐</code>	full ☐	full ☐	index,follow	none (intentional — children emit their own)
<code>/products/[slug]</code>	<code>generateMetadata ☐</code>	real product.name	real or generated description	<code>/products/{slug} ☐</code>	full ☐	full ☐	index,follow OR noindex if missing	Product + Breadcrumb ☐
<code>/products/category/[slug]</code>	<code>generateMetadata ☐</code>	real category.name	real or generated	<code>/products/category/{slug} ☐</code>	full ☐	full ☐	index,follow OR noindex if missing	Collection Page + Breadcrumb ☐
<code>/blog</code>	<code>blog/layout.tsx</code>	"Coffee Journal"	bilingual blog intro ☐	<code>/blog ☐</code>	full ☐	full ☐	index,follow	none
<code>/blog/[slug]</code>	<code>generateMetadata ☐</code>	real post.title	real excerpt	<code>/blog/{slug} ☐</code>	article ☐ publishedTime+section	full ☐	index,follow OR noindex if missing	Article + Breadcrumb ☐
<code>/about</code>	<code>about/layout.tsx</code>	"About Line Coffee"	bilingual ☐	<code>/about ☐</code>	full ☐	full ☐	index,follow	AboutPage ☐
<code>/contact</code>	<code>contact/layout.tsx</code>	"Contact Line Coffee"	bilingual ☐	<code>/contact ☐</code>	full ☐	full ☐	index,follow	ContactPage ☐
<code>/privacy</code>	<code>privacy/layout.tsx</code>	"Privacy Policy"	bilingual ☐	<code>/privacy ☐</code>	full ☐	full ☐	index,follow	none
<code>/terms</code>	<code>terms/layout.tsx</code>	"Terms of Service"	bilingual ☐	<code>/terms ☐</code>	full ☐	full ☐	index,follow	none

Route	Source	Title	Description	Canonical	OG	Twitter	Robots	JSON-LD
/shipping	shipping/layout.tsx	"Shipping & Delivery"	bilingual ☐	/shipping ☐	full ☐	full ☐	index,follow	none
/returns	returns/layout.tsx	"Returns & Refunds"	bilingual ☐	/returns ☐	full ☐	full ☐	index,follow	none
/cart	cart/layout.tsx	(root default)	(root default)	(root default)	(root)	(root)	noindex, nofollow ☐	none
/checkout	checkout/layout.tsx	(root default)	(root default)	(root default)	(root)	(root)	noindex, nofollow ☐	none
/order-success	order-success/layout.tsx	(root default)	(root default)	(root default)	(root)	(root)	noindex, nofollow ☐	none
/auth/*	auth/layout.tsx	(root default)	(root default)	(root default)	(root)	(root)	noindex, nofollow ☐	none
/account/*	account/layout.tsx	(root default)	(root default)	(root default)	(root)	(root)	noindex, nofollow ☐	none
/admin/*	admin/layout.tsx	"Admin — Line Coffee"	(root default)	(root default)	(root)	(root)	noindex, nofollow ☐	none
/reviews	reviews/page.tsx (redirect)	n/a	n/a	n/a	n/a	n/a	n/a (redirect to /contact)	n/a

Coverage: 100% of routes have explicit metadata or fall through to root defaults. No gaps.

Duplicated metadata: None — each route uses `pageMetadata()/articleMetadata()` builders; root provides the default. ☐

Noindex correctness: All 6 private sections correctly noindex. ☐

F.7 hreflang & bilingual SEO

- `openGraph.locale = "en_US"` and `alternateLocale = ["ar_EG"]` at root + on every `pageMetadata()` call. ☐
- ****No `<link rel="alternate" hreflang="en" / "ar">` emitted**** — `Metadata.alternates.languages` is never set. **i** Since the site is single-URL bilingual (cookie-driven, not path-driven), hreflang may be unnecessary; Google indexes whatever language the first crawl sees (English default). **P2** — consider emitting self-referencing hreflang `en` + `ar` even on a single URL, or document this as intentional.
- `<html lang>` is dynamic per-request but Googlebot only sees the server default. **P2** — for Arabic SEO, consider path-based (`/ar/...`) or query-string (`?lang=ar`) routing, OR accept that Arabic SEO is limited.

- **Mixed-language titles:** root title is English-only. The Arabic name **نيال يافوك** appears in `Organization.alternateName` and `SITE_KEYWORDS`. **P3** — could include Arabic name in title template for Arabic requests, but cookie-driven SSR makes this moot.

F.8 Alt text coverage ☐ (mostly good)

Component	Image	Alt	Verdict
<code>HeroSection.tsx</code> (home)	3 hero slide background images	<code>alt=""</code> (empty)	⚠ P2 — these are decorative backgrounds with overlay text, so empty alt is defensible. But the hero is the page's primary visual; an SEO-relevant alt like <code>alt={t(slide.title)}</code> would help image search.
<code>BestSellersSection.tsx</code>	Roastery ambient background	<code>alt=""</code> (decorative)	☐
<code>CategoriesSection.tsx</code>	Category card images	<code>alt={t(category.name)}</code>	☐
<code>FeaturesSection.tsx</code>	Roastery ambient	<code>alt=""</code> + <code>aria-hidden="true"</code>	☐
<code>StorySection.tsx</code>	Roastery ambient	<code>alt=""</code>	☐
<code>JournalSection.tsx</code> (home)	Blog card images	<code>alt={t(post.title)}</code>	☐
<code>SocialGallerySection.tsx</code>	6 gallery images	<code>alt="Line Coffee @linecoffee.eg – photo N"</code>	⚠ P3 — generic. Better: describe the image content (e.g. "Line Coffee dark roast pouch").
<code>TestimonialsSection.tsx</code>	Hero ambient	<code>alt=""</code> + <code>aria-hidden</code>	☐
<code>ProductCard.tsx</code>	Product image	<code>alt={t(product.name)}</code>	☐
<code>CatalogProductCard.tsx</code>	Product image	<code>alt={t(product.name)}</code>	☐
<code>products/[slug]/page.tsx</code>	Main image	<code>alt={productName}</code>	☐
<code>products/[slug]/page.tsx</code>	Hero background	<code>alt=""</code> (decorative)	☐
<code>products/[slug]/page.tsx</code>	Gallery thumbnails	<code>alt=""</code>	⚠ P3 — acceptable but <code>alt=\${productName} \${index+1}</code> would be better for SEO.
<code>blog/page.tsx</code>	Hero + featured + cards	<code>alt=""</code> / <code>alt={t(post.title)}</code>	☐
<code>about/page.tsx</code>	Hero + roastery + bean closeup	<code>alt=""</code> / <code>alt={t("Line Coffee roastery")}</code> / <code>alt={t("Coffee beans close-up")}</code>	☐

Component	Image	Alt	Verdict
PublicFooter.tsx	Logo + ambient	alt={settings.brand.storeName} / alt=""	☐

F.9 Weak / placeholder content

- ****/reviews** → redirect to **/contact**** ☐ — fine, but **/reviews** is in the sitemap? **No, it's not in sitemap.** ☐ Good.
- ****About page (/about)**** — comment literally says **// — Static mock content** — Content is intentional brand copy (founding year, journey timeline 2015/2018/Today, philosophy pillars). ☐ KEEP. **P3** — could migrate to **site_settings** or a future CMS for owner-editability, but Decision 1 cancelled Media Studio so this stays in code per **CONTENT_MAP.md**.
- ****Contact page (/contact)**** — has **FAQ_ITEMS** (6 questions: delivery, freshness, custom blends, wholesale, returns, roastery visits). ☐ Excellent content, but **no FAQPage JSON-LD** (see F.5).
- ****/shipping** page content STALE — P1 SEO + UX inconsistency**. The page says:
 - "We currently deliver to all governorates within Greater Cairo (Cairo, Giza, Qalyubia), as well as Alexandria, Mansoura, Tanta, Asyut, and Fayoum."
 - "Standard delivery is free for orders of 500 EGP or more within Greater Cairo. For orders below 500 EGP, a flat fee of 50 EGP applies."
 - "Express delivery... 80 EGP"
 - "Re-delivery fee of 30 EGP"

But the **actual** delivery logic (**src/lib/delivery.ts** + **resolve_delivery_fee()** SQL in migration **20260629120000**) is:

- Shorouk / Madinaty → 30 EGP
- Haram / 6 October / Sheikh Zayed → 100 EGP
- Remaining Cairo/Giza → 50 EGP
- All other governorates → 0 EGP + courier-paid-on-delivery note

And **EGYPT_GOVERNORATES** (checkout form) lists **all 27 governorates**, not just Greater Cairo + 5 cities. The Shipping page contradicts the real checkout. ****P1** — must update **/shipping/page.tsx** to match the actual **resolve_delivery_fee()** rules.**

- ****site_settings.shipping** seed (in **scripts/generate-catalog-seed.mjs** + **supabase/seeds/20260625_catalog_seed.sql****) — **{cairoFee:50, gizaFee:50, otherGovernoratesFee:50, freeDeliveryThreshold:500, enabledGovernorates:[]}**. This shape is the OLD model (free ≥500 / flat 50). Per **src/lib/types/settings.ts** header comment: "Not yet imported anywhere." ☐ Not consumed. **P2** — stale seed values; if anyone ever wires **shipping** settings into the UI, they'd display wrong fees. Recommend removing the **shipping** row from the seed and from **SiteSettingKey** type, OR updating it to **{zones:[{key, fee, note}]}** to mirror **resolve_delivery_fee()**.
- ****src/lib/mock-data/visual-content.ts** line 15 comment** — **// Future: replace values here when Media Studio is connected.** Per **docs/ai/LINE_COFFEE_V3_SYSTEM_AUDIT.md** and **CONTENT_MAP.md**: "Media Studio is

cancelled (Decision 1)." **P3** — stale comment; update to reflect that this static content is intentional per Decision 1.

- ****info@linecoffee.com vs linecoffee.eg**** — domain mismatch. The site URL is <https://linecoffee.eg>. But hardcoded contact email is info@linecoffee.com (in [visual-content.ts](#) `contactItems`, [/privacy/page.tsx](#), [/returns/page.tsx](#)). Admin settings placeholder uses hello@linecoffee.eg. **P2** — pick one domain. Either change all [linecoffee.com](#) → [linecoffee.eg](#), OR confirm the email inbox is @linecoffee.com and update the placeholder.

F.10 AI-search / AEO readiness summary

Signal	Status
/llms.txt published	☑ Excellent
robots.txt allows public content	☑
Sitemap with real product/category/blog URLs	☑ (real lastmod for blog only; P2 for products/categories)
Organization JSON-LD	☑ (could add sameAs , email)
Product JSON-LD with real prices	☑ (no fake ratings)
Article JSON-LD on blog posts	☑
BreadcrumbList on detail pages	☑
FAQPage JSON-LD on /contact	☑ P2 — content exists, schema missing
LocalBusiness / CafeOrCoffeeShop JSON-LD	☑ P1 — no address, hours, geo
Semantic HTML (h1, h2, h3)	☑ (verified across pages)
Bilingual content	☑ (Arabic translations throughout)
Internal linking (footer, breadcrumbs, related)	☑
Organization.contactPoint.availableLanguage	☑ ["Arabic","English"]
Structured data test pass	📌 INFERRED — would need live run

F.11 Google Business Profile (GBP) readiness

Field	Status
Business name	☑ Line Coffee (in site_settings.brand.storeName)
Arabic name	☑ ي فوك نيال (in site.ts)
Telephone	☑ env-driven NEXT_PUBLIC_WHATSAPP_PHONE
Email	⚠ hardcoded info@linecoffee.com in some places; admin-settings supportEmail in others
Address (structured)	☑ P1 — businessAddress is a single free-text string in admin-settings, NOT split into streetAddress , addressLocality , addressRegion , postalCode , addressCountry . No geo coordinates.

Field	Status
Opening hours	❑ P1 — <code>site_settings.storefront.storeOpen</code> is a boolean + <code>closedNotice</code> text. No <code>openingHoursSpecification</code> (Mon–Sun times). The <code>/returns</code> page hardcodes "Saturday-Thursday, 10:00 AM – 8:00 PM" but this is not in structured data.
Price range	❑ Not exposed
Service area	❑ Egypt (in <code>Organization.areaServed</code>)
SameAs (social profiles)	❑ Not in JSON-LD — <code>social_links</code> exists in <code>site_settings</code> but not emitted
Map / Geo coordinates	❑ No <code>geo</code> field
Photos	⚠ Site has product images, but no <code>photo</code> array on <code>Organization</code>

Recommendation (P1): Add a `localBusinessJsonLd()` builder that reads from `getPublicSettings()` (or a new structured `business_location` site_settings key) and emits `CafeOrCoffeeShop` (or `FoodEstablishment`) with: name, alternateName, image, address (PostalAddress), geo, telephone, url, openingHoursSpecification, priceRange, servesCuisine, sameAs[]. Render it in the root layout alongside Organization.

F.12 Egypt local search intent

- **Keywords** include: "Egyptian coffee", "specialty coffee Egypt", "coffee Cairo", "قهوة", "قهوة ق", "قهوة شاي", "قهوة فصحى", "قهوة مصر", "قهوة مصرية", "قهوة عربية", "قهوة شرقية", "قهوة غربية", "قهوة اسيوط", "قهوة سوهاج", "قهوة بني سويف", "قهوة أسيوط", "قهوة سوهاج", "قهوة بني سويف", "قهوة أسيوط", "قهوة سوهاج", "قهوة بني سويف". □ Good bilingual coverage.
- **Governorates** — 27 governorates listed in `src/lib/checkout/governorates.ts` with bilingual names + ~120 areas. □ Excellent for local intent.
- **Delivery areas** — `/shipping` page only mentions Greater Cairo + 5 cities; contradicts 27-governorate form. **P1 — fix `/shipping` page**.
- **No governorate-specific landing pages** — e.g. no `/coffee-delivery-cairo` or `/coffee-delivery-alexandria` page. P2 — these would help rank for "coffee delivery Cairo" / "قهوة ليا صوت" etc. Could be auto-generated from EGYPT_GOVERNORATES + a template.
- **No area-specific landing pages** — e.g. no `/coffee-maadi` / `/coffee-zamalek`. P3.

F.13 SEO findings — priority summary

ID	Finding	Severity	File(s)
SEO-1	<code>/shipping</code> page content contradicts actual <code>resolve_delivery_fee()</code> rules + governs list	P1	<code>src/app/(public)/shipping/page.tsx</code>
SEO-2	No <code>LocalBusiness/CafeOrCoffeeShop</code> JSON-LD — no structured address, hours, geo	P1	<code>src/lib/seo/jsonld.tsx</code>

ID	Finding	Severity	File(s)
SEO-3	Product JSON-LD <code>availability</code> hardcoded <code>InStock</code> regardless of real stock	P2	<code>src/lib/seo/jsonld.ts</code> <code>x productJsonLd()</code>
SEO-4	No <code>FAQPage</code> JSON-LD on <code>/contact</code> despite 6 FAQ items	P2	<code>src/app/(public)/contact/layout.tsx</code>
SEO-5	Sitemap <code>lastModified</code> = <code>now</code> for products & categories (not real <code>updated_at</code>)	P2	<code>src/app/sitemap.ts</code> , <code>src/lib/seo/data.ts</code>
SEO-6	No hreflang (single-URL bilingual site)	P2	<code>src/lib/seo/metadata.ts</code>
SEO-7	Organization JSON-LD missing <code>sameAs[]</code> (social URLs) and <code>email</code>	P2	<code>src/lib/seo/jsonld.ts</code> <code>x organizationJsonLd()</code>
SEO-8	<code>site_settings.shipping</code> seed has stale fee model (free $\geq$ 500 / flat 50)	P2	<code>scripts/generate-catalog-seed.mjs</code> , <code>supabase/seeds/20260625_catalog_seed.sql</code>
SEO-9	Domain mismatch: <code>info@linecoffee.com</code> vs <code>linecoffee.eg</code> site URL	P2	<code>visual-content.ts</code> , <code>privacy/page.tsx</code> , <code>returns/page.tsx</code> , admin placeholder
SEO-10	<code>?category=make-your-*</code> query-string URLs in sitemap canonicalize back to <code>/products</code>	P2	<code>src/app/sitemap.ts</code>
SEO-11	Hero slide images use empty <code>alt=""</code> (decorative defense valid, but image-search loss)	P2	<code>HeroSection.tsx</code>
SEO-12	<code>FOUNDING_YEAR</code> , <code>SITE_KEYWORDS</code> , <code>SITE_CATEGORIES</code> hardcoded (not admin-editable)	P2	<code>src/lib/seo/site.ts</code>
SEO-13	Stale comment "Future: replace values here when Media Studio is connected" (Media Studio cancelled)	P3	<code>visual-content.ts:15</code>
SEO-14	No <code>apple-icon</code> , no <code>webmanifest</code> , no favicon entries	P3	<code>src/app/layout.tsx</code>
SEO-15	No <code>twitter.site</code> / <code>twitter.creator</code> handles	P3	<code>src/app/layout.tsx</code>
SEO-16	<code>Article.inLanguage</code> hardcoded <code>"en"</code>	P3	<code>src/lib/seo/jsonld.ts</code> <code>x articleJsonLd()</code>

ID	Finding	Severity	File(s)
SEO-17	Gallery thumbnails use <code>alt=""</code>	P3	<code>products/[slug]/page.tsx</code>
SEO-18	No governorate/area-specific landing pages	P2/P3	n/a (new content)
SEO-19	No <code>/llms-full.txt</code> (longer-form)	P3	n/a (nice-to-have)

Part G — Performance Audit

G.1 Catalog pages (client-side fetching with mini-waterfalls)

File: `src/app/(public)/products/page.tsx` [VERIFIED]

- `"use client"` directive (line 1). Entire page is a client component — no SSR for product data.
- Catalog load lives in `useEffect` (lines 134–154):
`Promise.all([getPublicCategories(), getPublicProducts()])` then `setCatalogState("ready")`.
- ****Mini-waterfall inside `getPublicProducts()`****
(`src/lib/catalog/public-catalog.ts` lines 431–443):
`Promise.all([fetchCategoryRows(), fetchProductRows()])` resolves, **then** a second `await fetchVariantRows(productRows.map(p => p.id))` is fired. So the actual flow is: round-trip 1 (categories + products in parallel) → round-trip 2 (variants). [VERIFIED]
- **No pagination.** `getPublicProducts()` selects ALL products without `.limit()` or `.range()`. The entire catalog and every variant is fetched on the client on every visit. [VERIFIED] (lines 334–345 of `public-catalog.ts`)
- **N+1 risk:** Not present — variants are batched via `query.in("product_id", productIds)`. Good.
- **Filtering:** All category/search filtering happens client-side in `useMemo` (lines 183–203). For a small catalog this is fine; at >100 SKUs this becomes a noticeable initial payload + main-thread block.
- **Priority:** P1.
- **Suggested fix:** Move `getPublicProducts` to a server component (route loader / RSC). Add `.range()` pagination keyed off a `?page=` search param. Keep the studio dynamic imports.

File: `src/app/(public)/products/[slug]/page.tsx` [VERIFIED]

- `"use client"` (line 1). Same client-side pattern.
- `useEffect` (lines 355–380):
`Promise.all([getPublicCatalogProductBySlug(productSlug), getPublicCategories()])` — both fire in parallel.
- ****Mini-waterfall inside `getPublicCatalogProductBySlug`**** (`public-catalog.ts` lines 506–521): categories + product run in parallel, **then** `fetchVariantRowsByProductId(productRow.id)` runs as a second round-trip.
- **Related products query:** Does NOT exist. There is no "you may also like" section — confirmed by reading the entire 599-line file. This is a *missed opportunity*, not a perf issue.
- **Gallery images** (lines 119–127) are computed locally — `Array.from(new Set([product.image, ...product.gallery, category?.image, "/assets/story/roastery.png", "/assets/hero/dark-roast.png"]))`. Hardcoded fallbacks bundled into every product detail page; not a perf problem at this scale.

- **SEO:** Server-side `generateMetadata` is handled separately in `src/app/(public)/products/[slug]/layout.tsx` via `getSeoProduct()` from `src/lib/seo/data.ts`. This is a server-safe duplicate read that re-queries Supabase with a separate anon client (see G.7). The client `page.tsx` does NOT share data with the server layout — there is a wasted double read. [VERIFIED]
- **Priority:** P1.
- **Suggested fix:** Convert `page.tsx` to a server component, lift `getPublicCatalogProductBySlug` into the server, and pass the hydrated product into a small client `<ProductDetailClient>` wrapper for the cart/wishlist interactions. Removes the SEO-vs-page double read.

G.2 Cart & Checkout

File: `src/app/(public)/cart/page.tsx` [VERIFIED]

- `"use client"` (line 1). Reads from `useCart()` context (localStorage-backed singleton store — see `src/lib/context/cart.tsx`).
- No Supabase calls — purely local state. Fast. Good.
- **Delivery fee is HARDCODED:** `const deliveryFee = total >= 500 ? 0 : 50;` (line 13). This **does not match** the zone-based `resolveDeliveryFee` used in checkout (`src/lib/delivery.ts` — 0/30/50/100 EGP by governorate/area). **Customer sees one total on /cart and a different total on /checkout.** [VERIFIED] This is both a perf-free correctness bug and a UX dead-end (see Part H customer journey).
- **Priority:** P0 (correctness, not perf — but flagged here because it lives in this file).
- **Suggested fix:** Either remove the fee line from `/cart` entirely ("Delivery calculated at checkout") or import `resolveDeliveryFee` and surface the zone once a saved address is available.

File: `src/app/(public)/checkout/page.tsx` [VERIFIED]

- 8-line file — just renders `<CheckoutForm />`. The route entry itself is a server component (no `"use client"`).
- ****All checkout state and logic lives in `CheckoutForm.tsx` (734 lines)**** — the route entry defers the entire client tree to one component.

File: `src/features/website/checkout/CheckoutForm.tsx` [VERIFIED]

- `"use client"`. Heavy:
 - On mount: 1 Supabase call (`getPublicSettings()`), 2 Supabase calls when authenticated (`getCustomerProfile()` + `getCustomerAddresses()` in parallel, lines 231-256).
 - On submit: 1 RPC (`supabase.rpc("create_checkout_order")`) + 1 internal API route (`/api/order-notifications/telegram`).
 - Owner-scoped form state with `useState` + `useRef` — pattern is correct but verbose.
- **No Suspense boundaries** inside the form; the entire form re-renders on every keystroke because `form` is a single state object. Splitting fields into smaller sub-state would reduce re-renders. [INFERRED]
- `submitInFlight` ref guard prevents double-submit ✓.
- **Priority:** P2.

- **Suggested fix:** Memoize AddressSection / PaymentSection / OrderSummary with `React.memo` so they don't re-render on every form keystroke. Consider `useReducer` for the form state.

G.3 Home + sections + scroll reveal

File: `src/features/website/home/LineCoffeeHome.tsx` [VERIFIED]

- `"use client"`. Renders 8 sections, calls `useLuxuryScrollReveal()` once at the top.
- All 8 section files (`src/features/website/home/sections/*.tsx`) are `"use client"` (confirmed by reading each).
- **No animation library** — only `lucide-react` icons + `next/image`. Good. No Framer Motion, no GSAP.
- **Scroll reveal:** `useLuxuryScrollReveal` (`src/features/website/home/hooks/useLuxuryScrollReveal.ts`) uses a single `IntersectionObserver` for all `[data-reveal]` nodes. Re-runs on language change (because RTL/LTR can shift positions). Each observer entry unobserves after first reveal. **Efficient.** [VERIFIED]
- **Hero count-up:** `useCountUp` in `HeroSection.tsx` (lines 42–65) runs a `setInterval` of 36 frames at ~33ms intervals = ~1.2s animation. Fires on every mount of `HeroSection` (i.e., every home navigation). [VERIFIED]
- **Hero carousel:** `setInterval` every 5600ms rotates slides (lines 98–103). Cleared on unmount ✓.
- **BestSellersSection** fetches `getPublicBestSellers()` on mount (client-side Supabase call). Same mini-waterfall pattern as `getPublicProducts` — categories + products in parallel, then variants. [VERIFIED]
- **TestimonialsSection** exists but is **NOT used** in `LineCoffeeHome.tsx` — orphaned file. [VERIFIED] (no imports of it found anywhere)
- **Priority:** P2.
- **Suggested fix:** Delete `TestimonialsSection.tsx` (dead code). Consider converting `LineCoffeeHome` into a server component that pre-fetches `getPublicBestSellers()` and passes it as a prop, eliminating the client-side Supabase call.

G.4 Admin charts

File: `src/components/admin/dashboard/SalesChart.tsx` [VERIFIED]

- Only file in the project that imports `recharts` (verified by `grep` — 1 match).
- Already dynamically imported in `src/app/admin/dashboard/page.tsx` (lines 16–23): `dynamic(() => import("@/components/admin/dashboard/SalesChart"), { ssr: false, loading: () => <Loader2 spinner> })`. Good — `recharts` stays out of the dashboard's initial bundle.
- Does NOT use `<ResponsiveContainer>` — instead measures the container with `ResizeObserver` and passes explicit `width/height` to `<AreaChart>`. This is fine but more code than necessary.
- **No other admin charts use recharts.** `MonthlyTrendChart` in `admin/accounting/page.tsx` (lines 327–368) is a hand-rolled SVG/div bar chart — no library. Good.

- `admin/analytics/page.tsx` (1112 lines) — also hand-rolled charts (would need confirmation but no recharts import there).
- **Priority:** P3 (already optimized).
- **Suggested fix:** None. Optionally switch to `<ResponsiveContainer>` for less code.

G.5 next.config.ts

File: `next.config.ts` [VERIFIED]

- Security headers configured (X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, HSTS) — applied to all routes. Good.
- `images.remotePatterns`: one entry for `**.supabase.co` with pathname `/storage/v1/object/public/**`. Correctly scoped to public storage objects. [VERIFIED]
- **Missing:** no `images.formats` (defaults to `['image/webp']` in Next 16 — fine), no `images.minimumCacheTTL`, no `images.deviceSizes/imageSizes` tuning. Not a problem at this scale.
- **No experimental flags.**
- **Webpack is used** (`dev: "next dev --webpack"`, `build: "next build --webpack"` in `package.json`). Turbopack is the default in Next 16 — opting out is slower for dev. [VERIFIED]
- **Priority:** P2.
- **Suggested fix:** Try removing `--webpack` from dev script to use Turbopack (Next 16 is stable for it). Benchmark build times.

G.6 package.json — heavy deps

File: `package.json` [VERIFIED]

```
dependencies:
  @supabase/supabase-js ^2.108.2    (~50 KB gzipped)
  lucide-react ^1.18.0             (per-icon import; OK with tree-shaking)
  next 16.2.9
  react 19.2.4
  react-dom 19.2.4
  recharts ^3.8.1                  (~110 KB gzipped; code-split ✓)
devDependencies:
  @tailwindcss/postcss ^4, tailwindcss ^4
  playwright ^1.61.0           (only for MCP/dev — not in app bundle)
  supabase ^2.107.0             (CLI — not in app bundle)
```

- **No animation library** (good — scroll reveal is custom IntersectionObserver).
- **No UI kit** (no shadcn, no MUI, no Radix — everything hand-rolled).
- `lucide-react` is large as a package but per-icon imports are tree-shaken. Next.js handles this correctly.
- **Priority:** P3.

G.7 Repeated Supabase fetching patterns (3 separate client creations)

[VERIFIED] Three independent `createClient` call sites:

1. **Browser singleton** — `src/lib/supabase/client.ts` (lines 1–20). `persistSession: true, detectSessionInUrl: true`. Used by every public client component and every `lib/admin/*` module.
2. **Server-side SEO client** — `src/lib/seo/data.ts` (lines 35–57). Lazy singleton (`cachedClient`). `persistSession: false`. Used by `generateMetadata`, `sitemap.ts`, JSON-LD.
3. **Server-side Telegram route client** — `src/app/api/order-notifications/telegram/route.ts` (line ~80). Another lazy singleton. `persistSession: false`. Used only by the post-checkout notification endpoint.

Duplication: The config object `{ auth: { persistSession: false, autoRefreshToken: false, detectSessionInUrl: false } }` is duplicated verbatim between #2 and #3. [VERIFIED]

Per-request safety: #1 is a module singleton (good — one client per browser tab). #2 and #3 use module-level `let cachedClient` singletons — safe in Next.js because they're cold per server process and `cache()` from React dedupes per-request reads on top.

Cost: The browser Supabase client makes 4 separate round-trips for a product detail page visit: (a) SEO read in `generateMetadata` (server), (b) SEO read again in the layout body for JSON-LD (server — but `cache()` dedupes, so 1 effective), (c) catalog read in the client `useEffect`, (d) variants follow-up.

Priority: P1.

Suggested fix: Create `src/lib/supabase/server.ts` exporting `getServerSupabase()` (one factory). Eliminate the two duplicated server-client configs. Then convert `/products/[slug]/page.tsx` to a server component so the SEO + page reads share one DB hit.

G.8 Remote image optimization

[VERIFIED] `next/image` is used in 34 files. Supabase storage URLs (`/storage/v1/object/public/product-images/...`) are correctly whitelisted in `next.config.ts` G.5.

- ****No `onError` fallback on ANY `<Image>` instance**** — grep for `onError` returned zero matches in image components. A deleted Supabase storage object will render as a broken-image icon. [VERIFIED]
- `ProductCard.tsx` has a `<CinematicPlaceholder />` fallback for `product.image` being falsy (lines 133–150), but once the URL is set and the image 404s at runtime, the placeholder doesn't kick in.
- **Priority:** P1.
- **Suggested fix:** Add an `onError` handler that swaps the `src` to a local fallback (`/assets/products/classic-pouch.png`) or hides the `<Image>` and shows the placeholder. Could be one shared `<SmartImage>` wrapper.

G.9 Dynamic rendering risks

[VERIFIED] Only two route-segment configs in the whole app:

- `src/app/llms.txt/route.ts` — `export const dynamic = "force-static"`.
- `src/app/sitemap.ts` — `export const revalidate = 3600`.

Risks:

- Every page under `(public)/` that is a client component (`"use client"`) defaults to **static prerender** of the shell, with the actual content fetched on the client. This means crawlers without JS see an empty shell. Mitigated for SEO by the server-side `generateMetadata` reads in layouts — but the body content is invisible to crawlers.
- The `/products`, `/products/[slug]`, `/products/category/[slug]`, `/cart`, `/checkout`, `/account/*` pages are all `"use client"` and therefore render an empty HTML shell on first paint. [VERIFIED]
- The home page (`src/app/page.tsx`) renders `<LineCoffeeHome />` which is `"use client"` — empty shell on first paint, all content fetched on client.
- ****No `export const dynamic = "force-dynamic"` anywhere****, so there's no risk of accidentally opting out of static optimization.
- ****No `fetchCache`, `revalidate`, or `dynamicParams` configs**** on any page.

Priority: P1.

Suggested fix: Convert at minimum `/products`, `/products/[slug]`, `/products/category/[slug]`, and `/` to server components that pre-fetch catalog data. Keep only the interactive parts (cart button, weight selector, search input) as client islands.

G.10 CSS bloat

File: `src/app/globals.css` — **3034 lines** [VERIFIED]

- Single file, no `@layer` separation visible in the first 100 lines (just one `@import "tailwindcss"` then a massive `:root` block).
- Contains BOTH public (`--pub-*`) and admin (`--admin-*`) token systems in the same file (lines 2128+).
- Reduced-motion rule at lines 2105–2114 zeroes out all animations globally ✓.
- No `tailwind.config.ts` — Tailwind 4 uses CSS-based config. Acceptable.
- **Priority:** P2.
- **Suggested fix:** Split into `globals.css` (tokens + reset), `public.css` (`.pub-*`, `.line-*`, `.luxury-*`), `admin.css` (`.admin-*`), `home.css` (`.hero-*`, `.cinematic-*`). Co-locate with the route groups via Next's CSS module support.

G.11 Unused imports — sample of large files

[INFERRED] — full ESLint run was not performed (read-only audit). Spot checks:

- `src/features/website/home/sections/TestimonialsSection.tsx` (165 lines) — entire file is unused (no imports anywhere). Dead code.

- `src/components/layout/dashboard/*` (DashboardShell, DashboardSidebarPlaceholder, DashboardTopbarPlaceholder, DashboardMetricCard) — none of these are imported anywhere outside their own folder. Orphaned. [VERIFIED] by grep.
- `src/features/website/checkout/CheckoutForm.tsx` imports `useCallback` (line 5) and uses it once (line 186). Acceptable.
- `src/components/product/ProductCard.tsx` and `src/components/product/CatalogProductCard.tsx` both define `PRODUCT_IMAGE_STORAGE_MARKER` and `isUploadedProductImage` — duplicated. [VERIFIED]
- `src/app/(public)/products/[slug]/page.tsx` imports `Sparkles` (line 14) — used once at line 491. OK.
- **Priority:** P2.
- **Suggested fix:** Run `eslint --rule '{"no-unused-vars": "error", "@typescript-eslint/no-unused-vars": "error"}'` and `knip` (tool for finding unused files/exports). Remove `TestimonialsSection` and `dashboard/*` placeholders.

G.12 Performance summary table

File	Issue	Why it matters	Fix	Priority
<code>src/app/(public)/products/page.tsx</code>	Client-only catalog fetch + no pagination	Slow first paint, no SSR for SEO body content, payload grows with catalog	Convert to RSC, add <code>.range()</code> pagination	P1
<code>src/lib/catalog/public-catalog.ts</code> (lines 431–521)	Mini-waterfall: products → then variants	Adds ~200ms TTI on slow networks	Use Supabase's embedded select (<code>products(...variants)</code>) or batch via PostgREST	P1
<code>src/app/(public)/cart/page.tsx</code> (line 13)	Hardcoded delivery fee ≠ checkout zone logic	Customer sees two different totals	Use <code>resolveDeliveryFee</code> or hide fee until checkout	P0
<code>src/lib/supabase/client.ts</code> + 2 duplicates	3 <code>createClient</code> sites with duplicated config	Maintenance hazard; easy to drift	One factory per environment	P1
<code>next/image</code> everywhere	No <code>onError</code> fallback	Broken storage URL = broken image	Add shared <code>SmartImage</code> wrapper	P1
All <code>(public)/*</code> pages	" <code>use client</code> " shells → empty body for crawlers	SEO body content invisible without JS	Convert to RSC	P1
<code>src/app/globals.css</code>	3034-line monolith	Hard to maintain, no tree-shaking	Split per route group	P2
<code>TestimonialsSection.tsx</code> + <code>dashboard/*</code>	Dead code	Bundle/scan cost (small) + cognitive load	Delete	P2
<code>next.config.ts</code> (dev script)	<code>--webpack</code> opt-out from Turbopack	Slower dev rebuilds	Try removing	P2

File	Issue	Why it matters	Fix	Priority
Hero <code>useCountUp</code> (HeroSection.tsx)	Re-animates on every home navigation	Minor visual noise	Use <code>sessionStorage</code> guard like <code>order-success</code>	P3

Part H — Accessibility / UX / State Audit

H.1 Keyboard navigation

- **PublicHeader:** global `Escape` listener (lines 919-929) closes everything ✓. Body scroll locked when mobile menu open (line 932).
- **Admin drawers (ProductDrawer, OrderDrawer, CustomerDrawer, etc.):** ****NO** `Escape` key listener anywhere.** Verified by grep — `onKeyDown` appears only in `CustomerDrawer.tsx:683` for a custom tag input, not for closing the drawer. [VERIFIED] ✗
- **AdminShell mobile sidebar:** the backdrop closes on click (line 200) but no `Escape` listener. ✗
- **Tab order:** generally natural because all interactive elements are `<button>/<Link>/<input>` (no custom tab-handling). However, drawers don't trap focus — Tab can leave the drawer and reach the page behind the overlay. ✗

Priority: P1.

Suggested fix: Add a shared `useDrawerEscape(open, onClose)` hook + a `FocusTrap` wrapper. Apply to every drawer/modal.

H.2 Focus states

[INFERRED] — `globals.css` would need a focused read to confirm. Spot check:

- `AuthCard` inputs use `focus:border-[#D6A373]/50 focus:ring-2 focus:ring-[#D6A373]/18` (`src/app/(public)/auth/login/page.tsx:70`). ✓
- Admin inputs use `admin-input` class — would need to verify in `globals.css` that `:focus-visible` is styled.
- `premium-button` class — visible hover/active states. Focus-visible styling not verified.
- Many icon buttons (e.g., `ProductCard.tsx:175-194` wishlist button) don't show explicit `focus:` classes — rely on browser default `:focus-visible` outline, which Tailwind's preflight may reset.

Priority: P2.

Suggested fix: Add a global `:focus-visible` rule in `globals.css` that draws a 2px gold ring on every interactive element.

H.3 Dropdown accessibility

- **PublicHeader** cart/wishlist/bell/account buttons: have `aria-expanded` ✓ (lines 1079, 1095, 1113, 1138, 1177). Do NOT have `aria-controls` pointing to the dropdown's id. [VERIFIED] ✗ Minor.
- **AdminTopBar** notifications/profile buttons: have `aria-expanded` ✓ (lines 201, 275). No `aria-controls`. ✗ Minor.
- **Mobile menu button:** has `aria-expanded` ✓ (line 1177). No `aria-controls`. ✗

Priority: P2.

Suggested fix: Add `aria-controls={panelId}` and `id={panelId}` to link button → panel.

H.4 Modal/drawer accessibility

[VERIFIED] by grep — `role="dialog"` and `aria-modal="true"` presence:

Component	<code>role="dialog"</code>	<code>aria-modal</code>	Escape	Focus trap
ProductDrawer	✓ (line 471)	✓	✗	✗
ProductCreateDrawer	not verified	not verified	✗	✗
OrderDrawer	✓ (line 105)	✓	✗	✗
CustomerDrawer	✗ (line 264)	✗	✗	✗
AnnouncementsPanel modal	✓ (line 112)	✓	✗	✗
PromoCodesPanel modal	✓ (line 238)	✓	✗	✗
PackagingInventoryPanel modals (×2)	✓ (lines 189, 423)	✓	✗	✗
AddExpenseDrawer (accounting)	not verified	not verified	✗	✗
PublicHeader MobileMenu	✗ (line 365)	✗	✓ (global)	✗
PublicHeader CommercePopover	✗ (line 536)	✗	✓ (global)	✗

CustomerDrawer and the public **MobileMenu / CommercePopover** are missing `role="dialog"` and `aria-modal="true"` entirely.

Priority: P1.

H.5 Form labels and errors

- **Auth forms** (`/auth/login`, `/auth/signup`): `<label>` wraps `<input>` with visible text ✓. Error messages shown in red boxes (line 137-141 of login page). ✓
- **Checkout form:** uses `<AddressSection>`, `<PaymentSection>` with `<Field>` wrappers containing `<label>` + `<input>`. Errors displayed inline. ✓
- **Account address form:** similar pattern — would need full read to verify.
- **Admin forms** (settings, product drawer): use `<Field>` wrappers with `<label>`. ✓
- **Search inputs** (`/products`): `<input type="search">` without a visible `<label>` — relies on `placeholder`. **Missing `aria-label`.** [VERIFIED] ✗ (line 308-314 of `products/page.tsx`)
- **Admin orders search** (`/admin/orders` line 277-283): same — `<input type="search">` with `placeholder`, no `aria-label`. ✗

Priority: P2.

Suggested fix: Add `aria-label={t({ en: "Search products", ar: "جتنم نغ شحبا" })}` to all search inputs.

H.6 Alt text coverage

- **Decorative bg images:** `alt=""` ✓ (e.g., `HeroSection.tsx:136`, `BestSellersSection.tsx:94`, `CategoriesSection.tsx:30`).
- **Product images on cards:** `alt={t(product.name)}` ✓ (`ProductCard.tsx:137`).
- **Product detail gallery main image:** `alt={productName}` ✓ (`products/[slug]/page.tsx:219`).
- **Product detail gallery thumbnails:** `alt=""` ✓ (line 245) — acceptable because the main image has the name.
- **Wishlist thumbnail:** `alt={product.name.en}` (line 133 of `account/wishlist/page.tsx`) — uses English name even in Arabic mode. Should use `t(product.name)`. [VERIFIED] ✗ Minor.
- **Logo:** `alt="Line Coffee"` ✓.
- **Admin sidebar logo:** `alt="Line Coffee"` ✓.

Priority: P3.

H.7 aria-labels on icon-only buttons

[VERIFIED] Good coverage:

- Cart quantity +/- buttons in `cart/page.tsx:109, 120` — `aria-label` ✓
- Product detail quantity +/- in `products/[slug]/page.tsx:319, 331` — `aria-label` ✓
- Wishlist heart in `ProductCard.tsx:177` — `aria-label` ✓
- Close buttons in drawers — `aria-label` ✓
- Admin topbar hamburger, bell, profile, language — `aria-label` ✓
- **Close button in `MobileMenu`** (line 380-387): `aria-label` ✓
- **Quick Add button on ProductCard** (line 199-214): no `aria-label` — but has visible text "Quick Add". ✓
- **Admin sidebar collapse arrow** (line 96-105 of `AdminSidebar.tsx`): `aria-label={t("Close sidebar")}` ✓
- ****/admin/orders/[id] Back button**** (line 142): `aria-label="Back"` ✓ but the string is hardcoded English, doesn't go through `t()`.

Priority: P3.

H.8 Color contrast

[INFERRED] — full WCAG audit would need a contrast checker. Spot check of the palette:

- `--pub-text: rgb(227 210 184 / 0.94)` on `--pub-bg: #0b0806` — contrast ratio ~12:1 ✓ AAA.

- `--pub-muted: rgb(214 187 159 / 0.88)` on `--pub-bg` — ~10:1 ✓ AAA.
- `--pub-faint: rgb(198 153 116 / 0.78)` on `--pub-bg` — ~6:1 ✓ AA.
- `text-[#B79B85]/55` (used in `ErrorScreen.tsx:36`) — `#B79B85` at 55% opacity on `#0B0806` ≈ 3:1 — **fails AA for normal text.** [INFERRED]
- `text-[#D6B79A]/38` (used in `cart/page.tsx:196`) — very faint, likely fails.
- `text-[#D6B79A]/65` (used in `cart/page.tsx:77` for "Clear all") — borderline.
- `text-[10px] text-[#B6885E]/80` (used in `MobileMenu` labels) — 10px is below the 12px minimum readable size.
- Admin muted text `--admin-muted: rgb(216 191 165 / 0.88)` on `--admin-bg: #0d0a07` — ~10:1 ✓.

Priority: P2.

Suggested fix: Audit all `text-*/35`, `text-*/38`, `text-*/45` opacity utilities. Raise faint text to ≥ 65% opacity or use a lighter color.

H.9 Reduced motion

[VERIFIED] `globals.css` lines 2105-2114:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    scroll-behavior: auto !important;
    transition-duration: 0.01ms !important;
  }
}
```

This globally zeroes out:

- The hero carousel `setInterval` (still fires, but transitions are instant) — the interval itself isn't disabled, so slides still rotate, just without the fade animation. [INFERRED]
- The hero count-up `setInterval` (still fires 36 frames at 33ms — state updates still happen, but `transition-duration` is 0.01ms so the final value snaps).
- Scroll reveal — `IntersectionObserver` still adds `is-visible` class, but the transition is instant. Effectively becomes a no-op reveal. ✓
- The `animate-bounce` on the scroll cue (`HeroSection.tsx:317`) — animation duration becomes 0.01ms, effectively frozen. ✓

Gap: the JavaScript intervals still run even with reduced motion. They consume CPU but produce no visible effect. Minor.

Priority: P3.

Suggested fix: Guard `setInterval` calls with `window.matchMedia("(prefers-reduced-motion: reduce)").matches` and skip the interval when reduced motion is requested.

H.10 RTL/LTR issues

- The `dir` attribute is correctly set on `<html>` server-side from cookie (`layout.tsx:154-165`) and updated client-side via `useEffect` in `LanguageProvider`. ✓
- Some components use `dir="ltr"` to force a direction (e.g., `HeroSection.tsx:232` for the prev/next arrow container, `products/[slug]/page.tsx:209` for stat numbers). Documented reasoning. ✓
- `formatDate` uses `Intl.DateTimeFormat` with the active language. ✓
- `MixedNumeric` splits numbers from symbols so Arabic display fonts render correctly. ✓
- **Inconsistency:** `AdminTopBar.tsx:80-125` notification labels use inline `language === "ar" ? "...": "..."` ternaries with hardcoded Arabic strings, mixed with `t("...")` calls. [VERIFIED]
- `**OrderStatusBadge.tsx**` uses `t(ADMIN_ORDER_STATUS_LABELS[status])` ✓ — relies on the admin MutationObserver.
- `**PublicHeader.tsx**` cart drawer correctly uses `t()` for all visible copy. ✓

H.11 Loading states

[VERIFIED]

- `src/app/loading.tsx` (root) — imports `LoadingScreen`. ✓
- `src/app/(public)/loading.tsx` — imports `LoadingScreen`. ✓
- `src/app/admin/loading.tsx` — imports `LoadingScreen` with custom label. ✓
- `**No loading.tsx in dynamic route segments:**` `/products/[slug]/`, `/products/category/[slug]/`, `/blog/[slug]/`, `/admin/orders/[id]/`, `/admin/products/[slug]/`, `/account/orders/[id]/`. These pages handle loading inline with their own `useState` + spinner. Acceptable but inconsistent.
- `LoadingScreen.tsx` is a server component (no `"use client"`) with `role="status"` and `aria-live="polite"` ✓.
- Inline loaders (e.g., `products/page.tsx:280-288`) show text-only "Loading products" — no spinner, no `role="status"`. [VERIFIED] ✗

Priority: P2.

Suggested fix: Add `role="status"` to inline loaders. Add `loading.tsx` to dynamic admin routes for instant feedback.

H.12 Empty states

[VERIFIED]

- **Cart empty:** `cart/page.tsx:37-61` — icon, heading, description, CTA. ✓
- **Checkout empty cart:** `CheckoutForm.tsx:637-655` — icon, heading, description, CTA. ✓
- **Orders empty:** `account/orders/page.tsx:58-69` — text + CTA. ✓ (no icon)
- **Wishlist empty:** `account/wishlist/page.tsx:65-77` — icon, heading, description, CTA. ✓ (and 3 more empty states for loading/error/visible-empty — over-engineered but thorough)
- **Notifications empty:** `account/notifications/page.tsx:71-83` — icon, heading, description. ✓

- **Products search empty:** `products/page.tsx:344-352` — heading + description. ✓
(no icon)
- **Admin orders empty:** `admin/orders/page.tsx:312-316` — uses `admin-empty-state` class with icon. ✓
- **Admin accounting empty:** inline `EmptyState` component (line 307-318). ✓

Priority: P3 (good coverage).

H.13 Error states

[VERIFIED]

- `src/app/error.tsx` — root error boundary, uses `ErrorScreen`. ✓
- `src/app/(public)/error.tsx` — public error boundary. ✓
- `src/app/admin/error.tsx` — admin error boundary, uses `ErrorScreen` with `homeHref="/admin/dashboard"`. ✓
- `src/app/global-error.tsx` — must exist (Next requirement). ✓
- `ErrorScreen.tsx` — self-contained (no context deps), bilingual, shows digest. ✓ But hardcoded strings (see H.10).

H.14 Failed checkout handling

[VERIFIED] `CheckoutForm.tsx:375-470` — `getCheckoutError(message)` maps 12+ specific server error strings to bilingual user messages:

- "Store is closed"
- "Promo code has expired" / "is inactive" / "is not active yet" / "promo minimum" / "your usage limit" / "usage limit" / "Promo code rejected"
- "Insufficient stock"
- "not available for purchase" / "is not available"
- "must total 100" (custom blend validation)
- "is missing its bean selection" / "Invalid espresso blend" / "Duplicate bean" / etc.
- "Invalid email" / "Invalid Google Maps URL"
- Fallback: "We could not place your order. Please try again."

Excellent coverage. ✓

H.15 Failed image handling

[VERIFIED] — ****NO `<Image>` instance in the entire codebase has an `onError` handler.****
Grep for `onError` returned zero matches in image components (the only `onError` matches were in form-validation code).

A deleted Supabase storage object, an expired URL, or a network failure will render the browser's broken-image icon. `ProductCard` has a placeholder for `product.image` being falsy, but if the URL is set and 404s at runtime, the placeholder doesn't kick in.

Priority: P1.

Suggested fix: Create `src/components/shared/SmartImage.tsx`:

```
function SmartImage(props: ImageProps) {
  const [failed, setFailed] = useState(false);
  if (failed) return <CinematicPlaceholder />;
  return <Image {...props} onError={() => setFailed(true)} />;
}
```

Replace all `<Image>` calls in product contexts.

H.16 Failed Supabase query handling

[VERIFIED]

- `public-catalog.ts` — every fetch function wraps `supabase.from(...)` and throws `PublicCatalogReadError` on error. Caller pages catch and set `catalogState: "error"`. ✓
- `admin-dashboard.ts`, `admin-orders.ts`, `admin-accounting.ts`, etc. — all define custom error classes (`AdminDashboardError`, `AdminOrdersError`, `AdminPurchasingError`, `AdminSettingsError`). ✓
- Pages render error states with retry buttons (e.g., `admin/dashboard/page.tsx:59-75`). ✓
- `useWishlist.ts:155-159` — catches server fetch failures and keeps the seed cache. ✓
- `useAuth.ts` — would need a closer read, but signup/login pages show error messages in red boxes.

Priority: P3 (good coverage).

H.17 Customer journey audit

Home → Products → Product Detail → Cart → Checkout → Order Success → WhatsApp → Account Orders → Notifications

1. **Home** (/) → `<LineCoffeeHome />`. Hero carousel, categories marquee, best sellers (live Supabase), story, journal, social gallery, contact. ✓
2. **Home → Products** via hero CTA "Shop Coffee" (`HeroSection.tsx:53` `primaryHref`). ✓ Also via header nav "Products" and via footer "categories" links.
3. **Products → Product Detail** via `<ProductCard>` → `/products/[slug]`. ✓
4. **Product Detail → Cart** via "Add to Cart" button (line 563-575 of `[slug]/page.tsx`). Opens cart drawer (`openCart()`). ✓
5. **Cart drawer → /cart** via "View Cart" link in `CommercePopover` (would need to verify in `PublicHeader.tsx`). **INFERRED** the drawer has a checkout link but the user may need to scroll to see it.
6. **/cart → /checkout** via "Proceed to Checkout" button (line 214-222 of `cart/page.tsx`). ✓
7. **/checkout → /order-success** via successful RPC submit (line 624-626 of `CheckoutForm.tsx`). ✓
8. **/order-success → WhatsApp** — auto-opens via `useEffect` (lines 68-83 of `order-success/page.tsx`). Mobile uses `window.location.assign`, desktop uses `window.open`. ✓ Once per session (sessionStorage guard).

9. **/order-success** → **/products** or **/** via "Continue Shopping" / "Back to Home" buttons. ✓
10. **/order-success** → **/account/orders/[id]** — **NO direct link**. **[INFERRED]** The customer is told "Our team will contact you" but not "View this order in your account". They have to navigate to **/account/orders** themselves.

Dead ends / confusing UX:

- **DEAD END (P0):** **/cart** shows delivery fee = `total >= 500 ? 0 : 50`. **/checkout** shows fee = `resolveDeliveryFee(governorate, area)` which can be 0, 30, 50, or 100. The customer sees different totals on **/cart** vs **/checkout**. They may abandon at checkout thinking the price went up. **[VERIFIED]**
- **DEAD END (P1):** **/order-success** doesn't link to **/account/orders**. Customer who wants to track the order must know to log in and visit account. **[INFERRED]**
- **DEAD END (P2):** **/products** sidebar has "Make Your Espresso" / "Make Your Flavor" as studio entries. Clicking opens a dynamically-imported studio with `ssr: false` and a "Loading studio..." fallback. If the studio fails to load (network error), there's no error state — just an infinite "Loading studio..." text. **[INFERRED]** from `products/page.tsx:19-25`.
- **DEAD END (P2):** Product detail page has NO "related products" or "you may also like" section. Customer finishes reading a product and has to manually navigate back. **[VERIFIED]**
- **DEAD END (P2):** Product detail page has NO breadcrumb back to the category — well, it does (line 187-194), but the breadcrumb goes to `/products?category=...` not `/products/category/...`. Inconsistent with the category page URL. **[VERIFIED]**
- **CONFUSING (P2):** Cart drawer (CommercePopover) and **/cart** page show different things. Drawer has quantity controls and remove buttons; **/cart** has the same but with a sticky summary. Not a dead end but two parallel UIs. **[VERIFIED]**
- **CONFUSING (P3):** Hero count-up animation re-runs on every home navigation (no sessionStorage guard like order-success has). Minor annoyance.
- **BROKEN HANDOFF (P2):** Wishlist in header drawer shows products from `getPublicProductsBySlugs(wishlistIds)`. If a wishlisted product is deleted from the catalog, it stays in the wishlist forever (no cleanup). **[INFERRED]** — `useWishlist.ts` only adds/removes by slug, no validation against current catalog.

H.18 Admin journey audit

Login → Dashboard → Products → Inventory → Orders → Accounting → Marketing → CMS → Settings

1. **/admin** → redirect to **/admin/dashboard** (`admin/page.tsx:3`). ✓
2. **Middleware** (`src/middleware.ts`) checks `line-auth` cookie. If absent, redirects to `/auth/login?next=...`. ✓
3. **AdminShell** (`AdminShell.tsx`) gates on `useCurrentAdmin()` status: `loading` → spinner, `signed_out` → redirect to login, `error` → retry screen, `forbidden` → access-denied screen, `authorized` → render shell. ✓ Excellent coverage.
4. **Login** (`/auth/login`) — `useAuth().signIn(email, password)` → `resolvePostLoginDestination()` → if admin, `/admin/dashboard`; else `/`. ✓

5. **Dashboard** → 4 KPI cards + 5 stat cards + SalesChart (dynamic) + QuickActions + LatestOrders + AlertsCenter + BestSellers + LatestReview. ✓
6. **Sidebar** has 12 nav items. ✓
7. **Topbar** has hamburger + page title + language toggle + notifications bell + profile dropdown. ✓
8. **Products** → tab "products" / "categories". Click product → **ProductDrawer** (right-side). Drawer has tabs (general/pricing/blend/inventory/images/etc). ✓
9. **Products** → **/admin/products/[slug]** — exists as a standalone page. Back button uses `router.back()`. **DEAD END if user landed directly via URL** (no breadcrumb to `/admin/products`). [VERIFIED] ✗
10. **Orders** → list of orders. Click → **OrderDrawer**. Drawer has "Open full order page" link → **/admin/orders/[id]**. ✓
11. **/admin/orders/[id]** — Back button uses `router.back()`. Has link to **/admin/orders** in the not-found state (line 129). ✓
12. **Inventory** → 6 tabs (products/beans/packaging/movements/lots/suppliers). No URL hash for active tab — switching tabs doesn't update the URL, so refresh resets to default tab. [INFERRED] ✗
13. **Accounting** → 6 tabs (overview/revenue/purchases/expenses/suppliers/activity). Same URL issue. [INFERRED] ✗
14. **Marketing** → 2 tabs (promos/announcements). ✓
15. **CMS** → 4 tabs (blog/reviews/legal/contact). Same URL issue. [INFERRED] ✗
16. **Settings** → 4 surfaces (brand/contact/social/storefront). ✓
17. **Profile dropdown** → "My Account" → **/account/profile** — **leaves the admin shell and enters the customer account shell**. Confusing handoff. The admin is now in the public site, with the public header/footer, and has to navigate back to **/admin** manually. [VERIFIED] ✗
18. **Profile dropdown** → "Website Preview" → opens **/** in a new tab. ✓
19. **Profile dropdown** → "Sign Out" → `signOut()` + redirect to **/auth/login**. ✓
20. **Notifications bell** → dropdown with order/low-stock alerts. Each alert links to **/admin/orders** or **/admin/inventory**. ✓

Dead ends / missing back-links / confusing modals:

- **DEAD END (P1):** **/admin/products/[slug]** has only `router.back()` — if user lands directly (via URL paste or bookmark), back goes to the browser's previous page (could be external). No breadcrumb to **/admin/products**. [VERIFIED] ✗
- **DEAD END (P1):** Admin drawers (Product, Order, Customer) don't close on Escape. User must click the X or the backdrop. [VERIFIED] ✗
- **DEAD END (P2):** No focus trap in any drawer — Tab can escape the drawer to the page behind the overlay. [VERIFIED] ✗
- **CONFUSING (P2):** "My Account" in the admin profile dropdown routes to the customer **/account/profile** page. An admin clicking this expects admin settings, not a customer account page. [VERIFIED] ✗
- **CONFUSING (P2):** Tab state in Inventory/Accounting/CMS/Analytics is not URL-synced. Refresh resets the tab. [INFERRED] ✗
- **MISSING (P2):** No global search in admin (e.g., search by order code, customer phone, product SKU from any page). Each page has its own search.

- **MISSING (P2):** No "Recent items" / "Quick jump" in the sidebar or topbar.
- **MISSING (P3):** No breadcrumbs in admin sub-pages (`/admin/orders/[id]`, `/admin/products/[slug]`).
- **MISSING (P3):** No `loading.tsx` in `/admin/orders/[id]` or `/admin/products/[slug]` — the inline `<Loader2 />` works but no `Suspense` fallback for streaming.

H.19 Accessibility summary — Top Priorities

P0 (correctness, ship now)

1. **Cart vs checkout delivery fee mismatch** — `src/app/(public)/cart/page.tsx:13` hardcodes `50/0`; `src/lib/delivery.ts` uses zones `0/30/50/100`. Customer sees two different totals.

P1 (high impact, schedule soon)

2. Convert `(public)/products/*` and `/` to server components — currently empty HTML shells for crawlers.
3. Eliminate the products→variants mini-waterfall in `src/lib/catalog/public-catalog.ts`.
4. Add `onError` fallback to every `<Image>` (no broken-image icons for deleted storage objects).
5. Add `Escape` key listener + focus trap to every admin drawer and the public `MobileMenu` / `CommercePopover`.
6. Add `role="dialog" + aria-modal="true"` to `CustomerDrawer`, `MobileMenu`, `CommercePopover`.
7. Centralize the 3 Supabase `createClient` sites into one factory per environment.
8. Add `aria-label` to search inputs in `/products` and `/admin/orders`.
9. Fix `/admin/products/[slug]` to have a breadcrumb back to `/admin/products` (don't rely on `router.back()`).
10. Add a link from `/order-success` to `/account/orders/[id]` so customers can find their order in the account.

P2 (quality, schedule next sprint)

11. Split `globals.css` (3034 lines) into per-route-group CSS files.
12. Delete dead code: `TestimonialsSection.tsx`, `src/components/layout/dashboard/*` placeholders.
13. Extract shared `<AdminDrawer>`, `<Field>`, `<Surface>`, `<StatusPill>` UI primitives.
14. Extract shared `formatMoney`, `formatDate`, `slugify`, `margin` utilities.
15. Split the 5 files >1000 lines (`admin/accounting/page.tsx`, `admin/cms/page.tsx`, `admin/products/page.tsx`, `EspressoBlendStudio.tsx`, `PublicHeader.tsx`) into per-tab/per-section subcomponents.
16. Move admin features into `src/features/admin/{catalog,orders,...}/`.
17. Add URL-synced tab state for Inventory/Accounting/CMS/Analytics.
18. Replace inline `dir === "rtl"` conditionals with Tailwind logical properties.

19. Fix the "My Account" admin profile link — route to `/admin/settings` instead of `/account/profile`.
20. Try Turbopack for dev (remove `--webpack` flag).
21. Audit faint text colors (`/35`, `/38`, `/45` opacities) for WCAG AA.
22. Add `loading.tsx` to `/admin/orders/[id]` and `/admin/products/[slug]`.

P3 (polish)

23. Add `aria-controls` to dropdown buttons.
 24. Guard hero `setInterval` animations with `prefers-reduced-motion` check.
 25. Use `t(product.name)` for wishlist thumbnail alt text.
 26. Run `knip` to find more dead code.
 27. Add Playwright E2E for customer + admin journeys.
 28. Add Lighthouse CI budgets.
 29. Add `axe-playwright` to CI.
-

Part I — Architecture / Refactor Plan

I.1 Folder structure map

```
src/
├── app/
│   ├── (public)/                ← route group, no URL segment
│   │   ├── (layout.tsx, loading.tsx, error.tsx)
│   │   ├── products/           ← products/, products/[slug]/, products/category/[slug]/
│   │   ├── cart/, checkout/, order-success/
│   │   ├── account/            ← profile, orders, orders/[id], addresses, wishlist, notifications
│   │   ├── auth/               ← login, signup, forgot-password, reset-password
│   │   ├── about/, blog/, contact/, privacy/, terms/, shipping/, returns/, reviews/
│   │   └── admin/              ← dashboard, orders, orders/[id], products, products/[slug],
│   │                                   espresso-manager, flavor-manager, inventory, customers,
│   │                                   marketing, accounting, analytics, cms, settings
│   ├── api/order-notifications/telegram/route.ts
│   ├── llms.txt/route.ts
│   ├── layout.tsx, page.tsx, loading.tsx, error.tsx, global-error.tsx
│   ├── robots.ts, sitemap.ts
│   └── globals.css
├── features/
│   └── website/
│       ├── home/               ← LineCoffeeHome + sections/ + hooks/
│       ├── checkout/           ← CheckoutForm + AddressSection/PaymentSection/PromoSection/Orders
│       ├── make-your-espresso/ ← EspressoBlendStudio + data/ + lib/
│       └── make-your-flavor/    ← FlavorMixStudio + data/ + lib/
├── components/
│   ├── layout/
│   │   ├── auth/ (AuthCard)
│   │   ├── account/ (AccountShell)
│   │   ├── public/ (PublicHeader 1205 lines, PublicFooter)
│   │   └── dashboard/ (DashboardShell + placeholders – UNUSED)
│   ├── product/ (ProductCard, CatalogProductCard)
│   ├── ui/ (LoadingScreen, SectionHeading, LegalPageLayout)
│   ├── shared/ (MixedNumeric)
│   ├── icons/ (SocialIcons, WhatsAppIcon)
│   ├── error/ (ErrorScreen)
│   └── admin/
│       ├── layout/ (AdminShell, AdminSidebar, AdminTopBar, AdminLanguageProvider)
│       ├── products/ (ProductDrawer, ProductCreateDrawer)
│       ├── customers/ (CustomerDrawer)
│       ├── marketing/ (PromoCodesPanel, AnnouncementsPanel)
│       ├── dashboard/ (SalesChart, KPICard, WelcomeHero, LatestOrders, AlertsCenter, ...12 files)
│       ├── orders/ (OrderDrawer, OrderDetails, OrderFinancePanel, OrderStatusBadge)
│       ├── inventory/ (PackagingInventoryPanel)
│       └── shared/ (AdminPlaceholder)
├── lib/
│   ├── admin/ (19 files: admin-catalog, admin-orders, admin-il8n, admin-dashboard, admin-analyti
│   │           admin-accounting, admin-purchasing, admin-inventory, admin-packaging, admin-flav
│   │           admin-espresso, admin-cms, admin-marketing, admin-announcements, admin-product-i
│   │           admin-customers, admin-metrics, admin-settings, admin-il8n)
│   ├── auth/ (admin.ts)
│   ├── catalog/ (public-catalog.ts, catalog-mappers.ts)
│   ├── checkout/ (governorates.ts)
│   ├── cms/ (public-cms.ts, public-blog.ts)
│   ├── content/ (announcements.ts)
│   ├── context/ (language.tsx, cart.tsx)
│   └── hooks/ (useAuth, useWishlist, useCurrentAdmin,
```

... [truncated] ...

I.2 Files > 500 lines

[**VERIFIED**] — from `wc -l` over all `.ts/.tsx` files:

Lines	File
2491	src/app/admin/accounting/page.tsx
1613	src/app/admin/cms/page.tsx
1333	src/app/admin/products/page.tsx
1235	src/features/website/make-your-espresso/EspressoBlendStudio.tsx
1205	src/components/layout/public/PublicHeader.tsx
1170	src/components/admin/products/ProductDrawer.tsx
1159	src/lib/admin/admin-catalog.ts
1112	src/app/admin/analytics/page.tsx
1063	src/lib/admin/admin-orders.ts
969	src/lib/admin/admin-il8n.ts
914	src/components/admin/inventory/PackagingInventoryPanel.tsx
874	src/lib/admin/admin-purchasing.ts
806	src/app/admin/inventory/page.tsx
782	src/components/admin/marketing/PromoCodesPanel.tsx
762	src/features/website/make-your-flavor/FlavorMixStudio.tsx
751	src/components/admin/customers/CustomerDrawer.tsx
734	src/features/website/checkout/CheckoutForm.tsx
707	src/lib/admin/admin-dashboard.ts
707	src/lib/admin/admin-analytics.ts
700	src/lib/admin/admin-accounting.ts
647	src/components/admin/orders/OrderFinancePanel.tsx
622	src/lib/admin/admin-customers.ts
613	src/app/admin/customers/page.tsx
599	src/app/(public)/products/[slug]/page.tsx
589	src/features/website/make-your-espresso/lib/espressoBlendEngine.ts

Lines	File
578	src/app/(public)/contact/page.tsx
575	src/app/(public)/account/addresses/page.tsx
540	src/lib/catalog/public-catalog.ts
527	src/components/admin/marketing/AnnouncementsPanel.tsx
519	src/app/(public)/products/category/[slug]/page.tsx

Total source: 51,661 lines of TS/TSX. 30 files over 500 lines. **5 files over 1000 lines.**

I.3 Duplicated logic

[VERIFIED] by grep:

Supabase client creation (3 sites):

- src/lib/supabase/client.ts:14
- src/lib/seo/data.ts:47
- src/app/api/order-notifications/telegram/route.ts:~80

All three pass `createClient(url, key, { auth: {...} })`. The auth config differs slightly between browser (`persistSession: true`) and server (`persistSession: false`) — should be centralized.

Price/money formatters (5 copies):

- src/components/admin/customers/CustomerDrawer.tsx:42 — `fmt(n)`
- src/lib/admin/admin-dashboard.ts:166 — `fmtInt(value)`
- src/app/admin/analytics/page.tsx:129 — `fmt(value) + money(value) + pct(value)`
- src/app/admin/accounting/page.tsx:106 — `fmt(value) + money(value) + signedMoney(value) + pct(value) + shortDate(value)`
- src/app/admin/customers/page.tsx:30 — `fmt(n)` (one-liner)

All use `Intl.NumberFormat("en-US", { maximumFractionDigits: 2 })` or `.toLocaleString()`. Should be one shared `src/lib/utils/formatMoney.ts`.

Date formatters (5+ copies):

- src/lib/utils/formatDate.ts:1 — canonical `formatDate(dateStr, lang)`
- src/app/admin/orders/page.tsx:38 — local `formatDate` using `toLocaleString("en-EG")`
- src/components/admin/inventory/PackagingInventoryPanel.tsx:89 — local `formatDate`
- src/components/admin/orders/OrderFinancePanel.tsx:41 — `formatDateTime`
- src/components/admin/orders/OrderDetails.tsx:16 — `formatDateTime`
- src/components/admin/customers/CustomerDrawer.tsx:23 — `formatDate`

Slugify (3 copies):

- `src/app/admin/cms/page.tsx:104` — `slugify(value)`
- `src/app/admin/products/page.tsx:97` — `slugifyCategoryName(value)`
- `src/components/admin/products/ProductCreateDrawer.tsx:13` — `slugify(value)`

All three implement the same NFKD-normalize + lowercase + replace non-alphanumerics with `-` pattern. Should be `src/lib/utils/slugify.ts`.

Margin calculator (2 copies):

- `src/app/admin/products/[slug]/page.tsx:18` — `margin(salePricePerKg, costPerKg)`
- `src/components/admin/products/ProductDrawer.tsx:118` — `marginPct(sale, cost)`

Same formula $(\text{sale} - \text{cost}) / \text{sale} * 100$. Should be `src/lib/admin/margin.ts`.

Image URL marker (2 copies):

- `src/components/product/ProductCard.tsx:24` — `const PRODUCT_IMAGE_STORAGE_MARKER = "/storage/v1/object/public/product-images/"`
- `src/components/product/CatalogProductCard.tsx:15` — same constant verbatim

Already centralized in `src/lib/admin/admin-product-images.ts:70` — but the two card components don't import from there. Should reuse.

Governorates lookup (2 sites):

- `src/features/website/checkout/CheckoutForm.tsx:38`
- `src/app/(public)/account/addresses/page.tsx:15`

Both import `EGYPT_GOVNORATES` as `GOVS` from `src/lib/checkout/governorates.ts` — correctly centralized. Good.

Delivery fee logic (2 sites, NOT matching):

- `src/app/(public)/cart/page.tsx:13` — hardcoded total $\geq 500 ? 0 : 50$
- `src/lib/delivery.ts:44` — `resolveDeliveryFee(governorate, area)` zone-based (0/30/50/100)

Honesty boundary pattern: `src/lib/delivery.ts:6-10` documents this clearly: "this is a DISPLAY mirror of the authoritative SQL function". The cart page doesn't even use the display mirror.

I.4 Duplicated UI patterns

[VERIFIED] by grep:

Drawer/modal markup — 7+ sites use the same `<div className="admin-modal-overlay fixed inset-0" /> + <aside className="admin-drawer-surface fixed right-0 top-0">` pattern:

- `ProductDrawer.tsx:457-484`
- `OrderDrawer.tsx:91-110`

- `CustomerDrawer.tsx:260-274`
- `AnnouncementsPanel.tsx:108-113` (modal centered, not drawer)
- `PromoCodesPanel.tsx:234-239` (modal)
- `PackagingInventoryPanel.tsx:185-190, 419-424` (two modals)
- `AddExpenseDrawer` in `admin/accounting/page.tsx:389-391`

Should be one shared `<AdminDrawer open onClose>` and `<AdminModal open onClose>` component.

Field wrapper — 5+ copies of a `<Field label children>` component:

- `src/app/admin/accounting/page.tsx:172-181`
- `src/app/admin/products/[slug]/page.tsx:23-32`
- `src/app/admin/settings/page.tsx:60-90`
- `src/components/admin/marketing/AnnouncementsPanel.tsx:27-45`

KPI/status pill — 4+ copies of `<StatusPill label tone>`:

- `src/app/admin/accounting/page.tsx:185-195`
- `src/app/admin/orders/page.tsx:54-92` (KpiCard)
- `src/components/admin/dashboard/KPICard.tsx`
- `src/components/admin/dashboard/LowStockCard.tsx`
- `src/components/admin/orders/OrderStatusBadge.tsx` (the canonical one — but other places reinvent it)

Surface/Panel wrapper — 3+ copies:

- `src/app/admin/accounting/page.tsx:197-248` (Surface)
- `src/app/admin/inventory/page.tsx:55-84` (Panel)
- `src/app/admin/settings/page.tsx:29-58` (Surface)

I.5 Route ownership clarity

[VERIFIED] The `src/features/` folder is **underused**:

- Only contains `website/` subfolder with 4 features (home, checkout, make-your-espresso, make-your-flavor).
- **No features/admin** folder. All admin features live in `src/app/admin/*` (route-adjacent) and `src/components/admin/*` (UI) and `src/lib/admin/*` (data). Three homes per admin feature.
- **No features/account, features/auth, features/catalog, features/cart** folders. These features' UIs live in `src/app/(public)/*` directly, with shared UI in `src/components/layout/{auth,account,public}/*` and state in `src/lib/context/*`.

Recommendation: Move admin features into `src/features/admin/{catalog,orders,inventory,customers,marketing,accounting,analytics,cms,settings}/`. Each feature folder owns its page, drawer(s), and `lib/` data layer. Reduce `src/app/admin/` to thin route entries.

I.6 Naming consistency

[VERIFIED] Inconsistencies:

- **Admin layout:** `AdminSidebar`, `AdminTopBar`, `AdminShell`, `AdminLanguageProvider` — `Admin` prefix.
- **Admin dashboard components:** `SalesChart`, `KPICard`, `LatestOrders`, `AlertsCenter`, `WelcomeHero`, `BestSellersMonth` — no `Admin` prefix. Mixed.
- **Admin drawers:** `ProductDrawer`, `OrderDrawer`, `CustomerDrawer` — no `Admin` prefix, but live in `src/components/admin/`.
- **Public layout:** `PublicHeader`, `PublicFooter` — `Public` prefix.
- **Public auth/account:** `AuthCard`, `AccountShell` — feature-name prefix, not `Public` prefix.
- **Product cards:** `ProductCard` (used on home + `/products`) vs `CatalogProductCard` (used on `/products/category/[slug]`). Both exist with very similar logic.
- **Index files:** `src/components/layout/dashboard/index.ts` exists but `src/lib/types/index.ts` also exists. Inconsistent use of barrel files.

Recommendation: Pick one convention (e.g., no prefix — rely on folder path for namespacing) and apply across all components.

I.7 RTL / LTR handling

[VERIFIED]

- `src/lib/context/language.tsx:47-52` — `useEffect` sets `document.documentElement.lang` and `dir` on every language change. **Client-side only** — initial server render sets `lang/dir` from cookie in `src/app/layout.tsx:154-165`. Good — no flash.
- `src/components/admin/layout/AdminLanguageProvider.tsx:223-229` — sets `lang` and `dir` on a wrapping `<div>` for the admin shell.
- `AdminLanguageProvider` uses a **DOM MutationObserver + TreeWalker** to translate every text node and attribute (placeholder/aria-label/title) on the fly (lines 102-130, 160-198). This is a heavy approach — every DOM mutation in the admin tree triggers re-translation. [VERIFIED]
- Many components use hardcoded `dir === "rtl" ? "rotate-180" : ""` or `dir === "rtl" ? "left-0" : "right-0"` conditionals (e.g., `HeroSection.tsx:177`, `PublicHeader.tsx:219-220`, `MobileMenu` line 365). Inconsistent with Tailwind's logical properties (`ps-*`, `pe-*`, `ms-*`, `me-*`, `start-*`, `end-*`) which would handle RTL automatically.
- `MixedNumeric.tsx` correctly splits numbers from symbols so Arabic display fonts don't fall back to Latin glyphs. Good.

I.8 Arabic/English text duplication

[VERIFIED]

- **src/components/error/ErrorMessage.tsx** — hardcoded bilingual strings (lines 30-38): English heading "Something went wrong", then a separate `<p dir="rtl">شرح أطخ عقوق وتم ريغ أطخ`, then English body. Not using the `t()` function from language context. Inconsistent with the rest of the app.
- **src/lib/admin/admin-i18n.ts** (970 lines) — a giant `ARABIC_TRANSLATIONS` dictionary mapping English → Arabic, used by the MutationObserver in `AdminLanguageProvider`. This is unusual: the admin UI is authored in English and translated at runtime via DOM walking. The public site uses `{ en, ar }` objects passed through `t()`. Two completely different i18n strategies coexist.
- **src/app/admin/layout/AdminTopBar.tsx** (lines 80-125) — has inline `language === "ar" ? "...": "...` ternaries for notification copy. Mixed with `t()` calls. Inconsistent.
- **src/app/admin/accounting/page.tsx** (lines 80-87) — `STATUS_LABEL: Record<OrderStatus, string>` is English-only, relies on the MutationObserver to translate.
- Hardcoded Arabic strings mixed with English in `AdminTopBar.tsx:80-125` — same file has both `t("...")` calls and raw Arabic string literals.

I.9 Phased refactor plan (do NOT implement — proposal only)

Phased Refactor Roadmap — 9 Phases

Recommended implementation order. Each phase unblocks the next. Total: ~9 sprints.



Figure 6 — 9-phase refactor roadmap from launch blockers to testing automation.

Phase A — Docs consolidation

- Move `LINE_COFFEE_V3_*.md` (8 root-level files) and `docs/ai/*.md` (6 files) and `docs/archive/*.md` + `DESIGN_SYSTEM_FOUNDATION.md` + `AI_HANDOFF_MARKETING.md` into a single `docs/` tree: `docs/architecture/`, `docs/decisions/`, `docs/operations/`, `docs/archive/`.

- Consolidate `CLAUDE.md`, `AGENT_WORK_PROTOCOL.md`, `PRODUCT.md`, `README.md` into one `README.md` + `docs/contributing/`.
- Estimated: -10 root-level files, -20% docs navigation cost.

Phase B — Old docs/archive cleanup

- Audit `docs/archive/LINE_COFFEE_V3_PRODUCTS_PHASE_READINESS_AUDIT.md` — if its findings are resolved, move to `docs/archive/resolved/`.
- Audit `LINE_COFFEE_V3_PROJECT_LOG.md` — likely a chronological log; convert to `CHANGELOG.md` or archive.

Phase C — Mock/static data decisions

- `src/lib/mock-data/visual-content.ts` (386 lines) — holds `heroSlides`, `heroStats`, `visualCategories`, `contactItems`, etc. Used by `HeroSection`, `CategoriesSection`, `ContactSection`, `BestSellersSection` (partially).
- **Decision needed:** is this content editable via CMS, or hardcoded? Currently CMS (`src/lib/admin/admin-cms.ts`) handles blog/reviews/legal/contact-messages, but NOT hero slides or category hero copy. Either:
 - (a) Move hero content into `site_settings` and make it admin-editable, or
 - (b) Document that hero is intentionally hardcoded brand content.
- Same decision for `categoryExperiences` in `src/app/(public)/products/category/[slug]/page.tsx:38-127` (90 lines of per-category Arabic+English marketing copy hardcoded in the page file).

Phase D — Security fixes

- **Middleware is a UX redirect, not a security boundary** (`src/middleware.ts:5-18` documents this honestly). The `line-auth` cookie is spoofable. RLS is the real gate. **Action:** ensure every admin RPC and table has `is_admin()` RLS — spot-check 3-5 migrations.
- **3 Supabase client creations** — consolidate to ensure no service-role key leak into client bundle. (Currently no service role is used in the browser — verified by grep — but a future feature could accidentally introduce one.)
- ****AccountOwnerBoundary.tsx** and **CheckoutOwnerBoundary.tsx** — verify they actually prevent IDOR for `/account/orders/[id]` (the ID is the order code, which is sequential like `LC-000001` — enumerable).
- Rate-limit `/api/order-notifications/telegram` and `/checkout` RPC (no rate limiting visible).

Phase E — SEO/AI search upgrades

- Convert client-component catalog pages to RSC (see G.9) so crawlers see real content.
- Add `Image` structured data to product pages (currently only `Product` + `BreadcrumbList` JSON-LD in `src/app/(public)/products/[slug]/layout.tsx`).
- Add `FAQPage` JSON-LD to `/contact` page (it has hardcoded FAQ items at `src/app/(public)/contact/page.tsx:23-66`).
- Add `Article` JSON-LD to blog post pages (verify in `src/app/(public)/blog/[slug]/layout.tsx` — likely already there).

- Add `hreflang` alternates for `ar` and `en` in metadata.
- Submit `llms.txt` to AI search consoles (already exists at `src/app/llms.txt/route.ts` — good).
- Add `og:image` per-product (currently uses default OG image — see `src/lib/seo/site.ts`).

Phase F — Performance refactor

- Phase F.1: Convert `/products`, `/products/[slug]`, `/products/category/[slug]`, `/` to RSC.
- Phase F.2: Eliminate the products→variants mini-waterfall using Supabase's embedded select or a Postgres function.
- Phase F.3: Centralize Supabase client factories.
- Phase F.4: Add `onError` fallback to `<Image>` everywhere.
- Phase F.5: Split `globals.css` into route-group CSS files.
- Phase F.6: Memoize checkout form sections.
- Phase F.7: Try Turbopack for dev.
- Phase F.8: Add `loading.tsx` to dynamic admin routes (`/admin/orders/[id]`, `/admin/products/[slug]`).

Phase G — Architecture cleanup

- Phase G.1: Move admin features into `src/features/admin/{catalog,orders,...}/`.
- Phase G.2: Extract shared `<AdminDrawer>`, `<AdminModal>`, `<Field>`, `<Surface>`, `<StatusPill>` into `src/components/admin/ui/`.
- Phase G.3: Extract shared `formatMoney`, `formatDate`, `slugify`, `margin` into `src/lib/utils/`.
- Phase G.4: Decide between `ProductCard` vs `CatalogProductCard` — merge or clearly differentiate.
- Phase G.5: Delete dead code: `TestimonialsSection.tsx`, `src/components/layout/dashboard/*` placeholders, `src/lib/mock-data/product-catalog.ts` (verify usage).
- Phase G.6: Unify `i18n` — either port the admin `MutationObserver` approach to the public site, or (better) port the admin UI to use `{ en, ar }` objects + `t()`.
- Phase G.7: Replace inline `dir === "rtl"` conditionals with Tailwind logical properties.
- Phase G.8: Split files > 1000 lines: `admin/accounting/page.tsx` (2491) into per-tab components, `admin/cms/page.tsx` (1613) into per-tab components, `PublicHeader.tsx` (1205) into `PublicHeader/index.tsx` + `MobileMenu.tsx` + `CommercePopover.tsx` + `NotificationsDropdown.tsx` + `UserMenu.tsx`.

Phase H — Testing/QA automation

- Phase H.1: Add Playwright E2E for the customer journey (Home → Products → Product → Cart → Checkout → Order Success → WhatsApp). `playwright` is already in devDeps.
- Phase H.2: Add Playwright E2E for the admin journey (Login → Dashboard → each admin route → drawer interactions).

- Phase H.3: Add a cart-vs-checkout delivery-fee equality test (would have caught the G.1 bug).
 - Phase H.4: Add a Lighthouse CI workflow ([.github/workflows/lighthouse.yml](#)) with budgets for LCP, CLS, TBT.
 - Phase H.5: Add [knip](#) to CI to catch dead code (would have caught [TestimonialsSection](#)).
 - Phase H.6: Add a visual regression test (Playwright screenshot diffs) for the home page and the 8 admin tabs.
 - Phase H.7: Add an accessibility audit step (axe-playwright) to CI.
-

Part J — Final Master Guide Proposal

J.1 Proposed document

`docs/LINE_COFFEE_V3_MASTER_SYSTEM_GUIDE.md`

This is the single document that should be created later to consolidate the system into one canonical reference. It should NOT duplicate the live audit (which lives in chat / can be exported to PDF). Instead it should be the **operating manual** for the owner and any future developer.

J.2 Detailed Table of Contents

1. BRAND IDENTITY
 - 1.1 Brand name (English / Arabic)
 - 1.2 Founding year + story
 - 1.3 Visual identity (colors, typography, logo variants, brand assets)
 - 1.4 Voice & tone (English / Arabic)
 - 1.5 Service area (Egypt, 27 governorates)
2. TECH STACK
 - 2.1 Framework (Next.js 16.2.9, React 19.2.4)
 - 2.2 Language (TypeScript)
 - 2.3 Styling (Tailwind CSS 4, CSS-based config)
 - 2.4 Backend (Supabase – Postgres + Auth + Storage)
 - 2.5 Charts (Recharts 3.8.1, code-split for admin only)
 - 2.6 Icons (lucide-react)
 - 2.7 Animation (custom IntersectionObserver – no library)
 - 2.8 Fonts (Playfair Display + Aligarh Arabic self-hosted; Cairo + Tajawal via Google)
 - 2.9 Build tooling (webpack for both dev + build – Turbopack opt-out)
3. SETUP / RUN
 - 3.1 Prerequisites (Node version, npm/pnpm)
 - 3.2 Clone + install
 - 3.3 Env variables setup (copy .env.example, fill 7 vars)
 - 3.4 Supabase project setup (apply migrations in order)
 - 3.5 Seed catalog (run scripts/generate-catalog-seed.mjs)
 - 3.6 Dev server (npm run dev – note webpack)
 - 3.7 Production build (npm run build + npm start)
 - 3.8 Storage bucket creation (product-images)
4. ENV VARIABLES
 - 4.1 Public (NEXT_PUBLIC_*)
 - NEXT_PUBLIC_SUPABASE_URL
 - NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY (preferred)
 - NEXT_PUBLIC_SUPABASE_ANON_KEY (fallback)
 - NEXT_PUBLIC_SITE_URL
 - NEXT_PUBLIC_BASE_URL (fallback)
 - NEXT_PUBLIC_WHATSAPP_PHONE
 - 4.2 Server-only
 - TELEGRAM_BOT_TOKEN
 - TELEGRAM_CHAT_ID
 - 4.3 Supabase config-only (supabase/config.toml – not in app code)
 - 4.4 Why no service-role key in app code
5. SUPABASE TABLES

- 5.1 Catalog: categories, products, product_variants
 - 5.2 Customers: customers, customer_addresses, customer_wishlist
 - 5.3 Orders: orders, order_items, order_status_events
 - 5.4 Inventory: inventory_stock, inventory_lots, inventory_movements, order_lot_allocations
 - 5.5 Espresso builder: espresso_beans, espresso_bean_stock, espresso_bean_lots, espresso_bean_m
 - 5.6 Flavor builder: flavor_bases, flavor_items
 - 5.7 Packaging: packaging_items, packaging_lots, packaging_movements, order_packaging_lines, or
 - 5.8 Payments: order_payments, order_refunds, order_returns, order_return_items
 - 5.9 Purchasing: suppliers, purchases, purchase_items, supplier_payments, expenses
 - 5.10 Marketing: promo_codes, promo_redemptions, announcements
 - 5.11 CMS: blog_posts, reviews, legal_pages, contact_messages
 - 5.12 Operations: site_settings, order_notifications, admin_users
 - 5.13 Public views: public_categories, public_products, public_product_variants
 - 5.14 Disabled RLS: customer_wishlist (access via SECURITY DEFINER RPCs only)
 - 5.15 Migration order reference (38 migrations, chronological)
6. PUBLIC WEBSITE MAP
- 6.1 / (home – 8 sections)
 - 6.2 /products (catalog + studio sidebar)
 - 6.3 /products/[slug] (detail + JSON-LD)
 - 6.4 /products/category/[slu
 - ... [truncated] ...

J.3 Recommended doc consolidation (do NOT change yet — proposal only)

Action	Doc	New location
MERGE	README.md + CLAUDE.md + AGENT_WORK_PROTOCOL.md + PRODUCT.md	One README.md (operational) + docs/contributing/AGENT_PROTOCOL.md
KEEP AS CANONICAL	docs/ai/LINE_COFFEE_V3_CURRENT_STATE.md	docs/architecture/CURRENT_STATE.md
KEEP AS CANONICAL	docs/ai/LINE_COFFEE_V3_MASTER_EXECUTION_PLAN.md	docs/architecture/MASTER_EXECUTION_PLAN.md
KEEP AS REFERENCE	docs/ai/LINE_COFFEE_V3_CONTENT_MAP.md	docs/architecture/CONTENT_MAP.md
KEEP AS REFERENCE	docs/ai/LINE_COFFEE_V3_DATA_CONTRACTS_AND_MIGRATIONS.md	docs/architecture/DATA_CONTRACTS.md
KEEP AS REFERENCE	docs/DESIGN_SYSTEM_FOUNDATION.md	docs/design/DESIGN_SYSTEM.md
KEEP AS REFERENCE	docs/AI_HANDOFF_MARKETING.md	docs/operations/AI_HANDOFF.md
ARCHIVE	LINE_COFFEE_V3_PUBLIC_WEBSITE_MASTER_VISUAL_PLAN.md	docs/archive/PUBLIC_WEBSITE_VISUAL_PLAN.md
ARCHIVE	LINE_COFFEE_V3_CUSTOM_BUILDERS_VISUAL_BLUEPRINT.md	docs/archive/CUSTOM_BUILDERS_BLUEPRINT.md
ARCHIVE	LINE_COFFEE_V3_CUSTOM_BUILDERS_REVIEW_AND_ENHANCEMENTS.md	docs/archive/CUSTOM_BUILDERS_REVIEW.md
ARCHIVE	docs/ai/LINE_COFFEE_V3_FINAL_DECISIONS_AND_ROADMAP.md	docs/archive/FINAL_DECISIONS_AND_ROADMAP.md
ARCHIVE	docs/ai/LINE_COFFEE_V3_SYSTEM_AUDIT.md	docs/archive/SYSTEM_AUDIT_2026_06_23.md

Action	Doc	New location
ARCHIVE or KEEP as deep reference	docs/ai/LINE_COFFEE_V3_OPERATING_MODEL_BLUEPRINT.md	docs/archive/OPERATING_MODEL_BLUEPRINT.md (with "OUTDATED" banner)
ARCHIVE (already there)	docs/archive/LINE_COFFEE_V3_PRODUCTS_PHASE_READINESS_AUDIT.md	Keep
CONVERT	LINE_COFFEE_V3_PROJECT_LOG.md	CHANGELOG.md or docs/operations/PROJECT_LOG.md
CREATE NEW	(this proposal)	docs/LINE_COFFEE_V3_MASTER_SYSTEM_GUIDE.md

Final Summary

What I inspected

- Cloned <https://github.com/muhmiied/line-coffee-fainal.git> (read-only) to `/home/z/my-project/audit/line-coffee-v3`
- All 38 SQL migrations in `supabase/migrations/*.sql` (~14k lines)
- `supabase/seeds/20260625_catalog_seed.sql` + `scripts/generate-catalog-seed.mjs`
- `supabase/config.toml`, `next.config.ts`, `package.json`, `tsconfig.json`
- All public routes under `src/app/(public)/**/*.page.tsx` + their `layout.tsx`
- All admin routes under `src/app/admin/**/*.page.tsx`
- All service files under `src/lib/admin/*.ts` (19 files)
- `src/lib/{checkout,delivery,catalog,cms,content,seo,auth,hooks,context,supabase,validation,utils}/**`
- `src/features/website/{home,checkout,make-your-espresso,make-your-flavor}/**`
- `src/components/{layout,product,admin,ui,shared,icons,error}/**`
- `src/middleware.ts`, `src/app/api/**`
- All `.md` docs at root + `docs/**` + `docs/ai/**` + `docs/archive/**`
- `src/lib/mock-data/**`, builder data files, engine files

What I found

- **Architecture is unusually careful for a Supabase browser-only app.** Trust boundary is the database, enforced via SECURITY DEFINER RPCs that re-validate every authoritative number server-side.
- **No launch-blocking critical security issues** — but SMTP is not configured (F1), which breaks signup/password-reset in production.
- **3 critical data-flow disconnects (P1):** espresso manager, flavor manager, and legal pages all persist to Supabase but the public site reads static files/arrays instead — admin edits don't propagate.
- **1 P0 correctness bug:** `/cart` delivery fee (`total >= 500 ? 0 : 50`) $\neq$ `/checkout` delivery fee (zone-based 0/30/50/100) — customer sees different totals.
- **Performance risk:** all public pages are client components $\rightarrow$ empty HTML shells for crawlers. SEO metadata is server-side (good), but body content is invisible without JS.
- **5 files over 1000 lines**, 30 files over 500 lines — maintenance hazard.
- **Honest codebase:** no fake ratings in JSON-LD, `trafficTrackingConnected: false` flag instead of fake traffic numbers, `QuickActions` documented as static navigation, no fake visitor counts.

Biggest risks

1. **(P0)** `/cart` vs `/checkout` delivery fee mismatch — customer abandonment
2. **(P1)** Espresso/flavor/legal disconnect — admin effort wasted, public site shows stale data
3. **(P1)** `/shipping` page contradicts actual `resolve_delivery_fee` logic + 27 governorates
4. **(P1)** Store-closed does not block checkout (verify migration applied)
5. **(P1)** Scan caps (5000 orders / 1000 customers / 250 order list) silently truncate — financial/dashboard accuracy degrades at scale
6. **(P1)** No LocalBusiness JSON-LD — no structured address/hours/geo for Google Business Profile
7. **(P1)** Client-only catalog pages → SEO body content invisible without JS
8. **(P1)** No image `onError` fallback → broken-image icons on deleted Storage objects
9. **(P1)** No Escape/focus-trap in admin drawers + missing `role="dialog"` on CustomerDrawer/MobileMenu/CommercePopover
10. **(Medium)** SMTP not configured → signup/password-reset broken in production
11. **(Medium)** No rate limit on Telegram endpoint + no captcha on signup
12. **(Medium)** No `.env.example` — operations onboarding risk

Biggest cleanup opportunities

1. **Delete dead code:** `visualProducts`, `visualTestimonials`, `DashboardShell` + placeholders, `AdminPlaceholder`, `TestimonialsSection` (verify)
2. **Archive 5-6 self-declared historical/superseded docs** to `docs/archive/`
3. **Extract shared utilities:** `formatMoney`, `formatDate`, `slugify`, `margin`, `PRODUCT_IMAGE_STORAGE_MARKER` — currently 2-5 copies each
4. **Extract shared UI:** `<AdminDrawer>`, `<AdminModal>`, `<Field>`, `<Surface>`, `<StatusPill>` — currently 3-7 copies each
5. **Split 5 files >1000 lines** into per-tab/per-section subcomponents
6. **Centralize 3 Supabase client factories** into one per environment
7. ****Update stale `site_settings.shipping` seed**** (free ≥ 500 / flat 50 model — not consumed)
8. **Unify i18n strategy** (public uses `{en,ar} + t()`; admin uses `MutationObserver` + dictionary — pick one)
9. ****Replace inline `dir === "rtl"` conditionals**** with Tailwind logical properties (`ps-*`, `pe-*`, `ms-*`, `me-*`)
10. **Move admin features** into `src/features/admin/{catalog,orders,...}/` for clear route ownership

Recommended next implementation order

Phase 1 — Launch blockers (do first)

1. Configure SMTP in `supabase/config.toml` (F1)

2. Confirm which migrations are applied to production Supabase (F12)
3. Fix `/cart` delivery fee to use `resolveDeliveryFee` or hide fee until checkout (P0)
4. Fix `/shipping` page content to match actual delivery logic + 27 governorates (P1)

Phase 2 — Critical disconnects

5. Wire espresso/flavor managers to public builders (refactor to DB reads OR sync script)
6. Wire `legal_pages` table to public legal pages (or document as known limitation)
7. Wire `/account/settings` toggles to a real persistence layer (or remove them)
8. Fix `/account/settings` language toggle to set the `line-coffee-language` cookie

Phase 3 — Security hardening

9. Add `.env.example`
10. Add rate limit to `/api/order-notifications/telegram`
11. Enable captcha on signup
12. Add CSP header
13. Replace sessionStorage PII stash on `/order-success` with RPC fetch

Phase 4 — SEO/AI search

14. Add `LocalBusiness/CafeOrCoffeeShop` JSON-LD
15. Add `FAQPage` JSON-LD on `/contact`
16. Fix `Product` JSON-LD `availability` to reflect real stock
17. Add hreflang alternates
18. Add `Organization.sameAs[]` + `email`
19. Convert catalog pages to RSC for crawler-visible body content
20. Add real `lastModified` to sitemap for products/categories

Phase 5 — Performance

21. Eliminate products→variants mini-waterfall
22. Centralize Supabase client factories
23. Add `onError` fallback to every `<Image>`
24. Split `globals.css` (3034 lines) into per-route-group files
25. Memoize checkout form sections
26. Try Turbopack for dev

Phase 6 — Accessibility

27. Add Escape key + focus trap to every admin drawer + public `MobileMenu/CommercePopover`
28. Add `role="dialog"` + `aria-modal` to `CustomerDrawer/MobileMenu/CommercePopover`
29. Add `aria-label` to search inputs
30. Add breadcrumb to `/admin/products/[slug]`
31. Add link from `/order-success` to `/account/orders/[id]`

32. Audit faint text colors for WCAG AA

Phase 7 — Architecture cleanup

- 33. Delete dead code (visualProducts, visualTestimonials, DashboardShell, AdminPlaceholder, TestimonialsSection)
- 34. Archive 5-6 superseded docs
- 35. Extract shared utilities + UI primitives
- 36. Split 5 files >1000 lines
- 37. Move admin features into `src/features/admin/`
- 38. Unify i18n strategy

Phase 8 — Documentation

- 39. Create `docs/LINE_COFFEE_V3_MASTER_SYSTEM_GUIDE.md` per J.2
- 40. Consolidate root-level docs
- 41. Archive historical docs

Phase 9 — Testing/QA

- 42. Add Playwright E2E for customer + admin journeys
- 43. Add cart-vs-checkout delivery-fee equality test
- 44. Add Lighthouse CI budgets
- 45. Add `knip` to CI for dead-code detection
- 46. Add `axe-playwright` accessibility audit to CI

Confirmation

- **☐ No files were changed** — read-only audit only. `git status` is clean.
- **☐ No commit, no push** — the local clone at `/home/z/my-project/audit/line-coffee-v3` was used only for inspection.
- **☐ No migrations touched** — all 38 SQL files were read but not applied.
- **☐ No code refactored** — all recommendations are proposals for later implementation.
- **☐ This audit file** (`LINE_COFFEE_V3_MASTER_AUDIT.md`) was created in `/home/z/my-project/download/` — outside the repository — at the user's explicit request. The repository itself remains untouched.

End of audit.